

الجمهورية الجزائرية الديمقراطية الشعبية

ⵜⴰⴳⴷⴰⵢⵜ ⵜⴰⵖⴻⵔⴰⵢⵜ ⵜⴰⵎⴻⵔⴰⵢⵜ ⵜⴰⵖⴻⵔⴰⵢⵜ

République Algérienne Démocratique et Populaire

وزارة التعليم العالي والبحث العلمي

ⵙⴰⵏⴻⵔ ⵜⴰⵎⴻⵔⴰⵢⵜ ⵜⴰⵖⴻⵔⴰⵢⵜ ⵜⴰⵎⴻⵔⴰⵢⵜ

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique



ECOLE NATIONALE
SUPÉRIEURE
D'INFORMATIQUE

المدرسة الوطنية العليا للإعلام الآلي

ⵎⴰⵔⴷⴰⵢⵜ ⵜⴰⵖⴻⵔⴰⵢⵜ ⵜⴰⵎⴻⵔⴰⵢⵜ ⵜⴰⵖⴻⵔⴰⵢⵜ

École nationale Supérieure d'Informatique

Mémoire de fin d'études

Pour l'obtention du diplôme d'Ingénieur d'Etat en Informatique

Option : Systèmes d'Information et Technologies (ST)

Thème :

Algorithmes évolutionnaires pour la conception
automatique de circuits quantiques

Réalisé par :

BOUHADOUZA Rania

GHLIB Romaissa

Encadré par :

Pr. HNAIEN Faicel

Mme. CHARABI Leila

Mme. DJENNANE Nabila

Devant le jury composé de :

Président : Mme. MEZIANE L.

Rapporteur : Mme. KADRI N.

Examineur : Mme. LOUNES N.

Organisme d'accueil :

LIST3N - Université de Technologie de Troyes

Soutenu le 22 Septembre 2025

Promotion : 2024-2025

Dédicace

À **Allah**, le Tout-Puissant, qui a guidé mes pas, illuminé mon chemin et m'a accordé la force de persévérer.

À **ma mère**, flamme bienveillante, lumière et refuge. Par son amour inconditionnel, sa force et ses sacrifices, elle a façonné la personne que je suis aujourd'hui. Elle demeure le souffle qui m'a portée et poussée à ne jamais renoncer.

À **mon père**, pilier de sérénité et de chaleur, dont la confiance et les prières m'ont réconfortée dans les instants de doute.

À **mes frères**, mes compagnons de vie, et tout particulièrement à **Raouf**, mon bras droit et allié fidèle. À mon petit prince **Abdelmounaim**, source de joie pure, ainsi qu'à ma petite sœur et à ma douce **Nour**.

À mes enseignants, **Merazga Abdelkarim** et **Akhenak Assia**, qui ont été bien plus que de simples formateurs : des guides généreux et inspirants.

À mon binôme **Romaïssa**, partenaire de route dans cette aventure académique, avec qui j'ai partagé les défis et les réussites.

À ma cousine **Chourouk**, ainsi qu'à mes amies **Nada**, **Imane** et **Sonia**, pour leur présence, leur soutien et leurs sourires qui ont embelli ce parcours.

Enfin, à toutes celles et ceux qui, de près ou de loin, ont cru en moi, m'ont offert leur aide précieuse et ont contribué à bâtir le chemin qui m'a menée jusqu'ici.

Rania

Dédicace

À Allah, le Tout-Puissant, qui a guidé mes pas, illuminé mon chemin et m'a accordé la force de persévérer dans les moments de doute.

À ma chère mère et à mon père, piliers de ma vie, pour leur amour inconditionnel, leurs prières sincères et leur soutien qui ont été la source de ma motivation et de ma réussite.

À mon frère Abdou, ainsi qu'à mes sœurs Kheira, Souaad, Aïcha, Karima, Fatiha, Chaïma et Douaa, pour leur présence réconfortante et leur encouragement constant, vous avez été une lumière dans mes moments d'effort.

À mes beaux-frères Abdelhak, Youcef, Ahmed et les autres, pour vos encouragements et votre bienveillance.

À mes enseignants de l'ESI et de l'ESTIN, et particulièrement à Monsieur D. Sidali Bousla, pour ses précieux conseils, son encouragement et sa recommandation qui m'ont beaucoup inspiré.

À mon promoteur de stage Faicel Hnaïen et à toute l'équipe administrative de L'UTT, pour leur encadrement, leur disponibilité et leur aide précieuse qui ont grandement facilité ce travail.

À mes encadrantes de ce projet, pour leur patience, leur disponibilité et leurs conseils constructifs qui ont enrichi ce travail.

À ma binôme Rania, pour son soutien, sa collaboration sincère et le partage de chaque étape de ce projet avec détermination et complicité.

À mes amies Safa, Houda, Imane, et à tous mes amis rencontrés à l'ESTIN, en particulier Ouarda et Djihen, pour leur amitié sincère, leur générosité et tous les précieux souvenirs partagés.

Enfin, à toutes les personnes qui, de près ou de loin, ont contribué à la réalisation de ce travail, je vous exprime ma profonde gratitude.

Romaïssa

Remerciements

Le chemin qui a conduit à l'aboutissement de ce travail a été une véritable aventure, faite de croissance, de persévérance et de découvertes. Rien de tout cela n'aurait été possible sans le soutien, l'inspiration et la bienveillance de nombreuses personnes auxquelles nous adressons nos plus sincères remerciements.

Nous tenons tout d'abord à exprimer notre profonde gratitude à **Pr. Faicel Hnaïen**. Son soutien constant, ses conseils éclairés et sa confiance indéfectible ont guidé notre progression et façonné notre parcours de chercheurs.

Nous souhaitons également adresser nos remerciements les plus chaleureux à **Mme. Djennane Nabila** et **Mme. Charabi Leila**, qui ont été bien plus que de simples encadrantes : de véritables mentors. Leur encouragement, leurs analyses pertinentes et leur leadership inspirant ont marqué notre parcours de manière durable.

Nous exprimons aussi notre profonde reconnaissance aux **membres du jury**, pour le temps consacré à l'évaluation de ce travail et pour l'attention portée à nos recherches.

À nos familles, qui ont toujours été notre refuge et notre force, nous disons merci du fond du cœur. Nous souhaitons exprimer notre sincère gratitude à **nos parents**, véritables piliers de cette aventure. Leur présence constante, leurs encouragements sans faille, leur confiance en nous et, surtout, leur *amour inconditionnel* ont été notre force au quotidien. Ils ont cru en nous, parfois davantage que nous-mêmes, et nous ont soutenus sans relâche. Sans eux, rien de tout cela n'aurait été possible.

À nos sœurs et à nos frères, dont les encouragements et la confiance discrète nous ont accompagnés tout au long de ce chemin, nous adressons également toute notre gratitude.

Nous adressons nos sincères remerciements à l'ensemble du **corps enseignant de l'École Nationale Supérieure d'Informatique (ESI, Alger)** pour leur engagement constant et la qualité de leur enseignement. Leur accompagnement a contribué à façonner non seulement notre parcours académique, mais aussi les valeurs et l'éthique professionnelle que nous portons aujourd'hui.

Enfin, à toutes celles et ceux qui, de près ou de loin, par des gestes visibles ou invisibles, ont contribué à la réalisation de ce projet, nous disons un immense MERCI.

Résumé

L'augmentation de la complexité des problèmes dans des domaines tels que l'optimisation, la cryptographie ou la simulation a révélé les limites des ordinateurs classiques. L'informatique quantique, fondée sur les principes de *superposition*, d'*intrication* et d'*interférence*, apparaît comme une alternative prometteuse. Toutefois, les dispositifs actuels dits *NISQ* (*Noisy Intermediate-Scale Quantum*) demeurent fragiles, sensibles au bruit et aux erreurs, ce qui rend la conception manuelle de circuits quantiques inefficace.

La conception de circuits compacts, fidèles et adaptés aux machines quantiques, limitées en nombre de qubits et sujettes au bruit, constitue ainsi un défi majeur. Pour répondre à cette problématique, ce projet explore l'utilisation des algorithmes évolutionnaires, en particulier les algorithmes génétiques et leurs variantes multi-objectifs, appliqués à la synthèse et à l'optimisation automatique de circuits quantiques.

Nous présentons d'abord une approche mono-objectif, fondée sur une fonction de fitness liée à la fidélité, puis une extension multi-objectifs qui prend simultanément en compte la fidélité, la profondeur et le coût en portes. Enfin, afin de surmonter les limites liées à la complexité exponentielle du calcul de la fidélité, nous proposons des stratégies d'évaluation évolutives et scalables, permettant de traiter des circuits de grande taille.

Nos expérimentations démontrent l'efficacité des algorithmes évolutionnaires appliqués à la synthèse de circuits quantiques. Les tests ont montré des réductions notables de la profondeur (jusqu'à 40 %) et du nombre de portes (jusqu'à 50 % du coût global), tout en maintenant une fidélité élevée (≥ 0.85 , souvent proche de 0.9). De plus, l'introduction de stratégies d'évaluation scalables permet un gain de plus de $\times 10$ dans le temps de calcul de la fidélité, rendant possible l'optimisation de circuits de taille moyenne à grande, au-delà de 12 qubits.

Mots-clés : Informatique quantique, Algorithmes génétiques, Optimisation multi-objectifs, Synthèse de circuits.

Abstract

The increasing complexity of problems in fields such as optimization, cryptography, and simulation has revealed the limitations of classical computers. Quantum computing, based on the principles of superposition, entanglement, and interference, emerges as a promising alternative. However, current NISQ (*Noisy Intermediate-Scale Quantum*) devices remain fragile, sensitive to noise and errors, which makes the manual design of quantum circuits ineffective.

The design of compact, faithful, and hardware-adapted circuits—given the limited number of qubits and noise in current machines—thus constitutes a major challenge. To address this issue, this project explores the use of evolutionary algorithms, particularly genetic algorithms and their multi-objective variants, applied to the automatic synthesis and optimization of quantum circuits.

We first present a single-objective approach, based on a fitness function related to fidelity, and then extend it to a multi-objective formulation that simultaneously considers fidelity, circuit depth, and gate cost. Finally, to overcome the exponential complexity of fidelity evaluation, we propose scalable evolutionary strategies that enable the treatment of larger quantum circuits.

Our experiments demonstrate the effectiveness of evolutionary algorithms for quantum circuit synthesis. Tests have shown significant reductions in circuit depth (up to 40 %) and gate count (up to 50 % of the global cost), while maintaining high fidelity (≥ 0.85 , often close to 0.9). Moreover, the introduction of scalable evaluation strategies yields more than a $\times 10$ speed-up in fidelity computation time, making it possible to optimize medium- to large-scale circuits, beyond 12 qubits.

Keywords: Quantum computing, Genetic algorithms, Multi-objective optimization, Circuit synthesis.

ملخص

لقد كشف ازدياد تعقيد المسائل في مجالات مثل الأمثلة والتشفير والمحاكاة حدود الحواسيب الكلاسيكية. وتعدّ الحوسبة الكمومية، المعتمدة على مبادئ التراكب والتشابك والتداخل، بديلاً واعداً. غير أنّ الأجهزة الحالية ما تزال هشة، شديدة الحساسية للضجيج والأخطاء، مما يجعل تصميم الدوائر الكمومية يدوياً عملية غير فعّالة. إن تصميم دوائر كمومية مضغوطة وموثوقة، تتلاءم مع الآلات الكمومية المحدودة في عدد الكيوبتات والمعرّضة للضجيج، يمثل تحدياً رئيسياً. ولمعالجة هذه الإشكالية، يستكشف هذا المشروع استخدام الخوارزميات التطورية، وخاصة الخوارزميات الجينية وتفرّعاتها متعددة الأهداف، في التركيب التلقائي وتحسين الدوائر الكمومية. تقدّم أولاً مقارنة أحادية الهدف تعتمد على دالة ملائمة مرتبطة بمقياس المصدقية، ثم تطورها إلى مقارنة متعددة الأهداف تأخذ بعين الاعتبار في الوقت نفسه المصدقية، عمق الدائرة، وتكلفتها من حيث عدد البوابات. وأخيراً، نقترح استراتيجيات تقييم قابلة للتوسع لقياس المصدقية، مما يتيح التعامل مع دوائر كمومية كبيرة الحجم. تُظهر تجاربنا فعالية الخوارزميات التطورية في توليف الدوائر الكمومية. فقد أظهرت الاختبارات انخفاضاً ملحوظاً في العمق (حتى 40 %) وفي عدد البوابات (حتى 50 % من التكلفة الكلية)، مع الحفاظ على دقة عالية (≥ 0.85) وغالباً قريبة من 0.9. بالإضافة إلى ذلك، فإن إدخال استراتيجيات تقييم قابلة للتوسع يحقق تسريعاً يتجاوز $10 \times$ في وقت حساب الدقة، مما يجعل من الممكن تحسين دوائر متوسطة وكبيرة الحجم، بما يتجاوز 12 كيوبت.

الكلمات المفتاحية: الحوسبة الكمومية، الخوارزميات الجينية، الأمثلة متعددة الأهداف، تركيب الدوائر.

Table des matières

Table des figures	v
Liste des tableaux	vii
Liste des acronymes	viii
Introduction	1
I État de l’art	4
1 L’informatique quantique	5
Introduction	5
1.1 Genèse et évolution de l’informatique quantique	5
1.1.1 Informatique et mécanique quantiques	5
1.1.2 Le calcul quantique et ses origines	5
1.1.3 Évolutions expérimentales récentes	6
1.2 Postulats et notions mathématiques	7
1.2.1 Notion de qubit	7
1.2.2 Représentations d’un qubit	7
1.2.3 Systèmes multi-qubits	8
1.3 Modèles de calcul quantique	9
1.3.1 Opérations sur les états quantiques	9
1.3.2 Portes logiques quantiques	9
1.3.3 Registre quantique	11
1.3.4 Circuits quantiques	11
1.3.5 Propriétés fondamentales du calcul quantique	12
1.4 Algorithmes fondamentaux	13
1.4.1 Algorithme de Deutsch	14
1.4.2 Algorithme de Grover	14
1.4.3 Algorithme de Shor	14
1.4.4 Algorithmes Noisy Intermediate-Scale Quantum (NISQ) :	15
1.5 Les ordinateurs quantiques	15
1.5.1 Les simulateurs quantiques	15
1.5.2 Les ordinateurs quantiques à portes logiques	16
1.5.3 Simulation classique des circuits quantiques	16
1.6 Correction d’erreurs et tolérance aux fautes	17
Conclusion	17
2 Généralités sur les métaheuristiques bio-inspirées	18
Introduction	18
2.1 Classification des métaheuristiques bio-inspirées	18

2.1.1	Les algorithmes évolutionnaires :	19
2.1.2	Les algorithmes d'intelligence en essaim :	25
2.2	Justification du choix de l'algorithme génétique	30
	Conclusion	31
3	Algorithmes évolutionnaires et conception automatique de circuits quantiques	32
	Introduction	32
3.1	Rôle des algorithmes évolutionnaires dans la conception des circuits quantiques	32
3.2	Origines de l'application des algorithmes génétiques en circuits quantiques	33
3.3	Typologie des travaux réalisés	34
3.3.1	Synthèse automatique de circuits :	34
3.3.2	Optimisation structurelle des circuits :	35
3.3.3	Avancées vers l'optimisation multi-objectifs :	38
3.3.4	Développement de frameworks spécialisés :	42
3.4	Synthèse comparative	45
	Conclusion	48
II	Contribution et Expérimentation	49
4	Conception	50
	Introduction	50
4.1	Approche globale proposée	50
4.2	Algorithme génétique mono-objectif	51
4.2.1	Encodage des circuits et représentation	52
4.2.2	Fonction de fitness	53
4.2.3	Méthodes de sélection	54
4.2.4	Stratégies de mutation	58
4.2.5	Croisement (Crossover)	59
4.2.6	Recherche locale des angles de rotation	60
4.2.7	Raffinement structurel de circuit	61
4.2.8	Contrôle dynamique de la diversité	64
4.3	Vers une approche multi-objectifs	65
4.3.1	Architecture globale de l'algorithme	66
4.3.2	Définition des objectifs	67
4.3.3	Les étapes héritées de l'algorithme mono-objectifs	68
4.3.4	Sélection dans l'algorithme Nondominated Sorting Genetic Algorithm II (NSGA-II)	69
4.4	Stratégies d'estimation de fidélité évolutives	71
4.4.1	Méthode 1 : Évaluation de la fidélité par blocs indépendants	72
4.4.2	Méthode 2 : Évaluation de la fidélité par blocs séquentiels chevauchants (fenêtres glissantes)	76
	Conclusion	79
5	Réalisation	80
	Introduction	80
5.1	Technologies utilisées	80
5.2	Corpus de benchmarks pour l'évaluation	82

5.2.1	MQT Bench	82
5.2.2	QASMBench	82
5.2.3	VeriQBench	82
5.3	Réalisation des étapes de l'algorithme génétique mono-objectif	83
5.3.1	Génération aléatoire de chromosome	83
5.3.2	Traduction d'un chromosome à un circuit quantique	84
5.3.3	Implémentation du croisement via circuits quantiques	85
5.3.4	Implémentation de la mutation via circuits quantiques	86
5.3.5	Implémentation du mécanisme du raffinement d'un circuit quantique : .	89
5.4	Réalisation des étapes de l'algorithme multi-objectifs	92
5.4.1	Profondeur	92
5.4.2	Coût	93
5.4.3	Sélection dans l'algorithme NSGA-II	94
5.5	Réalisation des stratégies d'estimation de fidélité évolutives	96
5.5.1	Réalisation de la méthode d'évaluation de la fidélité par blocs indé- pendants :	96
5.5.2	Réalisation de la méthode d'évaluation de la fidélité via les blocs chevauchants :	97
	Conclusion	97
6	Tests et Résultats	98
	Introduction	98
6.1	Contexte expérimental	98
6.2	Configuration expérimentale	98
6.3	Méthodologie d'évaluation	99
6.4	Études d'ablation et choix des hyperparamètres	99
6.4.1	Impact de la mutation structurée	100
6.4.2	Impact de recherche locale des paramètres de rotation	100
6.4.3	Comparaison entre différentes stratégies de sélection	101
6.4.4	Comparaison entre différents taux de mutation	102
6.4.5	Comparaison entre croisements	103
6.4.6	Impact du contrôle de diversité	104
6.4.7	Paramètres retenus pour l'évaluation	104
6.5	Résultats expérimentaux et tests comparatifs	105
6.5.1	Tests de l'algorithme mono-objectif	105
6.5.2	Tests de l'algorithme multi-objectifs	111
6.5.3	Tests des stratégies d'estimation de la fidélité	116
	Conclusion	121
	Conclusion et perspectives	123
	Annexe A	131
	Annexe B	134

Table des figures

1.1	Visualisation d'un qubit sur la sphère de Bloch	8
1.2	Les opérateurs quantiques	11
1.3	Circuit quantique préparant un état de Bell.	12
1.4	Représentation de la mesure dans un circuit quantique	12
1.5	Circuit quantique de l'algorithme de Deutsch	14
2.1	Classification des métaheuristiques bio-inspirées	18
2.2	Les opérateurs utilisés dans l'AG	23
2.3	Principe général de l'AG	24
2.4	Les étapes de déroulement de Ant Colony Optimization (ACO)	28
2.5	Les étapes de déroulement de Particle Swarm Optimization (PSO)	30
3.1	Schéma de déroulement de l'algorithme génétique basé sur le modèle en îles	38
3.2	Le principe de fonctionnement de (Quantum Global Optimizer (QGo)). . .	43
3.3	Un exemple d'optimisation de circuit quantique avec (QGo).	44
3.4	Optimisation des circuits quantiques dans le cadre Genetic Algorithm for Quantum Circuit Optimization (GA4QCO).	45
4.1	Schéma général de l'approche proposée.	51
4.2	Déroulement de l'algorithme génétique proposé	52
4.3	Sélection par roulette	55
4.4	Sélection par tournoi	57
4.5	Sélection stochastique universelle	58
4.6	Architecture de l'approche multi-objectifs	66
4.7	Calcul de la profondeur logique	67
4.8	Pipeline d'optimisation de circuits quantiques par blocs	72
4.9	Pipeline d'optimisation de circuits quantiques par blocs séquentiels chevauchants	77
5.1	Logo de Qiskit.	80
5.2	Logo de PennyLane.	80
5.3	Interface Graphique pour l'implémentation de circuits quantiques avec le simulateur de IBM	82
5.4	Exemple de génération de deux chromosomes aléatoires distincts	84
5.5	Association entre un chromosome et son circuit quantique correspondant. .	85
5.6	Représentation chromosomique des parents et résultats du croisement . . .	85
5.7	Croisement	86
5.8	Mutation par substitution	87
5.9	Mutation par suppression	88
5.10	Mutation par insertion	88
5.11	Circuit quantique avant l'annulation de portes involutives	90
5.12	Circuit quantique après l'annulation de portes involutives	90
5.13	Circuit quantique avant la fusion de portes de rotation	92

5.14	Circuit quantique après la fusion de portes de rotation	92
5.15	Illustration du coût associé à un circuit quantique	93
5.16	Circuit quantique optimisé par NSGA-II	94
5.17	Processus d'optimisation en utilisant la décomposition des circuits quantiques des blocs indépendants	96
5.18	Processus d'optimisation en utilisant la décomposition spatial des circuit quantique en blocs chevauchants	97
6.1	Comparaison radar des stratégies de sélection	101
6.2	Analyse statistique par boîtes à moustaches des performances algorithmiques selon le taux de mutation	102
6.3	Évolution de la fidélité en fonction du nombre de générations pour différents taux de mutation	102
6.4	Analyse comparative multi-critères des performances selon le taux de mutation	103
6.5	Reconstruction de la porte CNOT via L'AG	105
6.6	L'évolution de la fidélité au fil des générations de la porte CNOT via L'AG	106
6.7	L'optimisation d'un circuit quantique aléatoire à l'aide d'un algorithme génétique mono-objectif.	107
6.8	L'évolution de la fidélité au fil des générations	107
6.9	L'optimisation d'un circuit quantique de maxcut à l'aide d'un algorithme génétique mono-objectif.	108
6.10	Temps d'exécution selon le nombre de qubits	110
6.11	Optimisation d'un circuit quantique aléatoire	111
6.12	Front de Pareto pour un circuit aléatoire	112
6.13	Évolution des métriques lors de l'optimisation	112
6.14	L'optimisation d'un circuit quantique vqe	113
6.15	pareto	114
6.16	evolution	114
6.17	Le temps d'exécution de NSGA-II en fonction du nombre de qubits	116
6.18	optimisation	117
6.19	Optimisation d'un circuit quantique aléatoire avec méthode d'évaluation de la fidélité par blocs séquentiels chevauchants	119
6.20	Comparaison des temps d'exécution entre l'évaluation globale par matrice unitaire et les méthodes scalables (par blocs indépendants et blocs superposés).	121

Liste des tableaux

1.1	Quelques simulateurs de circuits quantiques	16
2.1	Algorithmes SI importants	26
3.1	Comparaison des méthodes évolutionnaires pour la synthèse de circuits quantiques	46
3.2	Comparaison des approches multi-objectifs pour la synthèse de circuits quantiques	47
4.1	Résumé des coûts associés aux principales portes quantiques (LEE et al., 2006).	68
6.1	Plages de valeurs testées pour les principaux hyperparamètres	99
6.2	Étude d’ablation multi-circuits : impact de la mutation structurée. Chaque bloc (3 colonnes) correspond à un circuit testé.	100
6.3	Étude d’ablation : impact de l’optimisation locale des angles.	101
6.4	Étude comparative : impact des méthodes de sélection	101
6.5	Comparaison des performances entre croisement à un point et à deux points.	103
6.6	Étude d’ablation multi-circuits : impact du contrôle de diversité (recuit simulé + mutation adaptative).	104
6.7	Hyperparamètres fixés à partir des études d’ablation.	105
6.8	Résultats expérimentaux sur des circuits quantiques complexes (avant/après optimisation) à l’aide de l’algorithme génétique mono-objectif	106
6.9	Résultats expérimentaux sur les circuits aléatoires (avant/après optimisation) à l’aide de l’algorithme génétique mono-objectif.	107
6.10	Résultats expérimentaux sur circuits applicatifs (avant/après optimisation) à l’aide d’un algorithme génétique mono-objectif.	109
6.11	Fidélité finale obtenue sur 10 circuits aléatoires distincts – Évaluation de la stabilité	110
6.12	Résultats expérimentaux sur les circuits aléatoires (avant/après optimisation) à l’aide de NSGA-II	113
6.13	Résultats expérimentaux sur les circuits quantiques orientés problème à l’aide de NSGA-II	115
6.14	Résultats expérimentaux sur des circuits aléatoires avec la méthode des blocs indépendants	117
6.15	Résultats expérimentaux sur des circuits quantiques orientés problème avec la méthode des blocs indépendants	118
6.16	Résultats expérimentaux sur des circuits aléatoires avec la méthode des blocs chevauchants (fenêtres glissantes)	120
6.17	Résultats expérimentaux sur des circuits quantiques orientés problème avec la méthode des blocs chevauchants (fenêtres glissantes)	120

Liste des acronymes

A*	A-star Search Algorithm	36
ACO	Ant Colony Optimization	v
AE	Algorithme Évolutionnaire	19
AG	Algorithme Génétique	19
AGQ	Algorithme génétique quantique	34
BFGS	Broyden–Fletcher–Goldfarb–Shanno algorithm	37
CNOT	Controlled-NOT	10
COBYLA	Constrained Optimization BY Linear Approximations	37
CSS	Calderbank-Shor-Steane	35
CU	Controlled-Unitary	10
EQAS-PQC	Evolutionary Quantum Architecture Search for Parameterized Quantum Circuits	41
GA4QCO	Genetic Algorithm for Quantum Circuit Optimization	v
GASP	Genetic Algorithm for State Preparation	38
GPU	Graphics Processing Unit	36
IBM	International Business Machines	6
LRSP	Low-Rank State Preparation	40
MEAS-PQC	Multi-Objective Evolutionary Architecture Search for Parameterized Quantum Circuits	40
NISQ	Noisy Intermediate-Scale Quantum	ii
NSGA-II	Nondominated Sorting Genetic Algorithm II	iii
PSO	Particle Swarm Optimization	v
PQC	Parameterized Quantum Circuit	40
QAOA	Quantum Approximate Optimization Algorithm	15
QECC	Quantum Error Correcting Codes	17
QEQA	Quantum Encoded Quantum Evolutionary Algorithm	35
QFT	Quantum Fourier Transform	14
QGo	Quantum Global Optimizer	v
QUBO	Quadratic Unconstrained Binary Optimization	6
RL	Reinforcement Learning	41
RMN	Résonance Magnétique Nucléaire	6
RS	Random Search	35
RSA	Rivest–Shamir–Adleman	6
SDK	Software Development Kit	15
SI	Swarm Intelligence	25
SLSQP	Sequential Least Squares Programming	38
SOFTMAX-PQC	Softmax-Based Architecture Search for Parameterized Quantum Circuits	41

SUS	Stochastic Universal Sampling	22
UMAD	Uniform Mutation by Addition and Deletion	41
VQE	Variational Quantum Eigensolver	15

Introduction générale

Au cours des dernières décennies, l'augmentation de la complexité des problèmes à résoudre dans des domaines variés tels que l'optimisation combinatoire, la cryptographie, la simulation moléculaire ou encore le traitement de grandes quantités de données a mis en évidence les limites physiques des ordinateurs classiques. Face à ces contraintes, l'informatique quantique s'impose comme une alternative prometteuse. En s'appuyant sur des phénomènes fondamentaux de la mécanique quantique tels que la superposition, l'intrication et l'interférence, l'informatique quantique permet de représenter et manipuler l'information de manière plus efficace, offrant un potentiel de calcul exponentiellement supérieur à celui des systèmes classiques.

Des avancées majeures ont été réalisées grâce à des algorithmes pionniers tels que celui de SHOR (1994) pour la factorisation et celui de GROVER (1996) pour la recherche non structurée, ouvrant la voie à une véritable révolution dans le traitement de l'information. Aujourd'hui, les géants technologiques comme Google, IBM ou Microsoft investissent massivement dans la construction de processeurs quantiques tels que Sycamore (73 qubits) ou IBM Condor (1121 qubits), tout en offrant un accès en ligne via des plateformes comme IBM Q Experience (DEVITT, 2016).

Un circuit quantique est formellement représenté par une matrice unitaire qui décrit son action sur un registre de qubits. Ce formalisme, bien que puissant, devient difficile à manipuler car la taille de la matrice croît de façon exponentielle avec le nombre de qubits. Ainsi, la conception manuelle de circuits basée sur la factorisation de ces matrices se révèle rapidement impraticable, d'autant plus que les dispositifs NISQ souffrent de bruit, d'erreurs et de ressources limitées. La réduction du nombre de portes et de la profondeur du circuit constitue donc un objectif essentiel.

Face à ces contraintes, la conception automatique de circuits quantiques s'impose comme une solution prometteuse. Elle vise à générer, à partir d'un circuit initial ou d'une matrice unitaire cible, un circuit équivalent ou approximativement fidèle. Lorsque l'équivalence exacte n'est pas possible, la similarité entre les circuits est mesurée par une métrique de fidélité, qui évalue à quel point le circuit généré reproduit le comportement attendu, tout en minimisant le coût structurel et l'impact du bruit.

Un tel problème peut être vu comme une tâche d'optimisation combinatoire sur un espace de recherche immense. Les métaheuristiques se montrent particulièrement adaptées pour l'explorer efficacement. Parmi elles, les algorithmes évolutionnaires, et en particulier les algorithmes génétiques, offrent un cadre naturel : grâce à des opérateurs tels que la sélection, le croisement et la mutation, ils permettent d'approcher progressivement des circuits optimaux en équilibrant fidélité, profondeur et nombre de portes.

La problématique qui se pose est donc la suivante : *Comment générer automatiquement des circuits quantiques optimisés, capables de préserver la fidélité fonctionnelle tout en réduisant le coût, la profondeur et en restant adaptés aux contraintes des architectures NISQ ?*

L'objectif principal de ce mémoire est de proposer une approche évolutionnaire pour la conception automatique de circuits quantiques optimisés.

Cet objectif global se décline en plusieurs objectifs spécifiques :

- Génération d'un circuit optimisé à partir d'un circuit initial.
- Préservation du comportement fonctionnel du circuit initial, en augmentant la fidélité du circuit cible par rapport au circuit initial.
- Réduction du coût du circuit initial (nombre et type de portes).
- Réduction de la profondeur du circuit initial, afin de réduire le bruit lié au matériel NISQ.
- Développement d'une méthode évolutive et scalable, adaptée aux circuits de grande taille, reposant soit sur une décomposition en blocs indépendants pour les architectures faiblement connectées, soit sur des blocs chevauchants (fenêtres glissantes) pour les circuits fortement connectés, assurant ainsi un compromis entre précision et temps de calcul.

Pour répondre à ces objectifs, la méthodologie adoptée repose sur une approche en plusieurs étapes :

1. Conception d'un algorithme génétique initialement mono-objectif basé sur la fidélité.
2. Extension en une version multi-objectifs intégrant simultanément la fidélité, la profondeur et le coût structurel.
3. Définition de stratégies de représentation, d'opérateurs génétiques et de mécanismes d'adaptation favorisant la convergence.
4. Mise en place d'évaluations scalables de la fidélité via des méthodes de décomposition et d'approximation adaptées aux grands circuits.
5. Validation expérimentale de la méthode sur divers cas d'étude et comparaison avec les approches existantes.

Le mémoire est structuré en deux grandes parties complémentaires :

- **La première partie** est consacrée à l'étude bibliographique. Elle présente les fondements de l'informatique quantique et des algorithmes d'optimisation bio-inspirés, en particulier les algorithmes génétiques, ainsi que les travaux de recherche récents portant sur leur application à la synthèse automatique de circuits quantiques. Cette partie comprend trois chapitres :
 - **Chapitre 1 – Informatique quantique : concepts et définitions de base** : ce chapitre introduit les éléments fondamentaux de l'informatique quantique, tels que les qubits, les portes logiques, l'intrication, les mesures et les modèles de calcul. Il décrit également les spécificités des circuits quantiques dans les environnements bruités, en mettant l'accent sur les défis liés à la réalisation physique de ces circuits.
 - **Chapitre 2 – Généralités sur les métaheuristiques bio-inspirées** : ce chapitre offre une vue d'ensemble des principales méthodes d'optimisation issues de la nature. Il présente les grandes familles de ces approches, notamment les algorithmes évolutionnaires et les méthodes issues de l'intelligence en essaim, ainsi que certaines de leurs variantes représentatives.
 - **Chapitre 3 – Algorithmes évolutionnaires et conception automatique de circuits quantiques** : Revue des travaux récents qui ont démontré la faisabilité d'appliquer les algorithmes évolutionnaires notamment les algorithmes génétiques pour la conception des circuits quantiques, justifie l'utilisation de ce type d'algorithmes plutôt que l'utilisation des algorithmes classiques.

- **La seconde partie** est dédiée à notre contribution originale. Elle décrit la conception, la mise en œuvre et l'évaluation expérimentale d'un algorithme génétique dédié à la génération de circuits quantiques performants. Cette partie se décline en trois chapitres :
 - **Chapitre 4 – Conception** : Présentation d'un algorithme génétique initialement mono-objectif, puis enrichi en une version multi-objectifs intégrant simultanément la fidélité, la profondeur et le coût des portes. Nous détaillons la représentation des individus, les opérateurs génétiques et les mécanismes d'adaptation qui améliorent la convergence. Enfin, des stratégies d'évaluation scalable de la fidélité sont introduites, reposant sur des méthodes de décomposition et d'approximation adaptées aux circuits de grande taille.
 - **Chapitre 5 – Réalisation** : Ce chapitre est consacré aux aspects techniques de la mise en œuvre. Il décrit les outils logiciels (comme Qiskit ou PennyLane), les bibliothèques Python utilisées, l'environnement d'exécution, ainsi que la configuration des jeux de données et des expériences. Il sert de passerelle entre la conception théorique et l'application pratique.
 - **Chapitre 6 – Tests et Résultats** : Évaluation complète de l'algorithme sur plusieurs cas d'étude, avec comparaison aux approches existantes. Les résultats sont analysés selon divers critères (fidélité, profondeur, nombre de portes), permettant d'évaluer précisément l'efficacité de la méthode proposée.

Cette structure permet d'offrir une compréhension claire des enjeux de la synthèse de circuits quantiques et de proposer une réponse méthodologique concrète, validée expérimentalement, face aux défis identifiés dans la littérature.

Cette organisation permet d'introduire progressivement les fondements théoriques, de justifier la pertinence de l'approche évolutionnaire, puis de présenter et valider expérimentalement la solution proposée.

Première partie

État de l'art

Chapitre 1

L'informatique quantique

Introduction

Ce chapitre présente les fondements théoriques nécessaires à la compréhension de l'informatique quantique et à la formulation des méthodes d'optimisation étudiées dans ce mémoire. Nous y introduisons les concepts clés, tels que les qubits, les portes logiques quantiques et la structure des circuits quantiques.

Une section préliminaire rappelle également les notions essentielles d'algèbre linéaire — vecteurs, matrices, opérateurs unitaires — qui constituent le langage mathématique du calcul quantique. Cette base permettra de mieux appréhender les phénomènes de superposition, d'intrication et d'évolution d'état, ainsi que leur rôle dans la conception et l'optimisation de circuits quantiques.

Enfin, ce chapitre établit le lien entre ces principes fondamentaux et les problématiques rencontrées dans l'ère des dispositifs NISQ, préparant ainsi l'étude des méthodes évolutionnaires présentées dans les chapitres suivants.

1.1 Genèse et évolution de l'informatique quantique

1.1.1 Informatique et mécanique quantiques

La mécanique quantique, développée entre 1900 et 1925 par Planck, Einstein, Schrödinger et d'autres, bouleverse profondément notre intuition classique. À l'échelle microscopique, un système n'occupe pas un état unique, mais peut exister dans une *superposition cohérente* de plusieurs états simultanément (CHUANG, 2002 ; GAMBS, 2008). Ainsi, au lieu d'être localisé uniquement en p_1 ou p_2 , un objet quantique est représenté par la combinaison linéaire $|p_1\rangle + |p_2\rangle$.

L'informatique quantique exploite ces propriétés fondamentales — superposition, intrication, interférence — pour traiter et stocker l'information. Contrairement au bit classique, limité à 0 ou 1, un *qubit* est décrit par l'état $\alpha|0\rangle + \beta|1\rangle$, avec $|\alpha|^2 + |\beta|^2 = 1$, ce qui permet d'encoder simultanément 2^n configurations différentes à l'aide de n qubits (NIELSEN & CHUANG, 2000). Cette capacité donne naissance au *parallélisme quantique*, permettant d'explorer de nombreux résultats en une seule opération.

1.1.2 Le calcul quantique et ses origines

L'idée d'utiliser des systèmes quantiques pour effectuer des calculs plus puissants que ceux des machines classiques apparaît en 1982, lorsque Richard Feynman propose d'utiliser des dispositifs quantiques pour simuler des phénomènes physiques complexes

que les ordinateurs classiques sont incapables de modéliser efficacement (RICHARD & FEYNMAN, 1982).

Dans le même élan, Benioff (1980) pose les bases du *calcul réversible* dans un cadre quantique (BENIOFF, 1980), et DEUTSCH (1985) formalise la notion de *qubit* ainsi qu'un ensemble universel de portes logiques quantiques. Ces travaux ouvrent la voie au développement d'une véritable théorie du calcul quantique.

Le potentiel de l'informatique quantique s'affirme véritablement à partir de 1994 avec l'algorithme de Shor pour la factorisation des entiers, qui montre que certains problèmes autrefois considérés comme inabornables peuvent être résolus en temps polynomial sur un ordinateur quantique (SHOR, 1994). Cet algorithme menace directement la sécurité des systèmes cryptographiques classiques comme Rivest–Shamir–Adleman (RSA).

Deux ans plus tard, Grover (1996) propose un algorithme de recherche quantique, capable de retrouver une entrée dans une base de données non structurée en $O(\sqrt{N})$ opérations, contre $O(N)$ pour la recherche classique (GROVER, 1996). Cette accélération quadratique illustre la puissance combinée de la superposition et de l'interférence. En parallèle, la croissance exponentielle des performances des ordinateurs classiques, formulée dans la *loi de Moore* (1975) (MOORE, 1975), atteint progressivement ses limites physiques à partir des années 2010, notamment à cause de la miniaturisation extrême des composants électroniques (THEIS & WONG, 2017). Face à cette impasse annoncée, l'informatique quantique apparaît comme un nouveau paradigme capable de poursuivre l'amélioration exponentielle des capacités de calcul.

1.1.3 Évolutions expérimentales récentes

Les premières expériences de manipulation d'états quantiques datent de 1998 avec l'utilisation de la Résonance Magnétique Nucléaire (RMN) pour implémenter une porte logique élémentaire (GERSHENFELD & CHUANG, 1997). En 2001, la première démonstration d'un qubit supraconducteur contrôlé est réalisée à l'Université Yale (NAKAMURA et al., 1999). Puis, en 2005, la mise en œuvre d'une porte logique entre ions piégés marque une étape décisive dans la fiabilité du contrôle quantique (NIELSEN, 2003).

Avec l'émergence de l'ère dite NISQ, International Business Machines (IBM) propose en 2016 le premier accès public à un processeur quantique de 5 qubits via le cloud (RESEARCH, 2016). Depuis, les avancées se sont accélérées : IBM dévoile en 2021 un processeur de 127 qubits baptisé Eagle (RESEARCH, 2021), tandis que Rigetti annonce un processeur de 128 qubits pour 2022 (COMPUTING, 2022).

D'autres entreprises, telles que Google, Quantinuum et IonQ, se positionnent également sur le marché, tandis que D-Wave Systems développe des machines basées sur le recuit quantique, spécialisées dans la résolution de problèmes d'optimisation combinatoire, notamment des problèmes de type Quadratic Unconstrained Binary Optimization (QUBO) (SYSTEMS, 2019). Aujourd'hui, l'informatique quantique vise à simuler des systèmes physiques complexes, optimiser des problèmes industriels, et renforcer la sécurité informatique grâce aux propriétés intrinsèques du monde quantique. Bien que les dispositifs actuels soient encore bruyants et limités, les perspectives ouvertes par ce nouveau paradigme laissent entrevoir des applications majeures dans la cryptographie, la chimie moléculaire, l'intelligence artificielle, et la logistique.

1.2 Postulats et notions mathématiques

1.2.1 Notion de qubit

Le *qubit*, ou bit quantique, constitue l'unité élémentaire d'information dans un système quantique, tout comme le bit en est l'équivalent dans les systèmes classiques (JAFFALI, 2020). Un qubit peut exister dans une superposition de ses deux états de base. Cette propriété fondamentale découle directement des principes de la mécanique quantique.

Dans la base canonique, les deux états de référence (ou *états de base*) sont notés $|0\rangle$ et $|1\rangle$ selon la **notation de Dirac**, également appelée *ket*¹. Ils sont définis comme suit :

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}. \quad (1.2.1)$$

Un état général d'un qubit est alors représenté comme une combinaison linéaire complexe de ces deux états (NIELSEN & CHUANG, 2000) :

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle, \quad \alpha, \beta \in \mathbb{C}, \quad |\alpha|^2 + |\beta|^2 = 1, \quad (1.2.2)$$

où $|\alpha|$ et $|\beta|$ sont les *modules* des nombres complexes α et β respectivement².

Cette contrainte de normalisation garantit que la somme des probabilités de mesure soit toujours égale à 1. Lors d'une mesure dans la base $\{|0\rangle, |1\rangle\}$, l'état $|\psi\rangle$ s'effondre aléatoirement sur l'un des deux états de base avec des probabilités données par $|\alpha|^2$ et $|\beta|^2$.

1.2.2 Représentations d'un qubit

1.2.2.1 Représentation dans l'espace de Hilbert

Le qubit appartient à l'espace vectoriel complexe $\mathcal{H}_1 = \mathbb{C}^2$. On démontre mathématiquement qu'un état de qubit $|\psi\rangle$ peut être exprimé à l'aide de deux paramètres réels, appelés **angles d'Euler** (θ, φ) , selon la relation suivante (YEPEZ, 2013) :

$$|\psi\rangle = \cos \frac{\theta}{2} |0\rangle + e^{i\varphi} \sin \frac{\theta}{2} |1\rangle, \quad 0 \leq \theta \leq \pi, \quad 0 \leq \varphi < 2\pi. \quad (1.2.3)$$

Pour un système de n qubits, l'espace de Hilbert associé possède la dimension 2^n et tout état se décompose dans la base computationnelle (GAMBS, 2008) :

$$|\Psi\rangle = \sum_{i=0}^{2^n-1} a_i |i\rangle, \quad a_i \in \mathbb{C}, \quad \sum_i |a_i|^2 = 1. \quad (1.2.4)$$

où chaque coefficient $a_i \in \mathbb{C}$ est une amplitude complexe. L'état $|i\rangle$ représente ici l'encodage binaire de l'entier i sur n bits (par exemple $|3\rangle = |011\rangle$ pour $n = 3$).

1. La notation de Dirac (ou notation bra-ket) est un formalisme utilisé en mécanique quantique pour représenter les vecteurs d'état : un *ket* $|\psi\rangle$ désigne un vecteur colonne (état quantique), tandis qu'un *bra* $\langle\psi|$ désigne son conjugué hermitien (vecteur ligne).

2. Le module d'un nombre complexe $z = x + iy$ est défini par $|z| = \sqrt{x^2 + y^2}$, et représente sa distance à l'origine dans le plan complexe.

1.2.2.2 Sphère de Bloch :

La sphère de Bloch de la figure 1.1 fournit une représentation tridimensionnelle intuitive d'un qubit : les états $|0\rangle$ et $|1\rangle$ occupent les pôles, tandis qu'un état arbitraire $|\psi\rangle$ se situe au point de coordonnées (θ, φ) .

Cette représentation facilite la compréhension des opérations unitaires appliquées aux qubits.

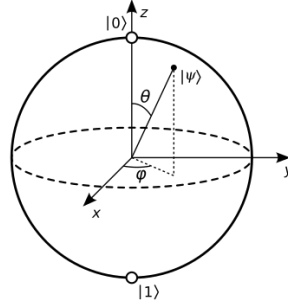


FIGURE 1.1 – Visualisation d'un qubit sur la sphère de Bloch (NIELSEN & CHUANG, 2000)

1.2.3 Systèmes multi-qubits

Lorsque plusieurs qubits sont combinés dans un même système, l'espace d'état résultant croît de manière exponentielle. Un système de deux qubits, par exemple, est représenté dans l'espace de Hilbert $\mathcal{H}_2 = \mathbb{C}^2 \otimes \mathbb{C}^2 = \mathbb{C}^4$, ce qui donne lieu à une base computationnelle composée de quatre états orthonormés :

$$\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}.$$

la représentation vectorielle de ces états :

$$|00\rangle = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}, \quad |01\rangle = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}, \quad |10\rangle = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix}, \quad |11\rangle = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}.$$

Un état général de ce système peut alors être exprimé comme une superposition linéaire de ces quatre états :

$$|\psi_{AB}\rangle = \alpha_{00}|00\rangle + \alpha_{01}|01\rangle + \alpha_{10}|10\rangle + \alpha_{11}|11\rangle, \quad \sum_{i,j} |\alpha_{ij}|^2 = 1, \quad (1.2.5)$$

où chaque $\alpha_{ij} \in \mathbb{C}$ représente l'amplitude de probabilité associée à l'état $|ij\rangle$ (JAFFALI, 2020 ; NIELSEN & CHUANG, 2000).

Ce formalisme s'étend naturellement à n qubits, dont l'état quantique est décrit comme une superposition de 2^n états de base. Par exemple, pour un registre de n qubits, l'état général s'écrit :

$$|\psi_N\rangle = \sum_{k=0}^{2^n-1} \alpha_k |k\rangle, \quad (1.2.6)$$

où $|k\rangle$ désigne les états binaires possibles, et α_k les amplitudes complexes normalisées.

Le comportement d'un système multi-qubits repose aussi sur le produit tensoriel entre états individuels. Si les qubits A et B sont respectivement dans les états $|\psi_A\rangle = \alpha_A|0\rangle + \beta_A|1\rangle$ et $|\psi_B\rangle = \alpha_B|0\rangle + \beta_B|1\rangle$, alors l'état global du système est donné par :

$$|\psi_{AB}\rangle = |\psi_A\rangle \otimes |\psi_B\rangle = \alpha_A\alpha_B|00\rangle + \alpha_A\beta_B|01\rangle + \beta_A\alpha_B|10\rangle + \beta_A\beta_B|11\rangle. \quad (1.2.7)$$

Cet état appartient à l'espace de Hilbert $\mathcal{H}_{AB} = \mathcal{H}_A \otimes \mathcal{H}_B$, de dimension $2^2 = 4$ (JAFFALI, 2020).

1.3 Modèles de calcul quantique

1.3.1 Opérations sur les états quantiques

Maintenant que nous avons défini les qubits et les systèmes quantiques, il est essentiel de comprendre comment manipuler ces états au moyen d'opérations quantiques.

Produit tensoriel. Pour modéliser plusieurs qubits, on utilise le *produit tensoriel* (ou de Kronecker). Si $M \in \mathbb{C}^{p \times q}$ et $N \in \mathbb{C}^{x \times y}$, alors :

$$M \otimes N \in \mathbb{C}^{(px) \times (qy)}, \quad (1.3.1)$$

qui agit sur l'espace de Hilbert $\mathcal{H}_p \otimes \mathcal{H}_x$. Pour deux qubits :

$$\begin{aligned} |0\rangle \otimes |0\rangle &= |00\rangle, & |0\rangle \otimes |1\rangle &= |01\rangle, \\ |1\rangle \otimes |0\rangle &= |10\rangle, & |1\rangle \otimes |1\rangle &= |11\rangle. \end{aligned}$$

1.3.2 Portes logiques quantiques

Les portes logiques quantiques manipulent les qubits, de manière analogue aux portes logiques classiques. Elles sont représentées par des matrices *unitaires* U satisfaisant la relation $U^\dagger U = I$, où $U^\dagger$ désigne la transposée conjuguée (ou adjointe) de U (NIELSEN & CHUANG, 2000).

1.3.2.1 Portes à un qubit

Pauli-X (NOT) : inverse l'état d'un qubit (analogue à une porte NOT classique) :

$$X|0\rangle = |1\rangle, \quad X|1\rangle = |0\rangle, \quad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

Pauli-Y : combine une inversion d'état d'un qubit et une inversion de phase :

$$Y|0\rangle = i|1\rangle, \quad Y|1\rangle = -i|0\rangle, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}.$$

Pauli-Z : change la phase du qubit $|1\rangle$:

$$Z|0\rangle = |0\rangle, \quad Z|1\rangle = -|1\rangle, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

Hadamard (H) : crée une superposition d'états :

$$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle), \quad H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle), \quad H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.$$

Rotation $R_x(\theta)$: effectue une rotation d'angle θ autour de l'axe x de la sphère de Bloch :

$$R_x(\theta) = \begin{pmatrix} \cos\left(\frac{\theta}{2}\right) & -i \sin\left(\frac{\theta}{2}\right) \\ -i \sin\left(\frac{\theta}{2}\right) & \cos\left(\frac{\theta}{2}\right) \end{pmatrix}.$$

Rotation $R_y(\theta)$: effectue une rotation d'angle θ autour de l'axe y :

$$R_y(\theta) = \begin{pmatrix} \cos\left(\frac{\theta}{2}\right) & -\sin\left(\frac{\theta}{2}\right) \\ \sin\left(\frac{\theta}{2}\right) & \cos\left(\frac{\theta}{2}\right) \end{pmatrix}.$$

Rotation $R_z(\theta)$: effectue une rotation d'angle θ autour de l'axe z (modifie uniquement la phase relative des états) :

$$R_z(\theta) = \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix}.$$

1.3.2.2 Portes multi-qubits

Les portes multi-qubits agissent sur deux qubits ou plus. Elles sont cruciales pour générer l'intrication et effectuer des calculs complexes.

Controlled-NOT (CNOT) : applique X sur le qubit cible si le qubit de contrôle est $|1\rangle$:

$$\text{CNOT}|c, t\rangle = \begin{cases} |0, t\rangle & \text{si } c = 0, \\ |1, Xt\rangle & \text{si } c = 1 \end{cases} \quad \text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

SWAP : échange l'état de deux qubits :

$$\text{SWAP}|q_1, q_2\rangle = |q_2, q_1\rangle, \quad \text{SWAP} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Toffoli (CCNOT) : applique X sur la cible si les deux qubits de contrôle sont $|1\rangle$.

Controlled-Unitary (CU) Porte contrôlée- U : applique U sur la cible si le contrôle est $|1\rangle$;

$$\text{CU} = \begin{pmatrix} I_2 & 0 \\ 0 & U \end{pmatrix} \in \mathbb{C}^{4 \times 4}$$

Le tableau de la figure 1.2 présente un récapitulatif des opérateurs quantiques les plus utilisés.

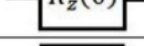
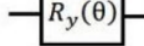
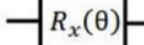
Gate	Symbol	Matrix
Pauli-X (NOT) Bit flip		$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$
Pauli-Y Bit + phase flip		$\begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$
Pauli-Z Phase flip		$\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$
S Phase shift ($\pi/2$)		$\begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}$
T Phase shift ($\pi/4$)		$\begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}$
Hadamard Superposition		$\frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$
Rotation Z Z-axis rotation		$\begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix}$
Rotation Y Y-axis rotation		$\begin{pmatrix} \cos(\frac{\theta}{2}) & -\sin(\frac{\theta}{2}) \\ \sin(\frac{\theta}{2}) & \cos(\frac{\theta}{2}) \end{pmatrix}$
Rotation X X-axis rotation		$\begin{pmatrix} \cos(\frac{\theta}{2}) & -i\sin(\frac{\theta}{2}) \\ i\sin(\frac{\theta}{2}) & \cos(\frac{\theta}{2}) \end{pmatrix}$
CNOT Conditional flip		$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$
Swap Swap qubits		$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$

FIGURE 1.2 – Les opérateurs quantiques d'après (JHA, 2023).

1.3.3 Registre quantique

Un *registre quantique* est un système composé de plusieurs qubits, permettant de manipuler des états quantiques de dimension supérieure. Formellement, un registre de n qubits est un vecteur d'état dans un espace de Hilbert de dimension 2^n , noté $\mathcal{H} = \mathbb{C}^{2^n}$. L'état général d'un tel registre est une superposition linéaire des 2^n états de base, exprimée par la formule 1.2.4.

1.3.4 Circuits quantiques

Un *circuit quantique* est une représentation schématique des calculs effectués avec des qubits. Il est composé de lignes horizontales représentant les qubits, et de portes quantiques qui agissent sur ces lignes.

Les circuits se lisent de gauche à droite, mais les opérations sont exprimées mathématiquement de droite à gauche (composition fonctionnelle inverse).

Préparation d'un état de Bell : La figure 1.3 montre un circuit simple utilisant une porte Hadamard suivie d'une porte CNOT pour préparer l'état intriqué :

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) \quad (1.3.2)$$

Ce circuit encode l'équation suivante :

$$\text{CNOT}(H \otimes I)|00\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

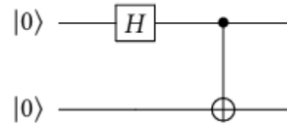


FIGURE 1.3 – Circuit quantique préparant un état de Bell.

Mesure dans les circuits quantiques : En mécanique quantique, mesurer un qubit change son état de manière irréversible. La mesure dans la *base computationnelle* $\{|0\rangle, |1\rangle\}$ revient à observer un état comme $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, ce qui donne :

- le résultat 0 avec probabilité $|\alpha|^2$ (le qubit devient $|0\rangle$),
- le résultat 1 avec probabilité $|\beta|^2$ (le qubit devient $|1\rangle$).

Généralement, le calcul quantique se termine par une mesure afin de réduire l'état quantique dans l'un des états de base. Dans le circuit quantique, cette opération est représentée comme le montre la figure 1.4 .La mesure est symbolisée dans les circuits par une icône en forme de compteur ou de mètre. Elle effondre l'état du qubit vers l'un des états de base.

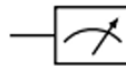


FIGURE 1.4 – Représentation de la mesure dans un circuit quantique (UMEANO, 2021).

Propriétés importantes :

- **Mesure différée** : une mesure peut souvent être reportée à la fin du circuit sans modifier le résultat final.
- **Mesure implicite** : les qubits non explicitement mesurés à la fin du circuit sont supposés l'être dans la base computationnelle.

1.3.5 Propriétés fondamentales du calcul quantique

L'informatique quantique se distingue fondamentalement de l'informatique classique grâce à des propriétés uniques issues des lois de la mécanique quantique (NATIONAL ACADEMIES OF SCIENCES, ENGINEERING, AND MEDICINE, 2019 ; NIELSEN & CHUANG, 2000). Ces propriétés confèrent aux ordinateurs quantiques un pouvoir de calcul inédit que l'on ne peut pas reproduire efficacement avec des systèmes classiques.

1.3.5.1 Parallélisme quantique

Le parallélisme quantique fait référence à la capacité d'un ordinateur quantique à évaluer simultanément une fonction sur un ensemble d'entrées en une seule opération, en exploitant la *superposition* et la linéarité des transformations unitaires.

Considérons une fonction $f : \{0, 1\}^n \rightarrow \{0, 1\}$. En appliquant l'opérateur unitaire U_f à la superposition uniforme de tous les états de base, on obtient

$$U_f \sum_{x \in \{0,1\}^n} |x\rangle|0\rangle = \sum_{x \in \{0,1\}^n} |x\rangle|f(x)\rangle \quad (1.3.3)$$

Une seule évaluation quantique suffit donc pour encoder les valeurs de f sur toutes les entrées, ce qui constitue la brique de base d'algorithmes comme Deutsch-Jozsa, Simon ou Grover (DEUTSCH & JOZSA, 1992 ; GROVER, 1996).

1.3.5.2 Superposition

La superposition décrit la capacité d'un système quantique à se trouver dans une combinaison linéaire d'états de base 1.2.2. Prenons, par exemple, un photon pouvant être polarisé Nord ou Est ; avant toute mesure, il peut exister dans une superposition de ces deux polarisations. L'expérience de pensée du chat de Schrödinger pousse cette idée à l'extrême, illustrant qu'un système peut conserver plusieurs issues macroscopiquement distinctes tant qu'aucune mesure n'est effectuée (SCHRÖDINGER, 1935).

1.3.5.3 Intrication (Entanglement)

Deux ou plusieurs qubits sont dits *intriqués* lorsque l'état global ne peut pas être factorisé en états individuels ; la connaissance de l'un détermine instantanément l'état des autres, quelle que soit la distance qui les sépare (BELL, 1964 ; EINSTEIN et al., 1935). L'état de Bell 1.3.2 en constitue l'exemple canonique. La mesure d'un qubit entraîne alors la disparition de la superposition et fixe corrélativement l'état du partenaire. L'intrication est le cœur de la téléportation quantique, de la cryptographie quantique (BB84, réseaux EPR) et des codes de correction d'erreurs (BENNETT et al., 1993).

1.3.5.4 Interférence

Lorsque plusieurs trajectoires d'amplitudes quantiques mènent au même état, celles-ci se combinent ; elles peuvent se renforcer (interférence constructive) ou s'annihiler (interférence destructive), à l'image des franges claires et sombres observées dans l'expérience des doubles fentes de Young (YOUNG, 1804). Dans les algorithmes quantiques, cette propriété est exploitée pour amplifier les probabilités des résultats souhaités et supprimer les indésirables : l'algorithme de Grover, par exemple, amplifie progressivement la probabilité de l'état solution jusqu'à l'obtenir avec une amplitude maximale, tandis que les trajectoires non solutions interfèrent destructivement (GROVER, 1996).

1.4 Algorithmes fondamentaux

Dans cette section, nous présentons quelques-uns des algorithmes quantiques les plus emblématiques, qui illustrent la supériorité potentielle du calcul quantique pour certaines classes de problèmes.

1.4.1 Algorithme de Deutsch

Proposé par David Deutsch en 1985, c'est l'un des premiers algorithmes montrant un avantage quantique (DEUTSCH, 1985). Il permet de déterminer, avec une seule évaluation de la fonction, si une fonction booléenne $f : \{0, 1\} \rightarrow \{0, 1\}$ est **constante** ou **équilibrée**.

Exemples :

- Fonction constante : $f(x) = 0$ ou $f(x) = 1$.
- Fonction équilibrée : $f(x) = x$ ou $f(x) = 1 \oplus x$.

L'algorithme commence avec l'état $|\psi\rangle = |0\rangle|1\rangle$. On applique la porte Hadamard H aux deux qubits, produisant une superposition. Ensuite, une porte unitaire U_f encode la fonction f comme suit :

$$U_f|x, y\rangle = |x, y \oplus f(x)\rangle \quad (1.4.1)$$

Puis on applique à nouveau H au premier qubit, avant de mesurer. Le résultat indique si f est constante (résultat 0) ou équilibrée (résultat 1), avec une seule évaluation de f . La Figure 1.5 illustre le circuit quantique associé à l'algorithme de Deutsch.

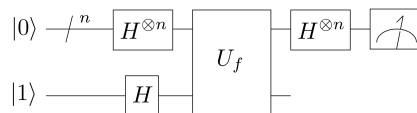


FIGURE 1.5 – Circuit quantique de l'algorithme de Deutsch (DEUTSCH, 1985).

1.4.2 Algorithme de Grover

Développé par Lov Grover en 1996, cet algorithme est destiné à la recherche dans une base de données non structurée (GROVER, 1996). Il permet de retrouver un élément parmi N avec une complexité $O(\sqrt{N})$, contre $O(N)$ pour un algorithme classique.

L'algorithme commence par la création d'une superposition uniforme sur tous les états possibles, puis applique de manière répétée un oracle (qui marque la solution) suivi d'une opération de diffusion qui amplifie les amplitudes des états solution. Ce processus est répété $O(\sqrt{N})$ fois. L'algorithme est utile dans des problèmes d'optimisation, de recherche exhaustive, ou d'analyse combinatoire.

1.4.3 Algorithme de Shor

Proposé par Peter Shor en 1994, cet algorithme résout efficacement le problème de factorisation d'entiers, menaçant des systèmes de cryptographie comme RSA (SHOR, 1994). Il repose sur :

- **Réduction à un problème de périodicité** : factoriser N revient à déterminer la période r d'une fonction $f(x) = a^x \bmod N$.
- **Utilisation de la Quantum Fourier Transform (QFT)** pour estimer efficacement cette période dans l'espace quantique.

L'algorithme fonctionne en temps polynomial et pourrait casser la sécurité de systèmes classiques, d'où le développement actuel de la cryptographie post-quantique.

1.4.4 Algorithmes NISQ :

Avec l'émergence de dispositifs quantiques bruyants à échelle intermédiaire NISQ, des algorithmes hybrides ont été proposés pour exploiter la puissance quantique malgré les limitations matérielles actuelles. Deux des plus importants sont : Variational Quantum Eigensolver (VQE) et Quantum Approximate Optimization Algorithm (QAOA).

VQE – Variational Quantum Eigensolver : est utilisé pour approximer l'état fondamental d'un Hamiltonien donné, un problème central en chimie quantique. Il combine un circuit quantique paramétré (appelé ansatz) avec un optimiseur classique pour ajuster les paramètres afin de minimiser l'énergie moyenne mesurée (PERUZZO et al., 2014).

QAOA – Quantum Approximate Optimization Algorithm : introduit par FARHI et al. (2014), a été conçu pour résoudre des problèmes d'optimisation combinatoire. Il utilise une alternance de circuits paramétrés basés sur l'Hamiltonien du problème et un Hamiltonien de mélange. Comme pour le VQE, un optimiseur classique ajuste les paramètres pour maximiser la probabilité d'obtenir la solution optimale.

Ces deux algorithmes incarnent l'approche hybride classique-quantique typique de l'ère NISQ, où la partie quantique explore des espaces de solutions complexes et la partie classique guide la convergence vers l'optimum.

1.5 Les ordinateurs quantiques

Un ordinateur quantique fonctionnel ne repose pas uniquement sur la création d'un matériel à faible taux d'erreur. Il nécessite également un écosystème logiciel complet comprenant des langages de programmation, des compilateurs, des simulateurs, des kits de développement (Software Development Kit (SDK)) ainsi que des solutions d'accès via le cloud.

Dans ce qui suit, nous présentons les principales catégories d'ordinateurs quantiques ainsi que leurs spécificités. Essentiellement, on distingue deux grandes familles :

- les **simulateurs quantiques**, destinés principalement à l'étude de phénomènes physiques complexes,
- les **ordinateurs quantiques à portes logiques**, plus proches du modèle universel et capables d'exécuter des algorithmes quantiques généraux (GONDRAN & GONDRAN, 2020).

1.5.1 Les simulateurs quantiques

Les simulateurs quantiques regroupent deux sous-catégories principales :

1. **Les simulateurs analogiques** (ou « ordinateurs à la Feynman ») : ils ont été conçus pour reproduire directement l'évolution de systèmes quantiques complexes, difficiles à simuler même sur les supercalculateurs classiques (GONDRAN & GONDRAN, 2020; RICHARD & FEYNMAN, 1982). Leur fonctionnement est comparable aux anciens ordinateurs analogiques classiques. Pour l'heure, aucune supériorité systématique par rapport aux supercalculateurs n'a été démontrée.

2. **Les ordinateurs adiabatiques**, popularisés par la société D-Wave : ils sont adaptés à une classe particulière d'algorithmes, dits *algorithmes de recuit quantique* (VERT, 2021). Leur principe repose sur une évolution lente et contrôlée du système depuis un état initial vers un état final représentant une solution approchée au problème étudié (EZRATTY, 2019). Bien qu'intéressants, ces dispositifs sont souvent qualifiés de « mi-quantiques, mi-classiques » (GONDRAN & GONDRAN, 2020).

1.5.2 Les ordinateurs quantiques à portes logiques

La deuxième grande catégorie repose sur le modèle universel d'ordinateur quantique. Ces machines utilisent des portes quantiques pour manipuler les qubits et peuvent, en principe, exécuter tout algorithme quantique (EZRATTY, 2019). Elles présentent un gain de puissance et de vitesse considérable par rapport aux simulateurs quantiques.

Cependant, leur utilisation reste aujourd'hui limitée :

- le nombre de qubits disponibles demeure faible (de l'ordre d'une centaine),
- le bruit quantique impacte fortement la fiabilité des résultats (GONDRAN & GONDRAN, 2020).

On distingue deux situations :

- les ordinateurs **sans correction d'erreur**, utilisés lorsque des solutions approchées suffisent,
- les ordinateurs **avec codes correcteurs d'erreurs**, indispensables pour des calculs exacts, par exemple la factorisation de grands entiers premiers. Dans ce cas, l'intégration d'un schéma de correction est primordiale afin d'obtenir des résultats corrects (GONDRAN & GONDRAN, 2020).

1.5.3 Simulation classique des circuits quantiques

En attendant des machines corrigées, les simulateurs classiques restent incontournables pour valider le matériel, explorer les algorithmes et tester la robustesse au bruit (NATIONAL ACADEMIES OF SCIENCES, ENGINEERING, AND MEDICINE, 2019). La mémoire requise croît comme 2^n amplitudes complexes ; simuler 50 qubits en double précision nécessite déjà ~ 2 Po (PEDNAULT et al., 2017). Les plus gros supercalculateurs atteignent aujourd'hui 50–60 qubits en *full-state*.

le tableau 1.1 présente un ensemble de simulateurs de circuits quantiques parmi les plus utilisés dans la communauté, en précisant leur capacité maximale en nombre de qubits, la possibilité d'exécution locale ainsi que le framework logiciel associé.

Simulateur	Qubits max.*	Exécution locale	Framework
qasm_simulator (Qiskit)	32	Oui	Python / Qiskit (TEAM, 2023)
aer_simulator (stabilizer)	63	Non	Qiskit Aer (TEAM, 2023)
QVM (Rigetti)	30	Non	PyQuil / Forest SDK (SMITH et al., 2016)
ProjectQ back-end	28–30	Oui	ProjectQ (STEIGER et al., 2018)
QDK full_state	30	Oui	Microsoft Q# / QDK (et al., 2018)
qsim (Google Cirq)	40 [†]	Oui	Cirq / Google (AI, 2020)
Qubit Toaster	> 40 [†]	Oui	Stand-alone C++ (KOŁODZIEJ, 2021)

* Limite indicative sur un poste de travail classique. [†] La limite dépend de la mémoire RAM et du parallélisme.

TABLE 1.1 – Quelques simulateurs de circuits quantiques largement utilisés.

1.6 Correction d'erreurs et tolérance aux fautes

Les systèmes quantiques sont extrêmement sensibles aux perturbations environnementales et aux imperfections de contrôle, ce qui entraîne des erreurs dans les calculs quantiques. Contrairement aux bits classiques, les qubits ne peuvent pas être clonés à volonté (théorème de non-clonage), ce qui rend la correction d'erreurs plus complexe.

Pour garantir un calcul fiable, même en présence de bruit, l'informatique quantique a développé des Quantum Error Correcting Codes (QECC), tels que le code de Shor, le code de Steane, ou encore les codes topologiques comme le code de surface (FOWLER et al., 2012; SHOR, 1995; STEANE, 1996).

Principes des codes correcteurs

- Les informations sont encodées de manière redondante sur plusieurs qubits physiques pour former un qubit logique.
- Des mesures de syndrome permettent de détecter les erreurs sans effondrer l'état logique.
- Les opérations correctrices sont ensuite appliquées en fonction du syndrome mesuré.

La mise en œuvre efficace de la correction d'erreurs nécessite également des architectures de calcul dites tolérantes aux fautes, qui permettent d'exécuter des algorithmes complets même avec des qubits imparfaits, à condition que le taux d'erreur reste en dessous d'un certain seuil (PRESKILL, 1998; TERHAL, 2015).

Conclusion

À travers les concepts fondamentaux abordés dans ce chapitre, il apparaît clairement que le calcul quantique repose sur des principes physiques radicalement différents de ceux du calcul classique. Des notions telles que la superposition, l'intrication et l'interférence confèrent aux ordinateurs quantiques un potentiel inédit pour résoudre des problèmes complexes.

L'intégration de ces propriétés avec des algorithmes adaptés et des mécanismes de correction d'erreurs constitue le socle sur lequel reposera la prochaine révolution technologique. Ce chapitre a ainsi fourni les bases théoriques nécessaires à la compréhension des méthodes d'optimisation qui seront explorées dans la suite de ce mémoire.

Les prochains chapitres s'attacheront à exploiter ces fondements pour aborder la conception automatique des circuits quantiques, en proposant des approches innovantes afin d'améliorer leur efficacité et leur robustesse.

Chapitre 2

Généralités sur les métaheuristiques bio-inspirées

Introduction

L'optimisation de problèmes complexes, souvent non linéaires, à grande dimension ou à contraintes multiples, constitue un enjeu majeur dans de nombreux domaines scientifiques et techniques. Dans ce contexte, les approches classiques atteignent rapidement leurs limites, tant en termes de performance que de flexibilité. C'est pourquoi les *métaheuristiques bio-inspirées* suscitent un intérêt croissant : elles offrent une alternative robuste et adaptable, capable de fournir des solutions de haute qualité, même approximatives, dans des espaces de recherche vastes et mal structurés. (BLUM & ROLI, 2003 ; TALBI, 2009)

Dans ce chapitre, nous présenterons les fondements théoriques des *métaheuristiques bio-inspirées*, en exposant leurs principales catégories, leurs mécanismes de fonctionnement ainsi que les principes spécifiques à chaque paradigme. Cette base conceptuelle vise à fournir les éléments nécessaires pour appréhender leur potentiel dans la résolution de problèmes complexes, à l'instar de ceux abordés dans les sections suivantes.

2.1 Classification des métaheuristiques bio-inspirées

Les métaheuristiques bio-inspirées désignent une famille de méthodes d'optimisation stochastiques fondées sur des mécanismes observés dans la nature, tels que l'évolution biologique, le comportement des essaims ou l'adaptation dynamique. Elles simulent des phénomènes naturels comme la sélection naturelle (dans les algorithmes évolutionnaires), ou les comportements collectifs (dans les algorithmes d'intelligence en essaim) (KATOCH et al., 2021).

Nous illustrons via la figure 2.1 comment les métaheuristiques bio-inspirées sont organisées en différentes approches.



FIGURE 2.1 – Classification des métaheuristiques bio-inspirées. (KATOCH et al., 2021)

Parmi l'ensemble des métaheuristiques bio-inspirées, certaines approches comme l'optimisation par essaims particulaires ou par colonies de fourmis connaissent un succès notable. Néanmoins, l'algorithme génétique (AG) occupe une place centrale dans ce travail. En effet, ses mécanismes d'évolution, sa capacité à explorer efficacement de grands espaces de recherche et son aptitude à traiter des objectifs multiples en font un outil particulièrement adapté à la problématique étudiée. Les sections suivantes présenteront donc les différentes familles de métaheuristiques, avec un accent particulier mis sur les AG, qui constitueront le fondement méthodologique de notre démarche.

2.1.1 Les algorithmes évolutionnaires :

L'*Algorithme Évolutionnaire (AE)* est une méthode d'optimisation inspirée des principes de la théorie de l'évolution biologique, traduite en langage algorithmique. Ces algorithmes opèrent sur une population de solutions potentielles, qu'ils font évoluer au fil des générations dans le but de trouver des solutions optimales ou satisfaisantes à un problème donné. (BÄCK & SCHWEFEL, 1993) Plus précisément, un algorithme évolutionnaire suit un cycle itératif basé sur les étapes suivantes :

- **Génération initiale** : Création aléatoire d'une population de structures (ou individus), chacune représentant une solution potentielle.
- **Évaluation** : Chaque structure est analysée via une fonction d'évaluation (ou *fitness*) qui mesure sa performance ou son aptitude à résoudre le problème.
- **Sélection** : Les structures les mieux évaluées sont sélectionnées pour engendrer la nouvelle génération. Cette étape favorise la survie des plus aptes.
- **Variation (croisement et mutation)** : Des opérateurs de variation sont appliqués aux structures sélectionnées pour créer de nouvelles solutions.
- **Remplacement** : La population est mise à jour, partiellement ou totalement, avec les nouvelles structures générées.

Ce processus n'est réellement efficace que s'il repose sur des fondations solides. Deux éléments sont particulièrement déterminants dans la réussite d'un algorithme évolutionnaire. Le premier concerne la représentation des solutions, qui doit être adaptée à la nature du problème et aux opérateurs de variation employés. Le second est la fonction d'évaluation, véritable moteur de l'évolution, qui guide la recherche en mesurant la qualité des solutions et en orientant l'algorithme vers les zones les plus prometteuses de l'espace de solutions (*Evolutionary Algorithms in Engineering Design Optimization*, 2022) .

Dans ce qui suit, nous abordons deux principales catégories d'algorithmes évolutionnaires à savoir l'algorithme génétique classique et l'algorithme génétique multi-objectifs.

2.1.1.1 Algorithme génétique classique :

L'*Algorithme Génétique (AG)* a été développé par John Holland en 1975 (HOLLAND, 1975 ; HOLLAND, 1984). Il constitue une méthode d'optimisation métaheuristique s'inspirant des principes de l'évolution biologique darwinienne. En reproduisant des mécanismes naturels tels que la sélection, le croisement et la mutation, ces algorithmes sont conçus pour résoudre efficacement une grande variété de problèmes complexes dans divers domaines. Leur capacité à explorer de vastes espaces de recherche tout en évitant les pièges des optima locaux en fait un outil particulièrement puissant pour la recherche de solutions

optimales (BARNIER & BRISSET, 1999). Nous présenterons ci-dessous les composants fondamentaux d'un algorithme génétique :

Gène : représente une unité élémentaire d'information, correspondant à un paramètre du problème (par exemple, une variable de décision).

Chromosome : séquence ordonnée de gènes constituant une solution candidate complète. Selon l'application, il peut être représenté sous forme de chaîne binaire, vecteur numérique ou autre structure adaptée.

au-delà d'une simple chaîne binaire, un chromosome peut encoder un circuit quantique de plusieurs façons selon le niveau d'abstraction recherché et les contraintes matérielles. Les schémas les plus courants sont :

- **Séquence linéaire de portes (gate list)** : Chaque gène code une porte G (type, qubits cibles/contrôle, paramètres d'angle), et le chromosome est la concaténation ordonnée de ces gènes :

$$C = (g_1, g_2, \dots, g_L), \quad g_i = (G_i, q_i^{\text{ctrl}}, q_i^{\text{tgt}}, \theta_i)$$

où L est la longueur du circuit. Utilisée pour la synthèse directe d'unitaires ou d'états cibles (CREEVEY et al., 2023 ; MIRANDA et al., 2023).

- **Encodage par blocs / couches (modulaire)** : Le circuit est structuré en une séquence de blocs unitaires :

$$C = (B_1, B_2, \dots, B_K), \quad B_j = \prod_{i=1}^{m_j} G_{j,i}$$

où B_j regroupe un ensemble local de portes. Favorise des croisements structurés (échange de blocs) (POTOČEK et al., 2018).

- **Grille $n \times m$ (qubits $\times$ profondeur)** : Un individu est représenté comme une matrice :

$$C = [c_{i,j}] \in \mathbb{G}^{n \times m}, \quad c_{i,j} \in \{\text{porte}, \emptyset\}$$

où n est le nombre de qubits, m la profondeur, et chaque cellule $c_{i,j}$ décrit une opération (ou vide). Bien adapté aux contraintes NISQ (SÜNKEL et al., 2023).

Individu : désigne une solution candidate au sein de la population. Dans le contexte des AG, un individu est équivalent à un chromosome, mais le terme « individu » insiste sur son appartenance à la population évolutive.

Population : Ensemble d'individus sur lequel s'opère l'évolution. Sa taille influence l'exploration (diversité) et l'efficacité computationnelle. L'initialisation est souvent aléatoire pour couvrir un large espace de recherche.

Croisement : Le croisement consiste à combiner deux parents pour produire un ou plusieurs nouveaux individus (fils), dans le but de mélanger leurs caractéristiques et ainsi explorer de nouvelles combinaisons de solutions. Cette opération repose souvent sur des mécanismes probabilistes qui déterminent la contribution de chaque parent à l'enfant. Dans le cadre de la synthèse de circuits quantiques, cela revient à combiner des séquences de portes afin de générer des circuits potentiellement plus performants.

L'objectif est de réutiliser des blocs efficaces tout en introduisant de la diversité pour améliorer la qualité globale de la population.

Plusieurs types de croisements sont couramment utilisés :

- **Croisement à k -points** : une généralisation du croisement à un point où plusieurs points de coupure sont choisis. Les segments sont alors alternés entre les parents. Cette variante introduit une recombinaison plus riche mais peut parfois perturber des schémas génétiques favorables (MITCHELL, 1998).
- **Croisement uniforme** : chaque gène de l'enfant est choisi aléatoirement à partir de l'un des deux parents, selon une probabilité fixée (souvent 50/50). Ce schéma favorise un mélange très diversifié mais peut détruire des structures globales utiles (SYSWERDA, 1989).
- **Croisement par ordre** : utilisé principalement pour les problèmes de permutation. Une sous-séquence de gènes est copiée directement du premier parent, tandis que les autres positions sont complétées en respectant l'ordre relatif des gènes du second parent. Cela garantit la validité de la permutation tout en préservant une partie de la structure de chaque parent (GOLDBERG, 1989).
- **Croisement par cycle** : les gènes sont hérités de manière cyclique entre les deux parents. Ainsi, chaque position dans l'enfant provient d'un seul parent, en suivant une logique de cycles. Ce type de croisement est particulièrement adapté aux représentations permutationnelles (GOLDBERG, 1989).
- **Croisement à appariement partiel** : largement utilisé dans les problèmes combinatoires comme le voyageur de commerce. Une portion de la séquence est copiée directement du premier parent, puis les gènes restants sont remplis en respectant une correspondance entre les deux parents. Ce mécanisme permet de préserver certaines positions clés tout en maintenant une permutation valide (GOLDBERG & LINGLE, 1985).

Ces différentes variantes permettent d'équilibrer la préservation des bonnes solutions déjà découvertes et l'exploration de nouvelles combinaisons génétiques.

Mutation : La mutation constitue un mécanisme central des algorithmes génétiques, jouant un rôle crucial dans le maintien de la diversité génétique au sein de la population. Elle agit en introduisant des changements aléatoires dans certains individus, ce qui permet de perturber légèrement leurs structures génétiques. Cette opération vise à éviter la convergence prématurée vers des solutions sous-optimales et à explorer de nouvelles régions de l'espace de recherche qui n'ont pas encore été visitées.

L'efficacité de la mutation repose en grande partie sur un réglage adapté de son taux. Un taux trop élevé risque de transformer l'algorithme en une simple recherche aléatoire, tandis qu'un taux trop faible peut mener à la stagnation dans un optimum local. Ainsi, ce paramètre clé est généralement maintenu à une valeur faible afin d'éviter des perturbations excessives, tout en assurant une exploration suffisante de l'espace des solutions.

Plusieurs types de mutation sont couramment utilisés dans la littérature :

- **Mutation par déplacement** : un gène est extrait de sa position et inséré ailleurs dans la séquence, modifiant l'ordre des éléments.
- **Mutation par inversion** : une portion de la séquence est inversée, ce qui change localement l'ordre des gènes.

- **Mutation par brouillage (scramble)** : les gènes d'un sous-ensemble choisi aléatoirement sont mélangés de manière aléatoire.
- **Mutation inversée** : variante où certains gènes sont remplacés par leurs valeurs inverses (utile si les représentations le permettent, par exemple codage binaire).

Ces différentes approches permettent d'introduire des perturbations variées, plus ou moins fortes, afin d'équilibrer l'exploration et l'exploitation au sein de l'algorithme.

Selection : La sélection est une étape clé dans le fonctionnement d'un algorithme génétique. Elle détermine quels individus seront retenus pour participer aux opérations de croisement (*sélection des parents*) et lesquels seront conservés dans la génération suivante (*sélection des survivants*). Cette étape doit assurer un équilibre entre deux objectifs contradictoires :

- **Exploitation** : favoriser les individus ayant la meilleure fitness, afin de guider rapidement la recherche vers des solutions de qualité.
- **Exploration** : maintenir une diversité génétique suffisante pour éviter la convergence prématurée vers des minima locaux.

Plusieurs types de stratégies de sélection sont couramment utilisées dans la littérature.

- La *sélection par roulette* attribue une probabilité proportionnelle à la fitness de chaque individu.
- La *sélection par rang* classe les individus et répartit les chances de manière plus équilibrée.
- La *sélection par tournoi* consiste à tirer aléatoirement un petit groupe d'individus et à retenir le meilleur.
- La *sélection de Boltzmann* repose sur un mécanisme inspiré de la physique statistique : la probabilité de sélection dépend d'une fonction exponentielle de la fitness et d'une « température » qui décroît au fil des générations, ce qui permet d'équilibrer exploration au début et exploitation par la suite.
- Enfin, la *sélection stochastique universelle* (Stochastic Universal Sampling (SUS)) introduit plusieurs pointeurs équidistants pour rendre la sélection plus régulière.

Ces différentes approches seront détaillées dans le chapitre 4.

La figure 2.2 présente les différents types d'opérateurs évoqués précédemment : croisement, mutation et sélection.

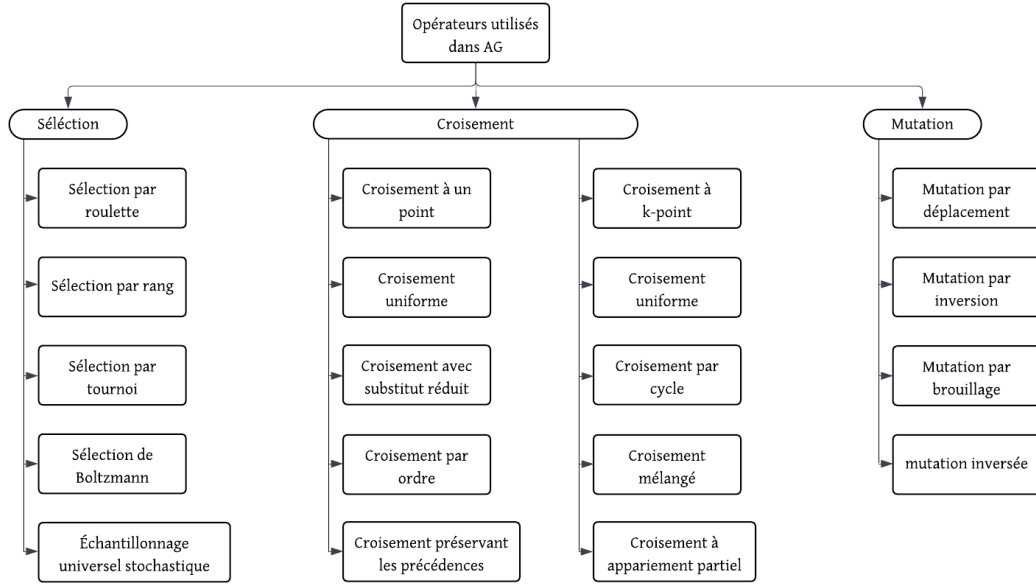


FIGURE 2.2 – Les opérateurs utilisés dans l'AG. (KATOCH et al., 2021)

Fonction de fitness : Fonction qui mesure la qualité d'une solution candidate en indiquant dans quelle mesure elle satisfait l'objectif fixé. Elle guide l'évolution en classant les individus, permettant de privilégier les meilleurs lors de la sélection. Cette fonction est définie selon le problème étudié et peut correspondre, par exemple, à la minimisation d'un coût ou à l'amélioration d'une performance. Dans le cas particulier de la synthèse de circuits quantiques, l'aptitude correspond à la fidélité avec laquelle un circuit candidat C reproduit le comportement d'un circuit cible G , ce qui permet de comparer les différentes solutions générées par l'algorithme. Les principales familles de fonctions de fitness utilisées sont :

- **Fidélité état (State-vector fidelity) :** Utilisée lorsque l'on vise un état cible $|\psi_{\text{target}}\rangle$.

$$F_{\text{state}} = \left| \langle \psi_{\text{target}} | \psi_C \rangle \right|^2$$

Avantages : intuitive, directement liée à la probabilité de retrouver l'état cible.

Inconvénients : dépend de l'état d'entrée choisi ; ne capture pas toute l'unitaire. (CREEVEY et al., 2023)

- **Fidélité unitaire (Trace overlap / Unitary fidelity) :** Comparaison globale entre deux opérateurs unitaires U_G (cible) et U_C (candidat).

$$F_{\text{unitary}} = \frac{1}{2^n} \left| \text{Tr} \left(U_G^\dagger U_C \right) \right|$$

Avantages : évalue l'équivalence fonctionnelle complète du circuit. *Inconvénients :* coûteux en calcul matriciel pour n grand. (POTOČEK et al., 2018)

Processus évolutif : Comme illustré dans la **Figure 2.3**, l'algorithme génétique commence par une population initiale à la génération k . Lors de la phase de *reproduction*, certains individus P sont sélectionnés selon une probabilité donnée. Deux parents, notés $P1$ et $P2$, sont choisis pour participer à l'opération de croisement (*crossover*), réalisée avec une probabilité P_c . Celle-ci génère de nouveaux descendants,

notés $C1$ et $C2$. Parallèlement, un individu P peut subir une *mutation* avec une probabilité P_m , donnant naissance à un nouvel individu modifié P' . L'ensemble des individus issus du croisement ($C1$, $C2$) et de la mutation (P'), ainsi que les individus non modifiés, passent ensuite par une phase d'*évaluation*. Cette étape consiste à calculer leur valeur d'aptitude (*fitness*) afin de déterminer leur qualité.

Enfin, les meilleurs individus sont retenus pour former la population de la génération suivante $k + 1$. Ce cycle est répété jusqu'à atteindre un critère d'arrêt défini, tel qu'un nombre maximal de générations ou une performance cible (BARNIER & BRISSET, 1999).

La figure 2.3 décrit le déroulement de l'AG.

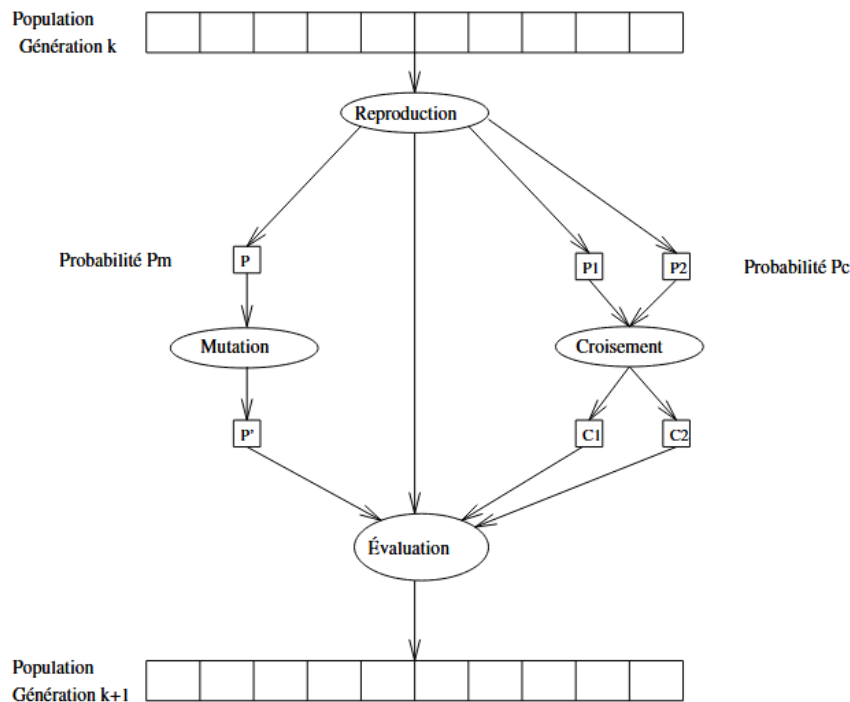


FIGURE 2.3 – Principe général de l'AG. (BARNIER & BRISSET, 1999)

2.1.1.2 Algorithme génétique multi-objectif (NSGA-II) :

NSGA-II est une extension de l'algorithme génétique classique, adapté à l'optimisation simultanée de plusieurs objectifs souvent contradictoires. Il conserve les mécanismes de base des algorithmes génétiques — sélection, croisement, mutation — mais se différencie principalement par la manière dont la fonction de fitness est évaluée. (KATOCH et al., 2021). *NSGA-II* repose sur la notion de dominance de Pareto pour classer les solutions, et vise à identifier un ensemble de solutions non dominées appelé front de Pareto, représentant différents compromis entre les objectifs. Pour éviter la convergence prématurée vers un seul optimum, ces algorithmes intègrent souvent des mécanismes de préservation de la diversité, tels que la crowding distance (ABDOU et al., 2013).

1. Dominance de Pareto

La base de comparaison entre solutions dans un *NSGA-II* repose sur la *dominance de Pareto*. Une solution A domine une solution B si :

- A est au moins aussi bonne que B sur tous les objectifs ;
- A est strictement meilleure que B sur au moins un objectif.

Une solution est dite *non dominée* si aucune autre ne la domine. Le **front de Pareto** est alors l'ensemble de toutes les solutions non dominées de la population. L'algorithme cherche à s'approcher et à densifier ce front au fil des générations (DEB, 1999).

2. Tri non dominé

Dans *NSGA-II*, les solutions sont classées en niveaux de dominance :

- Le premier niveau contient les solutions non dominées (front 1).
- Une fois celles-ci retirées, on extrait les non dominées restantes (front 2), etc.

Ce classement permet une sélection préférentielle des individus dans les meilleurs fronts (DEB, 1999).

3. Distance de densité (crowding distance)

Pour maintenir une bonne diversité des solutions sur le front de Pareto, *NSGA-II* utilise une métrique appelée **distance de densité**, qui évalue l'isolement d'un individu dans l'espace des objectifs. Cette distance permet de favoriser les solutions éloignées de leurs voisines (et donc de couvrir uniformément le front).

Formule de distance de densité : (IMANI et al., 2024)

$$d_i = \sum_{m=1}^M \frac{f_{i+1}^{(m)} - f_{i-1}^{(m)}}{f_{\max}^{(m)} - f_{\min}^{(m)}} \quad (2.1.1)$$

où :

- $f_{i+1}^{(m)}$ et $f_{i-1}^{(m)}$ sont les valeurs du m^e objectif pour les voisins de l'individu i (dans un front trié par objectif),
- $f_{\max}^{(m)}$ et $f_{\min}^{(m)}$ sont les valeurs extrêmes du m^e objectif dans le front.
- M est le nombre total d'objectifs.

Les solutions situées aux extrémités du front de Pareto (celles qui atteignent, pour au moins un objectif, la valeur $f_{\min}^{(m)}$ ou $f_{\max}^{(m)}$) se voient attribuer une distance infinie afin d'être toujours conservées, car elles définissent l'étendue du front.

4. Sélection avec tournoi et distance de densité

La sélection dans *NSGA-II* s'effectue par *tournoi binaire* entre deux individus (IMANI et al., 2024) :

- Si l'un appartient à un front de dominance inférieur, il est choisi ;
- Si les deux sont dans le même front, celui avec la plus grande **distance de densité** est retenu.

Cela assure un équilibre entre convergence vers le front et diversité des solutions.

2.1.2 Les algorithmes d'intelligence en essaim :

Swarm Intelligence (SI) est un paradigme d'optimisation inspiré des comportements collectifs observés dans la nature (colonies de fourmis, essaims d'abeilles, bancs de poissons), dans lequel des agents simples interagissent localement, sans contrôle centralisé, pour faire émerger une solution globale. Grâce à ses propriétés d'auto-organisation, de coopération et de décentralisation, elle permet de concevoir des méthodes heuristiques efficaces pour l'exploration de grands espaces de recherche, en assurant un bon compromis entre exploration et exploitation (BENKIRAT, 2019 ; KUMAR & PATHAK, 2019). Nous présentons ci-dessous les principes de ce paradigme :

Coordination : Les agents doivent échanger des informations sur leur environnement local afin d’agir collectivement de manière cohérente vers un objectif commun.

Coopération : Les agents doivent unir leurs efforts pour accomplir des tâches complexes qu’ils ne peuvent résoudre individuellement.

Diversité de réponse : Les ressources et agents sont répartis de manière à éviter toute concentration excessive, assurant ainsi une plus grande résilience face aux variations de l’environnement.

Stabilité : Le système maintient une certaine constance de comportement malgré des perturbations environnementales mineures.

Adaptabilité : La dynamique collective évolue en réponse aux changements significatifs dans l’environnement, permettant au système de s’ajuster de manière efficace.

Auto-organisation : Les comportements collectifs émergent des interactions simples et locales entre les agents, sans nécessiter de contrôle centralisé.

La tableau 2.1 présente les algorithmes les plus importants d’intelligence en essaim

Algorithmes	Auteur	Année
Optimisation par colonie de fourmis	Dorigo, M, et al.	1991
Optimisation par essais particuliers	Eberhart & Kennedy (1995)	1995
Optimisation par colonie d’abeilles	Dervis Karaboga (2005)	2005
Gouttes d’eau intelligentes	Shah-Hosseini	2007
Algorithmes des lucioles	Xin-She Yang (2008)	2008
Recherche du Coucou	Xin-She Yang et Suash Deb	2009
Algorithme des chauves-souris	Xin-She Yang	2010

TABLE 2.1 – Algorithmes SI importants (KUMAR & PATHAK, 2019)

Deux principales catégories d’algorithmes d’intelligence en essaim sont présentées ci-dessous :

2.1.2.1 Algorithme de colonie de fourmis (ACO) :

L’ACO est une méthode d’optimisation inspirée du comportement social des fourmis, notamment leur manière de découvrir et renforcer les chemins les plus courts vers une ressource à l’aide de marques chimiques appelées phéromones. Proposée initialement par Marco Dorigo dans les années 1990, cette approche modélise un ensemble d’agents coopératifs (les “fourmis artificielles”) qui explorent un espace de solutions et s’appuient sur des mécanismes d’apprentissage indirect. L’ACO est particulièrement efficace pour les problèmes d’optimisation combinatoire. Elle a ensuite été enrichie par des variantes hybrides, notamment en combinant des techniques classiques et quantiques pour élargir ses capacités de résolution dans des contextes complexes ou contraints (BLUM, 2024 ; de ANDOIN & ECHANOBÉ, 2022).

Les principes fondamentaux régissant cet algorithme sont présentés dans ce qui suit :

Construction des solutions : Un ensemble de "fourmis artificielles" construit des solutions en explorant un graphe représentant le problème à résoudre. Chaque nœud

du graphe peut être interprété comme un choix ou une étape dans la solution.

Utilisation des phéromones : Les fourmis sélectionnent les étapes suivantes en fonction d'une probabilité influencée par la quantité de phéromones déposées sur les chemins disponibles ainsi que par une fonction heuristique locale.

Mise à jour des phéromones : Après chaque itération, les niveaux de phéromones sont ajustés afin de renforcer les chemins correspondant aux solutions de meilleure qualité et d'affaiblir ceux associés aux solutions moins performantes, orientant ainsi l'exploration vers des régions prometteuses de l'espace de recherche.

Évaporation des phéromones : Un mécanisme d'évaporation réduit progressivement les niveaux de phéromones sur les chemins, permettant d'éviter une convergence prématurée vers un optimum local.

Dans ce qui suit, nous décrivons les étapes clés de l'ACO (de ANDOIN & ECHANOBE, 2022) :

1. **Initialisation :**

- **Paramètres :** Définir les paramètres principaux tels que :
 - m : nombre de fourmis.
 - ρ : taux d'évaporation des phéromones.
 - α : poids de l'influence des phéromones.
 - β : poids de l'information heuristique.
- **Traçage initial :** Initialiser les niveaux de phéromones τ_{ij} sur tous les chemins possibles à une valeur de base τ_0 .

2. **Construction des solutions :**

- **Déplacement des fourmis :** Chaque fourmi commence à une position initiale et construit une solution pas à pas.
- Le choix de l'étape suivante est basé sur une règle probabiliste (DORIGO & SOCHA, 2018) :

$$p_{ij} = \begin{cases} \frac{\tau_{ij}^\alpha \cdot \eta_{ij}^\beta}{\sum_{k \in N_i} \tau_{ik}^\alpha \cdot \eta_{ik}^\beta}, & \text{si } j \in N_i \\ 0, & \text{sinon} \end{cases} \quad (2.1.2)$$

où :

- p_{ij} est la probabilité de choisir le chemin de i vers j ,
- τ_{ij} est la quantité de phéromones sur le chemin (i, j) ,
- η_{ij} est l'information heuristique (par exemple, $\eta_{ij} = \frac{1}{d_{ij}}$ pour le TSP),
- N_i est l'ensemble des nœuds encore accessibles depuis i .

3. **Évaluation des solutions :**

- Chaque solution construite par une fourmi est évaluée selon une fonction objectif spécifique au problème traité.

4. **Mise à jour des phéromones :**

- **Évaporation globale :**

$$\tau_{ij} \leftarrow (1 - \rho) \cdot \tau_{ij} \quad (\text{DORIGO \& SOCHA, 2018}) \quad (2.1.3)$$

— **Renforcement des bonnes solutions :**

$$\tau_{ij} \leftarrow \tau_{ij} + \Delta\tau_{ij} \quad (\text{DORIGO \& SOCHA, 2018}) \quad (2.1.4)$$

avec $\Delta\tau_{ij}$ proportionnelle à la qualité de la solution (par exemple, $\Delta\tau_{ij} = \frac{Q}{L}$, où L est la longueur de la solution et Q une constante).

5. **Condition d'arrêt :**

- Répéter les étapes 2 à 4 jusqu'à ce que l'un des critères suivants soit atteint :
 - un nombre maximal d'itérations,
 - ou la convergence vers une solution satisfaisante.

La figure 2.4 décrit le déroulement de l'ACO.

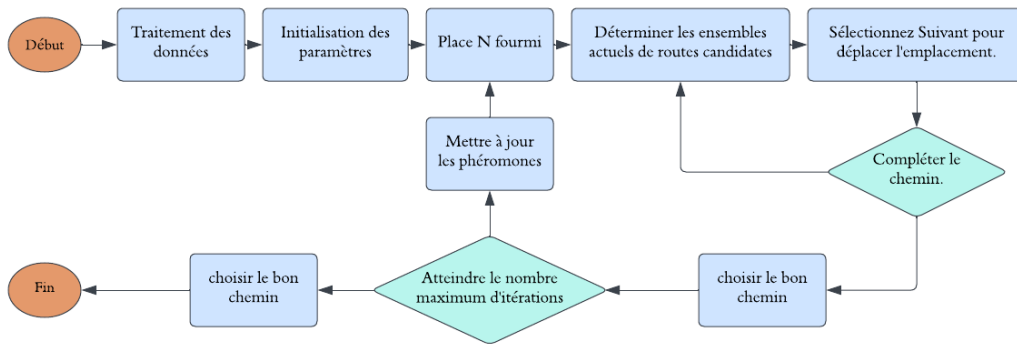


FIGURE 2.4 – Les étapes de déroulement de ACO

2.1.2.2 Algorithme d'optimization par essaim de particules (PSO) :

L'algorithme *PSO* est une méthode d'optimisation inspirée par le comportement de groupe observé chez des animaux tels que les oiseaux et les poissons. Chaque solution potentielle est modélisée sous forme de particule évoluant dans l'espace de recherche. Ces particules adaptent leur trajectoire selon leur propre expérience et celle des autres particules, dans le but de converger vers une solution optimale (KUMAR & PATHAK, 2019; SHAMI et al., 2022).

Les paramètres principaux de cet algorithme sont :

Taille de l'essaim (Swarm Size) : Nombre total de particules dans l'algorithme. Une grande taille permet d'explorer plus largement l'espace de recherche, mais augmente les coûts de calcul et le temps d'exécution (KUMAR & PATHAK, 2019).

Position de la particule (x_i) : Coordonnées représentant la position actuelle d'une particule dans l'espace de recherche. Cette position constitue une solution candidate à évaluer (KUMAR & PATHAK, 2019).

Vitesse (v_i) : Vecteur qui détermine la direction et la vitesse avec lesquelles une particule se déplace dans l'espace de recherche. Il est mis à jour à chaque itération pour guider l'exploration (KUMAR & PATHAK, 2019).

Meilleur personnel ($pbest_i$) : Position antérieure qui a donné la meilleure valeur de la fonction objectif pour la particule i . Elle sert de mémoire individuelle pour orienter les futurs déplacements (KUMAR & PATHAK, 2019).

Meilleur global ($gbest$) : Position globale de l'ensemble des particules qui a fourni la meilleure valeur de la fonction objectif jusqu'à présent. Elle guide collectivement toutes les particules (KUMAR & PATHAK, 2019).

Poids d'inertie (w) : Coefficient appliqué à la vitesse précédente. Il joue un rôle central dans l'équilibre entre l'exploration de nouvelles zones et l'exploitation des meilleures zones connues (SHAMI et al., 2022).

Coefficients d'accélération (c_1, c_2) : c_1 représente l'attraction vers le meilleur personnel (composante cognitive), tandis que c_2 mesure l'influence du meilleur global (composante sociale). Leur bonne calibration influence fortement la performance (SHAMI et al., 2022).

Fonction objectif $f(x)$: Fonction mathématique définie en fonction du problème à résoudre. Elle évalue la qualité d'une solution donnée et oriente l'optimisation (KUMAR & PATHAK, 2019).

L'algorithme PSO suit les étapes suivantes :

1. **Initialisation** : Positions et vitesses des particules sont générées aléatoirement. Chaque particule évalue sa solution, initialise $pbest_i$, et l'algorithme identifie le $gbest$ initial (SHAMI et al., 2022).

2. **Mise à jour de la vitesse** :

$$v_i(t+1) = w \cdot v_i(t) + c_1 \cdot r_1 \cdot (pbest_i - x_i(t)) + c_2 \cdot r_2 \cdot (gbest - x_i(t)) \quad (2.1.5)$$

avec r_1 et r_2 des valeurs aléatoires dans $[0, 1]$ (KUMAR & PATHAK, 2019; SHAMI et al., 2022).

3. **Mise à jour de la position** :

$$x_i(t+1) = x_i(t) + v_i(t+1) \quad (\text{KUMAR \& PATHAK, 2019}) \quad (2.1.6)$$

4. **Évaluation** : On réévalue $f(x_i)$. Mise à jour de $pbest_i$ et de $gbest$ si de meilleures solutions sont trouvées (SHAMI et al., 2022).
5. **Critère d'arrêt** : L'algorithme s'arrête après un nombre d'itérations donné ou si la convergence est atteinte (SHAMI et al., 2022).

La figure 2.5 décrit le déroulement de l'algorithme de PSO.

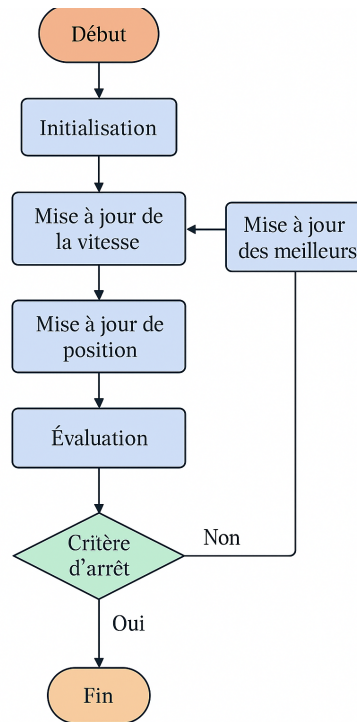


FIGURE 2.5 – Les étapes de déroulement de PSO

2.2 Justification du choix de l’algorithme génétique

Dans ce chapitre, nous avons présenté un panorama des principales métaheuristiques bio-inspirées, telles que les algorithmes évolutionnaires et les méthodes d’intelligence en essaim. L’objectif de cette présentation élargie est double :

- offrir une vision d’ensemble qui situe l’algorithme génétique parmi d’autres approches possibles,
- mettre en évidence les spécificités qui motivent son adoption dans le cadre de notre travail.

Le choix de l’algorithme génétique (AG) se justifie par plusieurs atouts majeurs. Sa robustesse lui permet d’explorer efficacement de vastes espaces de recherche et d’éviter la convergence prématurée vers des solutions locales. Sa flexibilité de représentation rend son adaptation aisée à divers types de problèmes, y compris ceux liés à la conception de circuits quantiques. Enfin, ses mécanismes intrinsèques (sélection, croisement, mutation) facilitent naturellement le passage à une optimisation multi-objectif, indispensable dans des contextes où plusieurs critères doivent être pris en compte simultanément.

Ainsi, même si d’autres métaheuristiques comme PSO ou ACO présentent des performances intéressantes, l’AG apparaît comme la méthode la plus pertinente et la mieux adaptée à notre problématique. C’est pourquoi il constituera le cœur méthodologique de la suite de ce travail.

Conclusion

Dans ce chapitre, nous avons introduit les métaheuristiques bio-inspirées en présentant leurs principales familles, notamment les algorithmes évolutionnaires et ceux issus de

l'intelligence en essaim. Nous nous sommes particulièrement focalisés sur les algorithmes génétiques, dans leur version classique et multi-objectif, car notre approche d'optimisation pour la conception de circuits quantiques repose directement sur ces méthodes. Cette base nous permettra, dans le chapitre suivant, d'explorer plus en détail comment ces algorithmes peuvent être exploités pour générer des circuits quantiques optimisés.

Chapitre 3

Algorithmes évolutionnaires et conception automatique de circuits quantiques

Introduction

Ce chapitre présente une analyse structurée de ces travaux, depuis les premières applications des AG à la synthèse de circuits quantiques jusqu’aux contributions récentes intégrant des critères multiples. On distingue ainsi deux grandes orientations : d’une part les approches dites mono-objectif, centrées principalement sur la génération de circuits corrects, et d’autre part les approches multi-objectifs, qui visent à optimiser simultanément plusieurs critères tels que la fidélité, la profondeur, le coût structurel ou encore la robustesse face aux contraintes matérielles.

3.1 Rôle des algorithmes évolutionnaires dans la conception des circuits quantiques

L’application des algorithmes évolutionnaires à la conception de circuits quantiques constitue une approche prometteuse pour répondre aux défis liés à la complexité croissante des systèmes quantiques. Ces méthodes permettent non seulement de générer automatiquement des circuits adaptés à des comportements cibles, mais aussi d’optimiser des architectures en maximisant certaines propriétés essentielles telles que l’intrication. À titre d’illustration, l’étude menée par Shailendra Bhandari et ses collaborateurs (BHANDARI et al., 2024) met en évidence deux axes principaux d’application des algorithmes évolutionnaires dans ce domaine :

- **Premier axe** : la synthèse de circuits quantiques destinés à reproduire des comportements spécifiques, en particulier ceux d’automates cellulaires déterministes et stochastiques. Les algorithmes évolutionnaires sont utilisés pour assembler automatiquement des circuits dont la performance est évaluée à l’aide de la divergence de Kullback-Leibler¹, mesurant ainsi la proximité entre les distributions de sortie obtenues et les comportements cibles souhaités.
- **Deuxième axe** : l’optimisation de circuits quantiques afin de générer des états fortement intriqués, élément fondamental pour le traitement de l’information quantique. L’approche repose sur l’amélioration de la structure des circuits en maximisant

1. **La divergence de Kullback-Leibler (KL)** mesure la différence entre deux distributions de probabilité. Dans ce contexte, elle compare la distribution des sorties du circuit aux comportements cibles. Plus la valeur est proche de 0, plus le circuit est fidèle au modèle attendu.

des mesures d'intrication telles que la mesure de Meyer-Wallach² et l'entropie de von Neumann³, permettant ainsi la conception de circuits compacts et performants, adaptés aux exigences des systèmes quantiques actuels.

3.2 Origines de l'application des algorithmes génétiques en circuits quantiques

L'application des algorithmes génétiques (AG) à la conception automatique de circuits quantiques a émergé à la fin des années 1990 et au début des années 2000, à un moment où la complexité croissante des systèmes quantiques rendait difficile la conception manuelle de circuits efficaces. L'une des premières démonstrations concrètes de l'application des algorithmes génétiques à la conception automatique de circuits quantiques est le travail de **Yabuki et Iba (2000)**. Les auteurs y présentent une méthode originale utilisant les algorithmes génétiques (AG) pour la conception automatique de circuits de téléportation quantique. Leur approche repose sur une structure génétique qui reflète les étapes fonctionnelles du protocole : génération de la paire **EPR**⁴, opérations d'Alice, et corrections de Bob. En utilisant uniquement des contraintes élémentaires (type de portes limité, division en étapes bien définies), leur algorithme a été capable de générer un circuit correct contenant seulement huit portes. Ce résultat est notable car il améliore le circuit de référence de Brassard, qui en contenait onze.

La qualité des circuits générés est mesurée à l'aide d'une fonction d'erreur qui compare les amplitudes du qubit téléporté (cf. Eq. 1.2.2). Cette fonction est définie par :

$$\text{error}_j = \frac{1}{n} \sum_{i=0,2,4,6} \left| \frac{a_{j,i}}{a_{j,i+1}} - \frac{p}{q} \right| \quad (3.2.1)$$

où $a_{j,i}$ représente les amplitudes mesurées et $\frac{p}{q}$ le ratio souhaité du qubit téléporté. L'intérêt de cette méthode réside dans le fait qu'il est possible d'évoluer un circuit fonctionnel sans fournir explicitement la matrice unitaire cible. Il suffit de spécifier le comportement attendu à la sortie, ce qui rend l'approche flexible dans les cas où les spécifications sont incomplètes ou complexes à modéliser mathématiquement.

Quelques années plus tard, **Lukac et Perkowski (2002)** ont ensuite généralisé cette approche à la synthèse de circuits quantiques arbitraires. Leur travail repose sur un AG classique, mais introduit un encodage très efficace pour représenter les circuits sous forme de chromosomes. Chaque circuit est représenté comme une séquence de blocs parallèles contenant des portes quantiques élémentaires (Hadamard, CNOT, Swap, etc.). Cette structure permet d'évaluer rapidement les matrices unitaires correspondantes, réduisant considérablement le temps de calcul.

Dans ce cadre, la méthode vise à reproduire une matrice unitaire cible, c'est-à-dire une matrice unitaire représentant l'opération quantique que le circuit doit approximer. En

2. **Mesure de Meyer-Wallach** est un indicateur global d'intrication pour un système multi-qubits. Sa valeur varie entre 0 (absence d'intrication) et 1 (intrication maximale), ce qui permet d'évaluer la capacité d'un circuit à générer des états quantiques intriqués.

3. **Entropie de von Neumann** mesure l'incertitude d'un sous-système quantique et sert ainsi à quantifier l'intrication entre différentes parties d'un système. Une entropie nulle correspond à un état pur non intriqué, tandis qu'une entropie positive traduit la présence d'intrication.

4. **EPR** : Einstein-Podolsky-Rosen. Désigne une paire de qubits intriqués, utilisée dans les protocoles de téléportation quantique.

d'autres termes, la matrice unitaire cible décrit la transformation idéale à appliquer sur un registre de qubits, tandis que l'algorithme cherche à construire un circuit qui en soit la réalisation la plus fidèle possible. La fonction de fitness est donc fondée sur la comparaison élément par élément entre la matrice cible et celle obtenue à partir du chromosome. Elle est donnée par la formule suivante :

$$F = \frac{1}{1 + \Lambda + \sum_{i,j} |U_{ij} - S_{ij}|} \quad (3.2.2)$$

où U est la matrice unitaire cible, S celle du circuit candidat, et Λ un facteur pénalisant les circuits trop longs (en nombre de blocs parallèles). Cette formulation permet d'encourager à la fois la justesse et la compacité du circuit.

Ce travail se distingue par sa généralité : il ne se limite pas à un problème particulier comme la téléportation, mais peut s'appliquer à tout type de circuit quantique, pourvu qu'on en connaisse la matrice unitaire. De plus, les résultats expérimentaux montrent que l'approche converge vers des solutions efficaces en un temps raisonnable, même pour des circuits à trois ou quatre qubits.

3.3 Typologie des travaux réalisés

Dans ce qui suit, nous présentons une synthèse de l'état de l'art portant sur les applications des algorithmes évolutionnaires pour la conception automatique de circuits quantiques.

3.3.1 Synthèse automatique de circuits :

Giraldi et al. (2004) proposent une étude approfondie sur l'application des algorithmes génétiques (AG) au calcul quantique, tout en ouvrant des perspectives vers les algorithmes génétiques quantiques (AGQs). Leur travail se distingue par deux contributions principales.

D'une part, ils introduisent une méthode originale d'apprentissage d'opérateurs unitaires à partir d'exemples d'entrée-sortie, sans exiger que les vecteurs d'entraînement soient orthogonaux. L'opérateur unitaire A à apprendre est optimisé de sorte que, pour chaque paire d'entraînement $(|\chi_i\rangle, |\psi_i\rangle)$, on minimise l'erreur de transformation. La fonction de fitness est définie comme la somme des normes ℓ_1 des différences entre les sorties produites et les sorties cibles :

$$\mathcal{F}(A) = \sum_{i=1}^N \|A|\chi_i\rangle - |\psi_i\rangle\|_1 \quad (3.3.1)$$

où N est le nombre d'exemples, et la norme ℓ_1 est utilisée pour quantifier l'écart entre la sortie produite par l'opérateur candidat A et l'état cible. Ce cadre permet une optimisation directe de matrices continues, et les mutations sont modélisées comme de légères perturbations locales appliquées aux coefficients de A .

D'autre part, ils formalisent un modèle théorique de Algorithme génétique quantique (AGQ), où les individus sont représentés par des superpositions quantiques, intriqués avec leurs valeurs de fitness, et où la sélection naturelle est simulée par un processus de mesure quantique. Cette approche introduit une dynamique évolutive fondamentalement non classique, où la diversité génétique et la convergence sont directement influencées par les lois de la mécanique quantique, notamment à travers la mesure.

Enfin, les auteurs critiquent les représentations classiques fixes des circuits (par exemple sous forme d'arbre ou de liste), et suggèrent d'explorer des structures plus flexibles, notamment celles fondées sur la théorie des groupes abéliens finis. Cette proposition vise à mieux capturer la richesse combinatoire de l'espace des circuits quantiques. L'ensemble de ces contributions étend significativement les capacités des algorithmes évolutionnaires traditionnels, en ouvrant la voie vers des modèles plus puissants et véritablement quantiques d'optimisation évolutionnaire.

Tandaitnik et Guerreiro (2022) proposent d'utiliser des algorithmes génétiques AG pour la conception automatique de circuits quantiques, en particulier pour l'évolution de circuits correcteurs d'erreurs stabilisateurs (QECC). Leur méthode repose sur plusieurs innovations fortes.

Tout d'abord, les circuits sont représentés génétiquement sous forme de matrices de portes Clifford⁵, ce qui permet une simulation efficace et évite le recours à des approximations coûteuses. Chaque circuit candidat est ainsi un génotype structuré, facilitant la manipulation par les opérateurs génétiques standards (crossover, mutation).

La fonction de fitness qu'ils introduisent combine deux critères fondamentaux : la capacité du circuit à corriger les erreurs, mesurée par un degré de corrigibilité C , et la profondeur D du circuit. La fonction de fitness est donnée par :

$$F = w \times C - D \quad (3.3.2)$$

où w est un poids élevé (par exemple, $w = 1000$) favorisant prioritairement la capacité de correction sur la compacité. Ce choix guide efficacement l'évolution vers des circuits à la fois performants et utilisables dans des architectures réelles.

Ils adaptent aussi la fonction de fitness pour orienter l'évolution vers des codes spécifiques, comme les codes topologiques (Calderbank-Shor-Steane (CSS)), en introduisant un terme correctif supplémentaire valorisant les stabilisateurs particuliers recherchés.

Une évaluation systématique est menée en comparant leur approche à une recherche aléatoire (Random Search (RS)). Les résultats montrent que le AG converge beaucoup plus rapidement et efficacement vers des circuits performants, et que cet avantage s'amplifie avec l'augmentation du nombre de qubits.

Ces éléments font de leur contribution un jalon important dans la conception automatique de circuits quantiques adaptés aux contraintes pratiques du calcul quantique bruyant.

3.3.2 Optimisation structurelle des circuits :

Krylov et Lukac (2018) proposent le Quantum Encoded Quantum Evolutionary Algorithm (QEQA), une approche hybride visant à la synthèse de circuits quantiques à partir d'une population de portes codées en qubits et qutrits. Contrairement aux algorithmes génétiques classiques AG qui manipulent des circuits entiers, leur méthode repose sur l'évolution indépendante de segments de circuit, chacun correspondant à une porte élémentaire codée quantiquement.

Chaque segment est représenté par deux registres quantiques : un qubit encodant l'angle de rotation θ (via ses amplitudes complexes α, β) et un qutrit spécifiant l'axe de

5. **Clifford** : Ensemble de portes quantiques qui stabilisent les états de type Pauli. Il comprend notamment les portes Hadamard (H), Phase (S), et CNOT.

rotation $r \in \{x, y, z\}$. Lors de la construction du circuit, une séquence de mesures répétées sur ces registres permet de générer des portes unitaires appliquées successivement sur les qubits cibles.

La mutation est réalisée via des opérateurs de rotation de type $SU(3)$ ⁶ sur l'état du qutrit, modifiant ainsi dynamiquement l'axe de la future rotation unitaire. Elle est proportionnelle à l'erreur actuelle du segment, rendant la mutation *adaptive* selon la performance.

Le groupe $SU(3)$, ou groupe unitaire spécial de degré 3, désigne l'ensemble des matrices unitaires 3×3 de déterminant 1. Il représente mathématiquement l'ensemble des transformations unitaires possibles appliquées à un qutrit, c'est-à-dire à un système quantique à trois niveaux. En utilisant des opérations dans $SU(3)$, l'algorithme assure que les mutations appliquées à l'état du qutrit respectent les contraintes physiques de l'évolution quantique, tout en explorant l'espace des rotations possibles sur les axes x , y ou z .

L'évaluation des circuits repose sur une fonction de fitness exprimée en fonction de la fidélité entre la matrice unitaire cible T et la matrice synthétisée S :

$$\mathcal{F}(S) = 1 - \sqrt{\frac{2^n - |\text{Tr}(S^\dagger T)|}{2^n}} \quad (3.3.3)$$

où n est le nombre de qubits du circuit, et Tr désigne la trace matricielle. Cette expression mesure l'écart unitaire entre la solution trouvée et l'objectif, en le normalisant par la dimension de l'espace de Hilbert.

Bien que les performances de QEQEA soient légèrement inférieures à celles d'un AG classique optimisé sur Graphics Processing Unit (GPU), l'intérêt majeur de cette approche réside dans sa compatibilité avec des architectures quantiques physiques. Elle ouvre la voie à des modèles d'évolution partiellement exécutables sur ordinateur quantique, notamment dans les paradigmes de type RMN ou calcul optique.

Davis et al. (2020) introduisent une approche pragmatique et efficace de synthèse de circuits quantiques, conçue pour produire des circuits de faible profondeur en minimisant le nombre de portes CNOT, tout en tenant compte des contraintes topologiques des architectures NISQ. L'algorithme repose sur une stratégie de recherche inspirée de A-star Search Algorithm (A^*), combinée à une optimisation numérique des paramètres des portes.

L'objectif est de minimiser une fonction de distance unitaire entre l'unitaire cible U et l'unitaire synthétisé U_S :

$$\min_{U_S} D(U, U_S) \quad \text{où} \quad D(U, U_S) = 1 - \frac{|\text{Tr}(U^\dagger U_S)|}{2^n} \quad (3.3.4)$$

La particularité de cette méthode est d'intégrer directement la topologie physique de la puce dans le processus de synthèse, qu'il s'agisse de topologies linéaires ou en « bowtie ». Cela permet d'éviter le recours à des portes SWAP coûteuses après coup, en produisant des circuits directement exécutables sur le matériel cible.

La recherche dans l'espace des circuits est guidée par une heuristique de type A^* , dans laquelle chaque nœud est évalué selon une fonction de coût $f(n) = g(n) + h(n)$, où $g(n)$ représente le nombre de portes CNOT et $h(n)$ correspond à une pénalité proportionnelle à la distance unitaire résiduelle, pondérée par un facteur empirique a .

6. **SU(3)** : Groupe unitaire spécial de degré 3, représentant l'ensemble des matrices unitaires 3×3 de déterminant 1, utilisé pour les rotations dans un espace à trois niveaux (qutrits).

Une fois la structure du circuit déterminée, les paramètres continus des portes de type **U3** sont optimisés numériquement à l'aide d'algorithmes tels que Broyden–Fletcher–Goldfarb–Shanno algorithm (BFGS) ou Constrained Optimization BY Linear Approximations (COBYLA), afin de minimiser finement la distance à l'unitaire cible.

L'approche est hautement configurable : elle prend en charge différents types de topologies (comme IBM Q5 ou QNL8QR-V5) et différents ensembles de portes natives (Rx, Rz, CNOT). Les résultats empiriques sont particulièrement convaincants, avec une réduction allant jusqu'à $5.3\times$ du nombre de portes CNOT par rapport à UniversalQ, une diminution de 36 % de la longueur des circuits pour des tâches de préparation d'état face à Qiskit, et la génération de circuits optimaux pour des portes classiques comme Toffoli ou Fredkin.

Miranda et al. (2023) proposent une approche innovante pour la conception automatique de circuits quantiques en utilisant un algorithme génétique AG basé sur le modèle en îles (Island Genetic Algorithm). Leur principale contribution est d'accroître la diversité génétique au sein de la population en faisant évoluer plusieurs sous-populations $\{P_1, P_2, \dots, P_k\}$ de manière indépendante avant d'autoriser des migrations contrôlées, ce qui réduit le risque de convergence prématurée vers des optima locaux.

Leur méthode vise la décomposition directe d'une matrice unitaire cible U_t sans recourir à des qubits auxiliaires, en générant un circuit candidat U_a composé d'une séquence de blocs unitaires $\{B_1, B_2, \dots, B_m\}$, chacun représentant un ensemble de portes quantiques.

La qualité d'un circuit est évaluée à l'aide de la **fidélité de trace**, définie par :

$$f(U_a, U_t) = \frac{1}{2^n} \left| \text{Tr} \left(U_a U_t^\dagger \right) \right| \quad (3.3.5)$$

où n est le nombre de qubits, et Tr désigne la trace matricielle. L'erreur associée est donnée par :

$$\varepsilon = 1 - f(U_a, U_t)^2 \quad (3.3.6)$$

et l'optimisation vise à minimiser ε jusqu'à atteindre un seuil d'arrêt fixé à 10^{-6} .

Les opérations de croisement et de mutation sont adaptées à la structure modulaire des circuits : les blocs sont échangés ou modifiés selon leur nature (contrôlé, non-contrôlé ou auxiliaire). Comme illustré dans la figure 3.1, l'algorithme repose sur une évolution parallèle de plusieurs sous-populations indépendantes, suivie de migrations contrôlées entre îles, ce qui permet d'augmenter la diversité génétique tout en évitant la convergence prématurée vers des optima locaux. Cette approche structurée permet de limiter la dégradation locale du circuit tout en favorisant une exploration efficace de l'espace des solutions.

L'efficacité de leur méthode est démontrée par la synthèse automatique des portes complexes telles que la porte Toffoli, la porte Fredkin, et l'opérateur Hadamard Coin, obtenant des circuits de haute fidélité compatibles avec les contraintes des dispositifs quantiques de type NISQ. La figure 3.1 illustre le schéma de déroulement de l'algorithme génétique basé sur le modèle en îles.

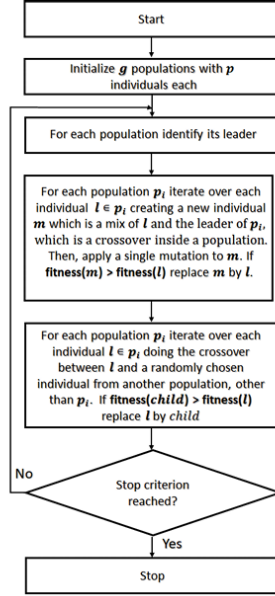


FIGURE 3.1 – Schéma de déroulement de l’algorithme génétique basé sur le modèle en îles (MIRANDA et al., 2023).

Creevey et al. (2023) proposent Genetic Algorithm for State Preparation (GASP), une approche évolutionnaire conçue pour générer des circuits quantiques capables de préparer un état cible $|\psi_{\text{target}}\rangle$ avec une haute fidélité. Cette méthode vise à produire des circuits courts et efficaces, mieux adaptés aux contraintes des dispositifs NISQ, en comparaison avec les synthèses exactes comme celle de Qiskit, basée sur la méthode (SBM (Shende, Bullock et Markov) (SHENDE et al., 2006)).

Chaque individu dans GASP est un circuit quantique, représenté comme une séquence de gènes de la forme $[q_t, G, q_c, \theta]$, où $G \in \{\text{Rx}, \text{Ry}, \text{Rz}, \text{CNOT}\}$ et θ est un angle de rotation. L’évaluation de la qualité d’un circuit repose sur la fidélité de l’état généré $|\psi(\vec{\theta})\rangle$ avec l’état cible :

$$\mathcal{F}(|\psi(\vec{\theta})\rangle) = |\langle \psi_{\text{target}} | \psi(\vec{\theta}) \rangle|^2 \quad (3.3.7)$$

GASP combine une sélection par roue de roulette, un croisement à un point, une mutation (5%) et une optimisation classique des angles par Sequential Least Squares Programming (SLSQP). En cas de stagnation, le nombre de gènes est augmenté pour accroître la complexité du circuit.

Les résultats expérimentaux montrent que GASP produit des circuits de bien moindre profondeur que ceux générés par Qiskit, tout en maintenant la fidélité cible, notamment pour les états Gaussian et W. Cette réduction de la profondeur rend les circuits générés plus robustes au bruit, ce qui est essentiel dans le contexte des architectures NISQ.

3.3.3 Avancées vers l’optimisation multi-objectifs :

Potoček et al. (2018) introduisent un cadre d’optimisation évolutionnaire multi-objectifs pour la découverte automatique de circuits quantiques à partir d’exemples d’entrée-sortie. Leur algorithme génétique est capable de retrouver non seulement des circuits canoniques comme la QFT ou Grover, mais également des variantes plus compactes ou approximatives, optimisées selon différents critères d’implémentation.

L’objectif général consiste à automatiser la découverte de circuits quantiques corrects à partir de spécifications fonctionnelles (I/O), sans hypothèse sur leur structure. Le modèle d’optimisation utilisé repose sur un vecteur d’objectifs multi-critères :

$$\min_{C_{\text{cand}}} [E_{\text{avg}}, E_{\text{max}}, \text{num gates}, \text{num.oracle}, \text{type-specific counts}] \quad (3.3.8)$$

où E_{avg} est l’erreur moyenne mesurée sur un ensemble d’états d’entrée, et E_{max} l’erreur maximale sur les pires cas testés. Le nombre total de portes (num gates) est utilisé comme indicateur direct de complexité structurelle, impactant la profondeur du circuit, sa robustesse au bruit et son exécution sur matériel NISQ. Les deux autres critères permettent de contrôler respectivement l’utilisation de portes spécifiques coûteuses (comme les oracles), ainsi que le profil de distribution des portes selon leur type (par exemple CNOT, Toffoli ou Rx).

Ce cadre permet à l’algorithme d’explorer des compromis pertinents en générant un *front de Pareto* de solutions, où chaque circuit représente un équilibre entre fidélité, taille et coût physique. Cela s’avère particulièrement pertinent dans les architectures bruitées, où l’optimisation structurelle est souvent plus déterminante que la précision absolue.

L’erreur de sortie est calculée différemment selon le contexte. Pour les circuits de type Grover, elle est définie par :

$$E = 1 - |\langle \psi_{\text{target}} | \psi_{\text{output}} \rangle|^2 \quad (3.3.9)$$

tandis que pour la QFT, elle est évaluée comme une erreur moyenne sur tous les vecteurs :

$$E = 1 - \frac{1}{N} \sum_i |\langle \psi_i | \chi_i \rangle| \quad (3.3.10)$$

Plusieurs contributions marquent l’originalité de leur approche. Premièrement, la découverte des circuits se fait sans connaissance a priori de leur structure, uniquement à partir d’exemples d’entrée-sortie. Deuxièmement, la recherche multi-objectifs permet de générer des variantes adaptées à différentes contraintes d’implémentation, notamment sur des architectures NISQ. L’algorithme utilise une représentation génétique souple, où les individus sont des séquences de portes de longueur variable. Il combine des mutations discrètes (permutation de qubits, suppression/insertion de portes) et continues (ajustement d’angles), ainsi que des opérations avancées comme l’insertion de séquences inverses ou la fusion automatique de portes adjacentes.

Enfin, les résultats empiriques montrent que l’approche permet de générer des circuits pour la QFT avec moins de 10 portes et une erreur inférieure à 0.001, ainsi que des versions optimisées de l’algorithme de Grover plus performantes que l’originale dans certains cas. Des circuits originaux à 4 qubits sont également découverts par une stratégie de décomposition modulaire.

Cette méthode illustre le potentiel des algorithmes évolutionnaires pour explorer des structures inédites dans l’espace des circuits quantiques. Elle est aussi proposée comme outil de *découverte*, capable de générer des circuits corrects dans des situations où la solution analytique n’est pas connue.

Rindell et al. (2023) présentent une méthode d’optimisation évolutionnaire pour la préparation approximative d’états quantiques sur des dispositifs NISQ, en tirant parti d’un algorithme génétique multi-objectifs. Leur approche vise à générer des circuits quantiques

courts qui, bien qu’approximatifs, offrent une meilleure fidélité finale que les circuits exacts lorsque le bruit physique est pris en compte.

Le processus d’optimisation repose sur deux objectifs antagonistes. Le premier est la maximisation de la fidélité entre l’état cible $|\psi_{\text{target}}\rangle$ et l’état préparé $|\psi_{\text{output}}\rangle$, mesurée par :

$$\mathcal{F} = |\langle \psi_{\text{target}} | \psi_{\text{output}} \rangle|^2 \quad (3.3.11)$$

Le second objectif est la minimisation du coût structurel du circuit, défini par :

$$\text{cost} = N_1 + 10 \times N_2 \quad (3.3.12)$$

où N_1 est le nombre de portes à un qubit, et N_2 le nombre de portes CNOT, plus coûteuses et sensibles au bruit.

Pour résoudre ce compromis, les auteurs utilisent l’algorithme NSGA-II, bien adapté à l’optimisation multi-objectifs dans des espaces de solutions complexes. L’évaluation des circuits intègre un modèle réaliste de bruit correspondant à l’architecture IBM **FakeVigo**, simulant les conditions d’exécution sur des processeurs quantiques actuels.

Une contribution théorique importante du travail est la mise en évidence du compromis optimal entre la fidélité théorique $\mathcal{F}_0$ et la robustesse au bruit. Ils montrent que la fidélité réelle $\mathcal{F}_{\text{noisy}}$ d’un circuit profond l s’exprime approximativement par :

$$\mathcal{F}_{\text{noisy}} \approx (1 - p)^l \cdot \mathcal{F}_0 = (1 - p)^l (1 - \varepsilon^2) \quad (3.3.13)$$

où p représente le taux de bruit par porte, et ε l’erreur d’approximation de l’état cible. Ce modèle met en évidence que, dans un environnement bruité, un circuit plus court et légèrement approximatif ($\varepsilon > 0$) peut surpasser en performance pratique un circuit exact mais plus long ($\varepsilon = 0$).

Sur le plan expérimental, la méthode proposée surpasse la méthode Low-Rank State Preparation (LRSP), améliorant la fidélité effective jusqu’à +38 % dans certains cas. De plus, l’approche est capable d’adapter automatiquement la structure des circuits aux contraintes topologiques réelles du matériel cible.

En résumé, ce travail démontre que dans les environnements quantiques bruités, la recherche d’une précision structurelle maximale est moins importante que l’optimisation globale prenant en compte les effets physiques réels. Il ouvre la voie à une génération de circuits spécifiquement conçus pour la robustesse plutôt que pour l’exactitude abstraite.

Ding et Spector (2023) introduisent Multi-Objective Evolutionary Architecture Search for Parameterized Quantum Circuits (MEAS-PQC), un cadre d’optimisation évolutionnaire pour la conception automatique de circuits quantiques paramétrés (Parameterized Quantum Circuit (PQC)). Ce cadre repose sur un algorithme génétique multi-objectifs de type NSGA-II, spécifiquement adapté aux particularités du calcul quantique.

L’objectif d’optimisation est triple : maximiser la performance moyenne $R(C)$ de l’agent contrôlé par le circuit C , tout en minimisant simultanément l’estimation du bruit $\Delta(C)$ et la taille du modèle $N(C)$. Formellement, le problème est formulé comme suit :

$$\min_C [-R(C), \Delta(C), N(C)] \quad (3.3.14)$$

Chaque circuit candidat est représenté par une séquence de blocs élémentaires x_i , parmi lesquels x_1 désigne les rotations paramétrées (R_x, R_y, R_z) , x_2 représente les opérations

d'intrication par portes CZ, x_3 encode les données d'entrée via une activation de type $\sigma(\lambda d)$, tandis que x_0 correspond aux mesures projectives.

Pour favoriser l'exploration des architectures de longueur variable, l'algorithme utilise une mutation appelée Uniform Mutation by Addition and Deletion (UMAD) (Uniform Mutation by Addition and Deletion), définie par des taux :

$$r_{\text{del}} = \frac{1}{\ell}, \quad r_{\text{add}} = \frac{1}{\ell + 1} \quad (3.3.15)$$

où ℓ est la longueur actuelle du circuit.

Des simplifications sont appliquées pour éliminer les redondances structurelles, en exploitant des identités telles que :

$$R_x(\theta_1) \cdot R_x(\theta_2) = R_x(\theta_1 + \theta_2), \quad \text{et} \quad CZ \cdot CZ = I \quad (3.3.16)$$

ce qui permet de réduire efficacement la complexité des circuits générés.

L'évaluation des architectures est effectuée dans un contexte d'apprentissage par renforcement (Reinforcement Learning (RL)), sur plusieurs environnements standards d'OpenAI, tels que **CartPole**, **MountainCar** et **Acrobot**. L'optimisation des paramètres continus θ des circuits est réalisée par descente de gradient classique, suivant la mise à jour :

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L \quad (3.3.17)$$

où L est la fonction de perte quantifiant la performance de l'agent.

Sur le plan expérimental, MEAS-PQC surpasse nettement les approches antérieures comme Evolutionary Quantum Architecture Search for Parameterized Quantum Circuits (EQAS-PQC) et Softmax-Based Architecture Search for Parameterized Quantum Circuits (SOFTMAX-PQC) en termes de rapidité d'apprentissage, de robustesse au bruit et d'efficacité des architectures produites. Le cadre multi-objectif permet en particulier de réduire simultanément le bruit et la taille du modèle sans compromettre les performances. L'analyse des circuits générés révèle que certaines structures récurrentes, telles que la succession d'un bloc x_1 suivi d'un bloc x_2 , sont particulièrement associées aux meilleures performances obtenues.

Rahman et al. (2025) proposent une approche novatrice pour la synthèse automatique de circuits quantiques en utilisant un algorithme génétique multi-objectifs assisté par logique floue. Leur méthode vise à générer des circuits compatibles avec les contraintes matérielles des dispositifs NISQ, tout en optimisant simultanément trois critères essentiels : la fidélité de la transformation, la profondeur du circuit, et le nombre de portes multi-qubits.

Le problème est formalisé comme une optimisation à trois objectifs, définie par la fonction :

$$F(C_{\text{syn}}) = [E(U_{\text{in}}, C_{\text{syn}}), D(C_{\text{syn}}), G_2(C_{\text{syn}})] \quad (3.3.18)$$

où $E(U_{\text{in}}, C_{\text{syn}})$ mesure l'erreur d'approximation du circuit synthétisé C_{syn} par rapport à l'unitaire cible U_{in} , à l'aide de la distance de Hilbert-Schmidt :

$$E = 1 - \frac{|\text{Tr}(U_{\text{in}}^{\dagger} U_{\text{syn}})|}{2^n} \quad (3.3.19)$$

Le terme $D(C_{\text{syn}})$ représente la profondeur du circuit, et $G_2(C_{\text{syn}})$ quantifie le nombre de portes multi-qubits utilisées, telles que les portes **CNOT**.

L’algorithme utilisé repose sur une version enrichie du NSGA-II, intégrant un système de logique floue pour ajuster dynamiquement le taux de mutation. Deux variables d’entrée sont surveillées : le taux d’amélioration observé sur les dernières générations et la valeur actuelle du taux de mutation. À partir de ces observations, des règles floues comme « *Si amélioration faible ET taux de mutation élevé, alors réduire mutation* » permettent d’adapter de manière intelligente la stratégie évolutive en fonction de la dynamique de l’optimisation.

Plusieurs contributions techniques renforcent l’efficacité de la méthode. L’encodage des circuits repose sur des séquences modulaires de couches parallèles, avec une optimisation automatique du mapping entrée-sortie des qubits. Les opérateurs génétiques sont spécialisés pour manipuler ces séquences : ajout ou suppression de couches, permutation de portes, et ajustement des qubits ciblés. Une fusion automatique des portes redondantes est également mise en œuvre, en exploitant des identités telles que :

$$R_z(\theta_1) \cdot R_z(\theta_2) = R_z(\theta_1 + \theta_2), \quad X \cdot X = I,$$

afin de réduire la taille effective des circuits.

Enfin, les paramètres continus des portes de rotation θ sont optimisés localement en utilisant la méthode BFGS, en résolvant :

$$\theta^* = \arg \min_{\theta} E(U_{\text{in}}, C_{\text{syn}}(\theta)) \quad (3.3.20)$$

pour minimiser l’erreur finale du circuit.

Cette approche hybride améliore à la fois la diversité de la population explorée et la qualité de convergence des solutions trouvées, tout en respectant strictement les contraintes topologiques et les restrictions sur les ensembles de portes imposées par les architectures NISQ. Elle constitue ainsi une avancée significative vers une compilation quantique robuste et adaptée au matériel réel.

3.3.4 Développement de frameworks spécialisés :

Wu et al. (2020) présentent *QGo*, un cadre de compilation hiérarchique pour l’optimisation de circuits quantiques reposant sur la synthèse automatisée de blocs. Dans le contexte des dispositifs NISQ, où les portes à deux qubits (comme les CNOT) sont plus bruyantes que les portes à un seul qubit, il est essentiel de minimiser leur nombre pour améliorer la fidélité du circuit.

Le principal défi de la synthèse quantique réside dans sa complexité exponentielle en fonction du nombre de qubits (la taille de la matrice unitaire étant $2^n \times 2^n$). Pour rendre la synthèse applicable à de grands circuits, les auteurs introduisent une méthode bloc par bloc, où le circuit est partitionné en sous-circuits de petite taille (3 ou 4 qubits), optimisés individuellement par un moteur de synthèse, puis recomposés.

Le cadre QGo repose sur une approche hiérarchique en quatre étapes pour optimiser les circuits quantiques tout en tenant compte de la connectivité matérielle des dispositifs NISQ :

- **Mapping physique** : QGo commence par mapper les qubits logiques sur le matériel cible à l’aide du compilateur `t|ket>`, en insérant les portes SWAP nécessaires dès le départ pour éviter de les ajouter après coup.
- **Partitionnement du circuit** : le circuit est représenté sous forme d’un graphe de dépendances. Il est ensuite découpé en blocs indépendants de 3 à 4 qubits, en

maximisant localement le nombre de portes CNOT, tout en respectant la connectivité physique.

- **Synthèse des blocs** : chaque bloc est optimisé indépendamment via une méthode de synthèse numérique qui minimise les CNOT tout en gardant une approximation précise de l'unitaire d'origine.
- **Recomposition du circuit** : les blocs optimisés sont recombinaés. Si la version synthétisée d'un bloc n'est pas meilleure, l'original est conservé. Une dernière passe fusionne les portes monobit adjacentes.

La figure 3.2 présente l'approche hiérarchique du cadre QGo.

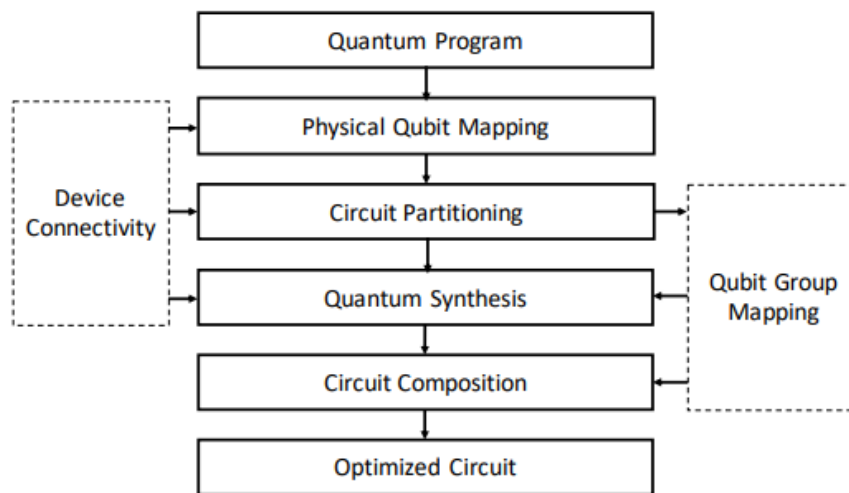


FIGURE 3.2 – Le principe de fonctionnement de (QGo).

Des expériences menées sur trois catégories de circuits (petits, moyens et grands) montrent que QGo permet de réduire en moyenne de 29,9 % le nombre de CNOTs par rapport à $\mathbf{t|ket\rangle}$, avec des pics allant jusqu'à 50 %. Les circuits optimisés par QGo démontrent une amélioration de la fidélité mesurée sur l'IBM Athens, corrélée à la réduction de CNOT (coefficient de 78 %). Une simulation bruitée confirme cette tendance avec une corrélation de 76 % pour les circuits moyens. L'analyse de sensibilité indique que plus l'erreur des portes est élevée, plus l'impact de QGo sur la fidélité est significatif.

La figure 3.3 suivante, illustre un exemple d'optimisation d'un circuit quantique à 5 qubits en utilisant l'outil QGo. Le circuit initial est tout d'abord décomposé en trois blocs, chacun contenant uniquement des portes opérant sur un sous-ensemble de trois qubits. Par exemple, le premier bloc agit sur les qubits q0, q1 et q4 ; le second sur q1, q2 et q4 ; et le troisième sur q0, q1 et q3. Cette partition respecte rigoureusement les dépendances de portes du circuit d'origine. Ensuite, une synthèse quantique est effectuée indépendamment sur chaque bloc, ce qui permet de réduire le nombre de portes CNOT dans les deux premiers blocs, tandis que le dernier bloc conserve deux portes CNOT. Finalement, les portes à un seul qubit sont combinées lorsque cela est possible, menant à un circuit optimisé, fonctionnellement équivalent au circuit de départ, mais avec une complexité réduite en termes de portes CNOT.

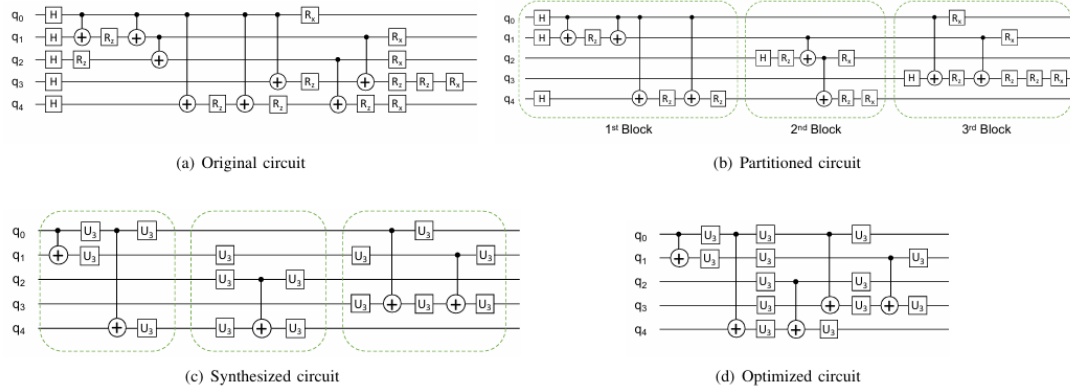


FIGURE 3.3 – Un exemple d’optimisation de circuit quantique avec (QGo).

Enfin, l’optimisation de circuits de grande taille (60+ qubits) confirme la **scalabilité** du cadre, avec un gain moyen de 28,9 % en CNOTs pour QGo-4 (blocs de 4 qubits), au prix d’un temps de compilation plus élevé. L’approche proposée ouvre ainsi une voie vers une intégration efficace de la synthèse dans les compilateurs quantiques industriels, avec des bénéfices concrets pour les dispositifs NISQ actuels.

Sünkel et al. (2023) proposent *GA4QCO*, un cadre modulaire et configurable permettant de générer automatiquement des circuits quantiques adaptés aux contraintes des dispositifs NISQ, via un algorithme génétique. Ce cadre repose sur l’optimisation de circuits en fonction d’objectifs définis par l’utilisateur à travers une fonction de fitness personnalisable.

Dans ce framework, Chaque individu (chromosomes) appartenant à la population représente un circuit quantique et est encodé sous forme d’une liste bidimensionnelle de taille $n \times m$:

- n : nombre de qubits (lignes) ;
- m : profondeur du circuit (colonnes).

Chaque cellule contient une porte quantique décrite par ses attributs : nom (e.g., R_Y , $CNOT$), qubits affectés, qubits de contrôle, qubits cibles et paramètres (par exemple, un angle θ pour les portes rotationnelles).

Ce modèle repose sur un processus évolutif inspiré des algorithmes génétiques classiques :

1. **Initialisation** : génération aléatoire d’une population d’individus (circuits quantiques).
2. **Évaluation** : chaque individu est évalué selon une fonction de fitness, par exemple :

$$F = |\langle \psi_{\text{target}} | \psi_{\text{output}} \rangle|^2 \quad (3.3.21)$$

pour mesurer la fidélité à un état quantique cible.

3. **Sélection** : individus sélectionnés pour reproduction selon plusieurs stratégies : aléatoire, tournoi, ou roue de la fortune.
4. **Croisement** : génération d’enfants via différentes méthodes (single-point, multi-point, blockwise), avec gestion automatique des circuits de tailles différentes.
5. **Mutation** : application de modifications aléatoires sur les circuits (flip de porte, permutation de colonnes, ajustement de paramètres, ajout/suppression de qubits/gates).

6. Survivor selection : sélection de la prochaine génération.

Ce cycle est répété pendant un nombre fixe de générations. Le processus est contrôlé par un fichier de configuration dans lequel l'utilisateur spécifie les paramètres du modèle (taille de population, taux de mutation, profondeur maximale, ensemble de portes autorisées, etc.).

La figure 3.4 décrit le processus d'optimisation fait par GA4QCO.

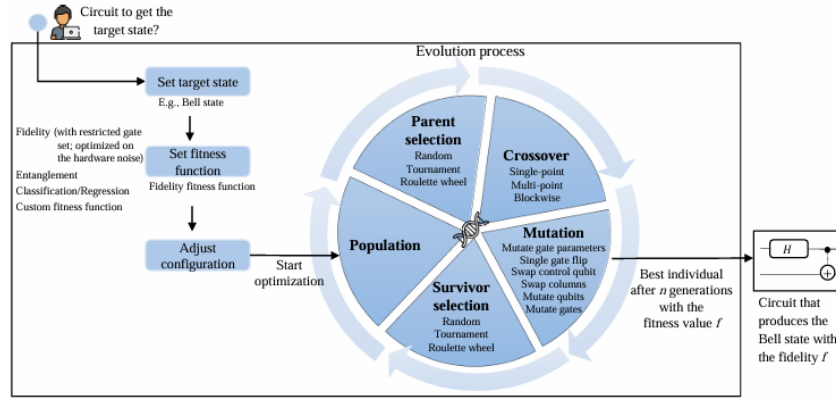


FIGURE 3.4 – Optimisation des circuits quantiques dans le cadre GA4QCO.

Les objectifs d'optimisation dans GA4QCO sont définis à travers une fonction de fitness que l'utilisateur peut personnaliser. Parmi les fonctions proposées, on trouve notamment la fidélité à un état cible $|\psi_{\text{target}}\rangle$, mesurée par le chevauchement avec l'état généré. Le cadre permet également de rechercher des états fortement intriqués, l'intrication étant quantifiée par l'entropie de von Neumann. Un autre objectif est la précision obtenue par un modèle quantique de classification ou de régression, entraîné au sein du processus évolutif. Enfin, GA4QCO offre la possibilité d'intégrer des fonctions de fitness entièrement personnalisées, ce qui élargit son champ d'application à de nombreux cas d'usage spécifiques.

Résultats expérimentaux Les auteurs appliquent GA4QCO à une tâche de préparation d'état sur 4 qubits avec une profondeur de 20. Deux configurations sont comparées :

- **Sans contrainte de portes** : la fidélité moyenne augmente graduellement et atteint ≈ 0.8 après 1000 générations.
- **Avec contrainte** (portes limitées à $\{\text{id}, \text{rz}, \text{sx}, \text{x}, \text{cx}\}$) : meilleure fidélité obtenue de 0.994, contre une moyenne de 0.29 pour une génération aléatoire.

Ces résultats montrent que GA4QCO est capable de produire des circuits courts, performants, et compatibles avec les restrictions physiques imposées par les dispositifs NISQ.

3.4 Synthèse comparative

Les tableaux suivants 3.1 et 3.2 récapitulent les travaux réalisés concernant la conception et l'optimisation de circuits quantiques à l'aide d'algorithmes génétiques, qu'ils soient classiques ou à objectifs multiples :

Référence	Cible	Approche	Représentation des circuits	Stratégie de fitness	Objectif principal	Réduction profondeur	Scalabilité	Réduction portes bruitées
YABUKI et IBA (2000)	Téléportation quantique	AG structuré par étapes	Étapes fonctionnelles codées avec portes élémentaires (type EPR $\rightarrow$ Alice $\rightarrow$ Bob)	Erreur d'amplitude (ratio)	Synthèse fonctionnelle minimale	Oui	Limité à 1 qubit	Non
LUKAC et PERKOWSKI (2002)	Circuits arbitraires	AG avec encodage par blocs	Blocs parallèles de portes quantiques élémentaires	Distance matricielle + pénalité longueur	Synthèse générale de circuits	Partielle	Jusqu'à 4 qubits	Non
GIRALDI et al. (2004)	Matrices unitaires et QGA	AG + modèle quantique (AGQ)	Matrices unitaires continues	Norme ℓ_1 sur exemples I/O	Apprentissage d'opérateurs	Non abordé	Modéré	Non
KRYLOV et LUKAC (2018)	Synthèse partielle de circuits	QEQEA (qubit + qutrit)	Qubits (θ) + qutrits (axes de rotation)	Fidélité matricielle via trace	Modèle partiellement quantique	Partielle	Faible	Non
DAVIS et al. (2020)	Circuits pour IBM Q	A*-search + optimisation locale	Séquence de portes natives (U3, CNOT)	Distance unitaire + CNOT count	Compilation topologie-aware	Oui (jusqu'à $5.3\times$)	Forte (reconfigurable)	Oui
TANDEITNIK et GUERREIRO (2022)	Circuits correcteurs (QECC)	AG sur portes Clifford	Matrices de portes Clifford	Corrigibility C et profondeur D	Codes stabilisateurs (CSS, Steane, Shor)	Oui	Jusqu'à 7 qubits	Oui (min CNOT)
MIRANDA et al. (2023)	Portes unitaires (Toffoli, Fredkin)	Island AG + mutation structurée	Blocs unitaires ($B_1, B_2, \dots$)	Trace fidelity, erreur ε	Synthèse modulaire sans qubit auxiliaire	Oui	Oui (migration îles)	Oui
CREEVEY et al. (2023)	Préparation d'états $ \psi\rangle$	AG (GASP) + SLSQP	Séquence de gènes : [qt, G, qc, θ]	$ \langle\psi_{\text{target}} \psi(\theta)\rangle ^2$	États cibles avec circuit minimal	Oui (vs Qiskit)	Modéré	Oui
SÜNKEL et al. (2023)	Circuits arbitraires	AG modulaire configurable	Matrice $n \times m$ de portes paramétrées	Fidélité à l'état cible ; entropie ; précision ML	Génération automatique sous contraintes NISQ	Oui	Moyenne (4 qubits, prof. 20)	Oui (portes compatibles NISQ)

TABLE 3.1 – Comparaison des méthodes évolutionnaires pour la synthèse de circuits quantiques

Référence	Objectifs optimisés	Fonction de fitness / Distance	Représentation du chromosome	Méthode / Spécificités	Spécificités techniques	Innovation / Contribution clé
POTOČEK et al. (2018)	E_{avg}, E_{max} , gates, oracle, types	Fidélité moyenne et maximale ; coût structurel	Séquence de portes à longueur variable	AG multi-objectifs	Mutations discrètes et continues ; fusions automatiques	Découverte automatique de circuits canoniques (QFT, Grover) à partir d'E/S
WU et al. (2020)	Nombre de CNOTs, fidélité, bruit	Corrélation réduction CNOT / fidélité ; simulation bruitée	Ensemble de sous-circuits (blocs de 3-4 qubits)	Compilation hiérarchique (QGo)	Découpage du circuit en blocs + re-composition	Synthèse évolutive scalable avec réduction CNOT jusqu'à 50 %
RINDELL et al. (2023)	Fidélité, coût pondéré ($N_1 + 10N_2$)	Fidélité sous bruit réaliste IBM Fake-Vigo	Liste séquentielle de portes avec types et angles	NSGA-II	Évaluation bruitée + front de Pareto	Compromis fidélité théorique vs robustesse matérielle
DING et SPECTOR (2023)	$-R(C), \Delta(C), N(C)$	Récompense RL, bruit, taille du modèle	Chaîne de blocs x_i (rotation, entanglement, encodage, mesure)	NSGA-II + UMAD	RL dans environnements standards d'OpenAI ; descente de gradient	Architecture PQC multitâche robuste au bruit
RAHMAN et al. (2025)	Erreur E , profondeur D , nb CNOT G_2	Distance de Hilbert-Schmidt	Couches parallèles avec paramètres θ et qubits mappés	NSGA-II + logique floue	Mutation adaptative ; fusion de portes ; BFGS local	Contrôle dynamique des mutations ; structure modulaire NISQ-compatible

TABLE 3.2 – Comparaison des approches multi-objectifs pour la synthèse de circuits quantiques

L'analyse du tableau 3.1 met en lumière la diversité des stratégies évolutionnaires appliquées à la synthèse de circuits quantiques. Les travaux fondateurs, tels que ceux de YABUKI et IBA (2000) et de LUKAC et PERKOWSKI (2002), se sont d'abord concentrés sur des cas spécifiques ou sur de petits circuits, illustrant la faisabilité de générer automatiquement des circuits fonctionnels à partir d'exemples ou de spécifications. Par la suite, des approches plus structurales ont émergé, comme QESEA KRYLOV et LUKAC (2018), qui introduit une représentation hybride basée sur qubits et qutrits, et DAVIS et al. (2020), dont la méthode tient compte de la topologie physique des processeurs NISQ.

Plusieurs méthodes mettent également l'accent sur la modularité, comme l'algorithme en îles de MIRANDA et al. (2023), ou la représentation configurable à base de matrices proposée dans GA4QCO (SÜNKEL et al., 2023). Toutefois, bien que ces travaux introduisent des idées puissantes pour la réduction de la profondeur des circuits ou la diminution du nombre de portes bruitées, la majorité reste limitée à des circuits de petite taille (souvent $n \leq 7$). Cela souligne une contrainte majeure : l'explosion combinatoire de l'espace de recherche dès que la taille du circuit augmente.

C'est dans ce contexte que la communauté a progressivement basculé vers des approches multi-objectifs, comme le montre le tableau 3.2. Ces méthodes notamment celles de (POTOČEK et al., 2018), (RINDELL et al., 2023), (DING & SPECTOR, 2023), (RAHMAN et al., 2025) — permettent d'optimiser simultanément plusieurs critères souvent antagonistes : la fidélité, la complexité structurale, la profondeur ou encore la robustesse face au bruit. Cette orientation répond à la nécessité de produire non pas un circuit parfait, mais un circuit exécutable sur les machines quantiques réelles, sujettes aux erreurs.

Néanmoins, plusieurs défis persistent. La plupart des approches utilisent des représentations de chromosomes modulaires (par blocs ou par couches), qui facilitent l'exploration, mais compliquent le contrôle fin de la profondeur et de la topologie. De plus, l'intégration du bruit, bien que présente dans certains travaux comme ceux de RINDELL et al. (2023), reste souvent simplifiée ou stationnaire, ne prenant pas en compte des effets dynamiques ou corrélés. Par ailleurs, même des méthodes adaptatives comme celles basées sur la logique floue (RAHMAN et al., 2025) reposent encore sur des heuristiques empiriques, dont la

robustesse n'est pas toujours prouvée à grande échelle.

Enfin, la comparaison des approches classiques (Tableau 3.1) et multi-objectifs (Tableau 3.2) montre une progression claire vers plus de réalisme, d'adaptabilité et de prise en compte des contraintes matérielles. C'est précisément cette complexité croissante des objectifs — fidélité, bruit, coût — qui a motivé l'adoption d'approches multi-objectifs. Ces dernières se sont imposées comme une réponse naturelle aux limites des méthodes mono-objectif, d'où notre choix de nous inscrire dans cette voie pour le développement de notre propre contribution au chapitre suivant.

Conclusion

Dans ce chapitre, nous avons présenté une synthèse structurée des travaux ayant recours aux algorithmes évolutionnaires pour la conception automatique de circuits quantiques. Cette analyse a permis de mettre en lumière plusieurs axes majeurs : la synthèse directe de circuits, l'optimisation structurelle, les approches multi-objectifs et les frameworks spécialisés. Malgré leurs performances prometteuses, ces méthodes restent limitées en termes de scalabilité, de modélisation réaliste du bruit, et d'adaptation fine aux contraintes matérielles des dispositifs NISQ. Toutefois, elles offrent une flexibilité précieuse pour explorer des architectures originales et optimiser plusieurs critères simultanément. Le prochain chapitre s'appuie sur ces constats pour introduire notre approche personnelle, visant à combiner efficacité, adaptabilité et respect des contraintes matérielles dans un cadre d'optimisation évolutionnaire hybride.

Deuxième partie

Contribution et Expérimentation

Chapitre 4

Conception

Introduction

Ce chapitre constitue le cœur de notre contribution. Nous y présentons la démarche de conception de notre algorithme évolutionnaire dédié à la synthèse automatique de circuits quantiques. L’objectif est de proposer une architecture capable de générer des circuits optimisés selon trois critères complémentaires : la fidélité, la profondeur et le coût structurel en portes.

Contrairement aux approches existantes, souvent limitées à un objectif unique, notre méthode adopte une vision plus flexible et adaptable, intégrant à la fois des mécanismes classiques d’algorithmes génétiques (sélection, croisement, mutation) et des extensions spécifiques aux contraintes des dispositifs NISQ. L’architecture proposée sera d’abord présentée sous sa forme **mono-objectif**, centrée sur la maximisation de la fidélité, avant d’être étendue à une version **multi-objectifs** (fidélité, profondeur, coût). Enfin, nous décrirons les mécanismes d’adaptation qui permettent à l’approche de rester applicable à des circuits de taille croissante, tout en tenant compte des contraintes computationnelles.

4.1 Approche globale proposée

L’approche que nous proposons s’articule autour d’une architecture évolutive conçue pour répondre aux défis de la synthèse automatique de circuits quantiques. Contrairement aux approches de la littérature, souvent limitées à des cas spécifiques (ex. : téléportation, QFT) ou à de petites tailles de circuits, notre solution se distingue par deux caractéristiques majeures :

- **Généralisation de l’algorithme** : le cadre proposé n’est pas restreint à un problème particulier mais peut traiter des circuits variés (synthèse d’unitaires, préparation d’états, optimisation topologie-aware, réduction de profondeur), grâce à une représentation flexible et à l’utilisation d’une fonction de fitness unitaire robuste.
- **Introduction de mécanismes avancés** : phase de raffinement (fusion et simplification symbolique des portes, optimisation locale des paramètres), mutations structurelles diversifiées, et stratégies de maintien de la diversité pour éviter la convergence prématurée.

La figure 4.1 illustre le schéma général de notre pipeline. Celui-ci démarre par un algorithme génétique **mono-objectif**, utilisé comme socle exploratoire basé sur la fidélité unitaire. Il est ensuite enrichi par une extension **multi-objectifs** (de type NSGA-II), permettant d’optimiser simultanément la fidélité, la profondeur et le coût en portes. Enfin, afin de rendre l’approche **scalable**, une stratégie adaptative est introduite :

- pour les circuits de petite taille ($nb_qubits < 10$), une évaluation directe est réalisée sans partitionnement ;
- pour les circuits de grande taille ($nb_qubits > 10$), une décomposition en blocs est appliquée :
 - **fortement denses** → blocs chevauchants pour préserver les corrélations,
 - **faiblement denses** → blocs indépendants pour réduire le coût de calcul.

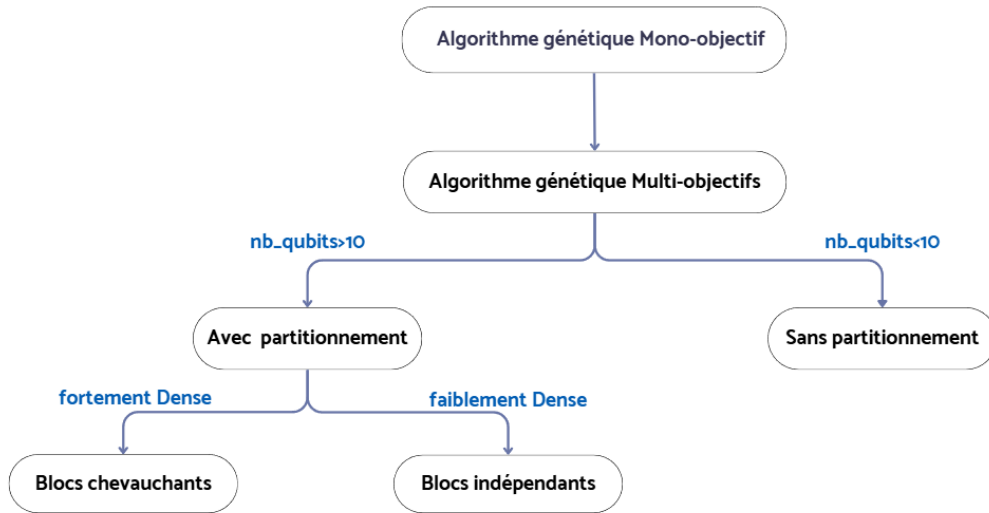


FIGURE 4.1 – Schéma général de l’approche proposée.

Ainsi, notre contribution se distingue de l’état de l’art par une architecture flexible, évolutive et adaptée aux contraintes des dispositifs NISQ, conciliant généralisation, optimisation multi-critères et scalabilité.

4.2 Algorithme génétique mono-objectif

Contrairement à de nombreux travaux centrés sur une tâche ou un jeu de portes spécifiques, nous proposons un AG *généralisé* et *robuste*. D’une part, la représentation des circuits et la fonction d’évaluation sont modulaires, ce qui rend l’algorithme applicable à divers problèmes (synthèse d’unitaires, préparation d’états, compilation contrainte par la topologie, réduction de profondeur/coût), quel que soit l’ensemble de portes ou l’architecture matérielle visée. Un point distinctif majeur réside dans l’utilisation d’une *fonction de fitness* qui ne se limite pas à comparer les états finaux produits par le circuit, mais évalue directement la proximité entre la matrice unitaire du circuit candidat et celle de la transformation cible. Cette approche permet de garantir la validité globale du comportement du circuit, indépendamment des états d’entrée, et offre une mesure plus fine de la qualité de la synthèse, tout en facilitant la généralisation à des problèmes variés.

D’autre part, nous introduisons deux mécanismes clés pour améliorer la qualité et la stabilité de la recherche : une *phase de raffinement* qui combine des simplifications symboliques (fusion, annulation et commutation de portes) et une ré-optimisation locale des paramètres continus, des *mutations structurelles* (insertion, suppression et ajout) couplées à des stratégies de maintien de la diversité (réduit simulé et adaptation du taux de mutation).

Ces choix limitent la stagnation et réduisent le risque de convergence prématurée vers un optimum local, tout en favorisant l'exploration de solutions compactes et exécutables sur dispositifs NISQ.

La figure 4.2 présente l'architecture générale de l'algorithme génétique mono-objectif appliqué à la synthèse automatique de circuits quantiques optimisés.

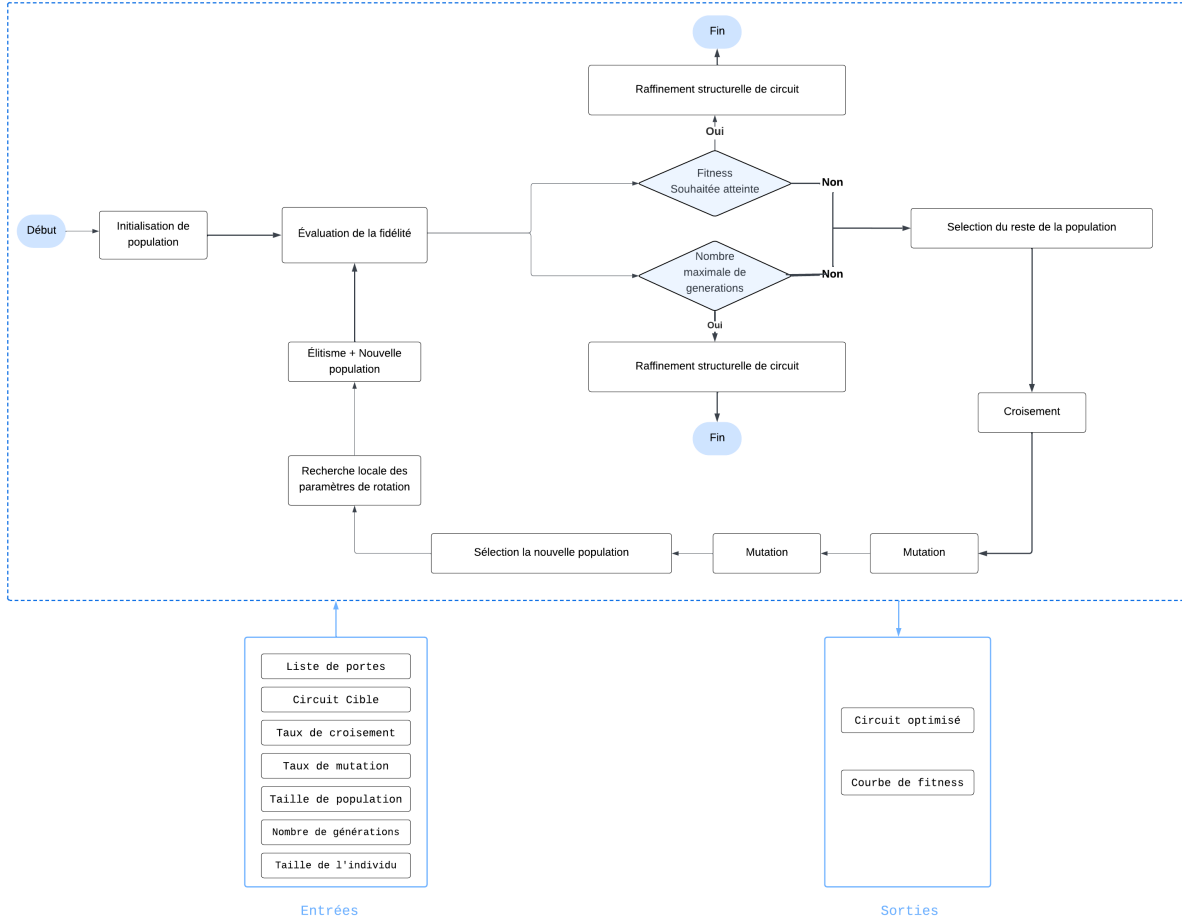


FIGURE 4.2 – Déroulement de l'algorithme génétique proposé

4.2.1 Encodage des circuits et représentation

L'encodage des circuits quantiques consiste à représenter un circuit sous une forme exploitable par un algorithme d'optimisation, en traduisant la séquence d'opérations appliquées aux qubits en une structure manipulable, comme un chromosome. Cela permet de générer, évaluer et faire évoluer automatiquement et d'une façon flexible les circuits dans un espace de recherche défini.

Dans notre approche, chaque circuit est encodé sous forme d'un *chromosome*, correspondant à une séquence ordonnée de *gènes*, chacun représentant une opération quantique élémentaire. Cette structure permet de simuler le comportement du circuit et d'y appliquer des opérateurs évolutifs.

Un gène est défini par un quadruplet :

$$\text{gène} = [q_i, \text{gate}, q_c, \theta]$$

où :

- q_i : identifiant du qubit cible,
- $gate$: type de porte quantique (par ex. Rx, Rz, CNOT, Toffoli...),
- q_c : qubit de contrôle, utilisé dans les opérations multi-qubits (optionnel),
- θ : paramètre de rotation pour les portes à angle (ex. Rx, Rz).

Ainsi, un *chromosome* complet représente un *circuit quantique* en listant ces opérations dans l'ordre d'exécution, sous la forme :

$$\text{chromosome} = [\text{gène}_1, \text{gène}_2, \dots, \text{gène}_n] \quad (4.2.1)$$

avec n désigne le nombre de *gènes* composant le circuit.

Pour illustrer cette représentation, prenons le cas d'un chromosome défini sur 3 qubits et contenant 7 gènes :

$$\begin{aligned} \text{Chromosome} = & [[0, X, \text{None}, \text{None}], [1, Z, \text{None}, \text{None}], [0, H, \text{None}, \text{None}], [2, RX, \text{None}, \frac{\pi}{3}], \\ & [1, CX, 0, \text{None}], [2, RZ, \text{None}, \frac{\pi}{2}], [2, Y, \text{None}, \text{None}]] \end{aligned}$$

Taille de chromosome : La taille du chromosome constitue un paramètre fondamental dans le processus d'optimisation. Fixée dès l'initialisation de l'algorithme, elle détermine le nombre maximal de gènes — et donc d'opérations quantiques — autorisés dans un circuit candidat. Bien que l'algorithme génétique mono-objectif soit principalement orienté vers la maximisation de la fidélité entre le circuit cible et le circuit généré, le contrôle de ce paramètre permet d'imposer une contrainte explicite sur la compacité de la solution. En pratique, en fixant la taille du chromosome à une valeur inférieure à celle du circuit initial (souvent proche de la moitié de sa longueur), on garantit que les circuits optimisés présentent un nombre de portes réduit par rapport à la version originale. Cette stratégie assure non seulement que le circuit obtenu aura un comportement fonctionnel très proche de celui du circuit cible, mais aussi une optimisation au niveau de la taille du circuit.

4.2.2 Fonction de fitness

L'évaluation de la qualité d'un circuit quantique généré repose sur une fonction de *fitness* basée sur la fidélité unitaire, une métrique standard dans la littérature (CREEVEY et al., 2023 ; DUMITRESCU, 2004).

La fonction de fitness utilisée s'écrit :

$$\text{fidélité}(C) = \frac{|\text{tr}(G^\dagger \cdot \lambda(C))|}{2^n} \quad (4.2.2)$$

où :

- $\lambda(C)$ est la matrice unitaire générée par le circuit candidat C ,
- G est la matrice unitaire cible à approximer,
- n est le nombre de qubits utilisés dans le circuit,

- $G^\dagger$ désigne la **conjuguée transposée** (ou adjointe) de G , définie par :

$$G^\dagger = \overline{G}^T,$$

où $\overline{G}$ est la matrice des coefficients conjugués complexes de G .

- $|\text{tr}(G^\dagger \lambda(C))|$ représente la norme complexe du trace, et garantit une valeur réelle et positive comprise entre 0 et 2^n .

Cette métrique est inspirée du produit scalaire de Hilbert–Schmidt (DUMITRESCU, 2004), et mesure la proximité unitaire entre le circuit généré et la cible.

Interprétation : La valeur retournée par $\text{fidélité}(C)$ est comprise entre 0 et 1 :

- 1 signifie que le circuit candidat C est équivalent à G à une phase globale près, soit $C \sim e^{i\theta} G$,
- 0 indique une absence totale de correspondance entre les deux circuits.

Cette métrique est invariante à la phase globale, ce qui est conforme aux principes de l’informatique quantique, où deux circuits globalement équivalents ne produisent pas de différence observable.

Démonstration de l’invariance à la phase globale : Supposons que :

$$\lambda(C) = e^{i\theta} G \Rightarrow \text{Tr}(G^\dagger \cdot \lambda(C)) = \text{Tr}(G^\dagger \cdot e^{i\theta} G) = e^{i\theta} \text{Tr}(G^\dagger G) = e^{i\theta} \text{Tr}(I_{2^n}) = e^{i\theta} \cdot 2^n$$

d’où :

$$\text{fidélité}(C) = \frac{|\text{Tr}(G^\dagger \lambda(C))|}{2^n} = \frac{|e^{i\theta}| \cdot 2^n}{2^n} = 1.$$

Cela montre que la fidélité reste maximale même en présence d’un décalage de phase, ce qui renforce la validité physique de cette métrique (DUMITRESCU, 2004).

Complexité et limitation : La principale limitation de cette métrique réside dans son coût computationnel élevé. En effet, le calcul de la matrice unitaire $\lambda(C)$ du circuit, de dimension $2^n \times 2^n$, devient rapidement intractable lorsque le nombre de qubits dépasse $n = 12$. Cette croissance exponentielle impacte considérablement le temps d’évaluation d’un individu dans la population évolutionnaire.

4.2.3 Méthodes de sélection

Dans le cadre de nos expérimentations, plusieurs stratégies de sélection ont été testées afin d’identifier celles qui favorisent la convergence et améliorent la qualité des solutions générées. Nous présentons ci-dessous les principales approches utilisées dans la littérature et intégrées à nos tests : sélection par roulette, sélection par rang, sélection par tournoi et sélection stochastique universelle.

1. **Sélection par roulette (Roulette Wheel Selection) :** Chaque individu est sélectionné avec une probabilité proportionnelle à sa fitness, comme décrit dans l’algorithme 1. Cette méthode, inspirée du tirage aléatoire sur une roue de loterie, favorise les individus les plus performants, tout en conservant une chance non nulle pour les individus moins adaptés.

Algorithm 1: Algorithme de sélection par la roulette

procédure *Sélection_Roulette* (*population*, *fitness_val*, *num_selected*) :

Entrée :

population : liste des individus

fitness_val : liste des valeurs de fitness associées à chaque individu

num_selected : nombre d'individus à sélectionner

Sortie : Liste des individus sélectionnés

begin

 Calculer la somme totale des fitness : $F_{total} \leftarrow \sum_i fitness_values[i]$

 Calculer les probabilités de sélection :

 Pour chaque individu i : $p_i = \frac{fitness_i}{F_{total}}$

 Sélectionner *num_selected* individus aléatoirement selon p_i , avec remise

 Retourner les individus sélectionnés

La figure 4.3 présente le principe de la sélection par roulette :

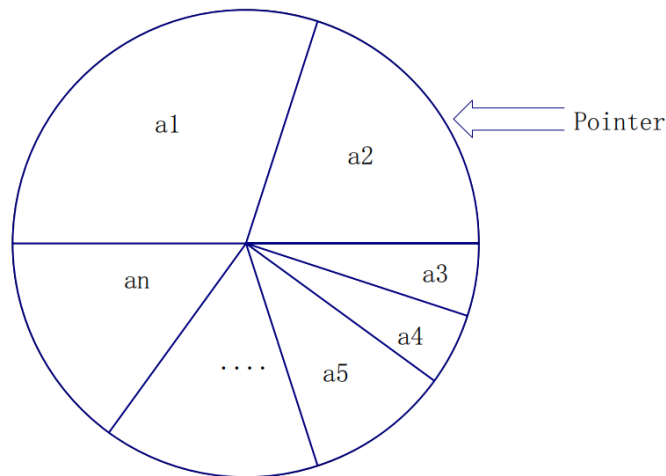


FIGURE 4.3 – Stratégie de sélection basée sur le mécanisme de la roue de la roulette (ZHONG et al., 2005)

2. **Sélection par rang (Rank Selection)** : Plutôt que de se baser sur les valeurs absolues de la fitness, cette méthode attribue une probabilité de sélection en fonction du rang de chaque individu dans la population triée. L'individu le plus faible a la plus petite probabilité, et le meilleur la plus élevée, mais de manière plus contrôlée qu'avec la roulette, comme décrit dans l'algorithme 2.

Algorithm 2: Algorithme de sélection par rang

procédure *Sélection_Par_Rang* (*pop*, *fitness_val*, *num_selected*) :
Entrée :*pop* : liste des individus*fitness_val* : liste des valeurs de fitness associées à chaque individu*num_selected* : nombre d'individus à sélectionner**Sortie :** Liste des individus sélectionnés**begin** Trier les indices des fitness par ordre décroissant $\rightarrow$ sorted_indices

Attribuer un rang à chaque individu (meilleur = rang le plus élevé)

Calculer les probabilités de sélection basées sur les rangs :

 Pour chaque rang r : $p_r = \frac{r}{\sum_r}$ Sélectionner aléatoirement *num_selected* individus selon p_r , avec remise

Retourner les individus sélectionnés

-
3. **Sélection par tournoi (Tournament Selection).** Cette méthode consiste à tirer au hasard un petit groupe d'individus (typiquement 2 à 5), puis à sélectionner le meilleur parmi eux pour la reproduction. Ce processus est répété jusqu'à former la nouvelle population. Les étapes de cette méthode sont bien expliquées dans l'algorithme 3.

Algorithm 3: Algorithme de sélection par tournoi

procédure *Sélection_Par_Tournoi* (*pop*, *fitness_val*, *num_selected*) :
Entrée :*pop* : liste des individus*fitness_val* : liste des valeurs de fitness associées à chaque individu*num_selected* : nombre d'individus à sélectionner*k* : Nombre d'individus par tournoi (par défaut $k = 3$)**Sortie :** Liste des individus sélectionnés**begin** **for** $i \leftarrow 1$ **to** *num_selected* **do** Choisir aléatoirement k individus distincts dans la population

Associer à chaque individu sa valeur de fitness

 Comparer les k individus selon leur fitness Sélectionner l'individu ayant la meilleure fitness parmi les k

Ajouter cet individu à la liste des individus sélectionnés

Retourner la liste des individus sélectionnés

La

figure 4.4 présente le principe de la sélection par tournoi :

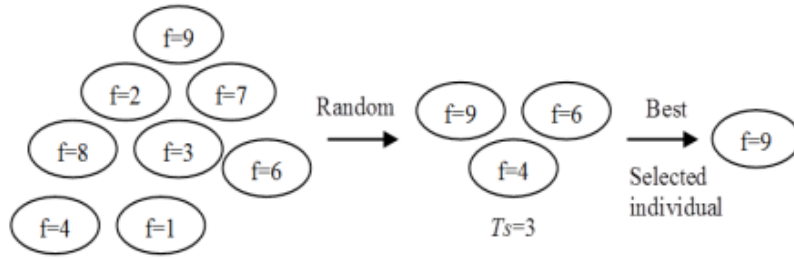


FIGURE 4.4 – Stratégie de sélection avec mécanisme de tournoi (RAZALI, GERAGHTY et al., 2011)

4. **Sélection stochastique universelle (SUS).** Cette méthode améliore la roulette en plaçant plusieurs pointeurs équidistants sur la roue de fitness cumulative. Elle garantit une répartition plus équitable et moins aléatoire des sélections. Les étapes de cette méthode sont bien expliquées dans l'algorithme 4.

Algorithm 4: Algorithme du sélection stochastique universelle

procédure *Sélection_stochastique* (*pop*, *fitness_val*, *num_selected*)

Entrée :

pop : liste des individus

fitness_val : liste des valeurs de fitness associées à chaque individu

num_selected : nombre d'individus à sélectionner

Sortie : Liste des individus sélectionnés

begin

Calculer la fitness totale : $F_{total} = \sum \text{fitnesses}$

Calculer la distance entre deux pointeurs : $d = F_{total} / \text{num_selected}$

Générer un point de départ aléatoire : $\text{start_point} \sim \mathcal{U}(0, d)$

Initialiser la liste des pointeurs :

for $i \leftarrow 0$ **to** $\text{num_selected} - 1$ **do**

$\text{pointers}[i] \leftarrow \text{start_point} + i \cdot d$

Initialiser $\text{cumulative_fitness} \leftarrow 0$ et $\text{idx} \leftarrow 0$

foreach *pointer* dans *pointers* **do**

while $\text{cumulative_fitness} < \text{pointer}$ **do**

$\text{cumulative_fitness} \leftarrow$

$\text{cumulative_fitness} + \text{fitnesses}[\text{idx}]$

$\text{idx} \leftarrow \text{idx} + 1$

 Ajouter $\text{population}[\text{idx} - 1]$ à la liste des individus sélectionnés

Retourner la liste des individus sélectionnés

La figure 4.5 présente le principe de la SUS :

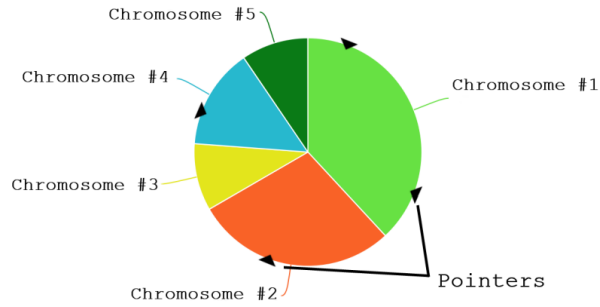


FIGURE 4.5 – Stratégie de la SUS. (CONSTANTINESCU, 2020)

4.2.4 Stratégies de mutation

Dans notre approche, le mécanisme de mutation a été revisité afin de dépasser son rôle initial de simple perturbation aléatoire. Plusieurs stratégies de mutation ont été intégrées et combinées dans le but d'améliorer l'exploration de l'espace de recherche, d'accroître la fidélité du circuit tout en évitant le piégeage dans des optima locaux, et de favoriser une optimisation structurelle plus fine, notamment par la réduction de sa profondeur. La mutation agit sur les gènes (portes quantiques) du circuit selon un taux de mutation global `mutation_rate`. Concrètement, dans un chromosome, chaque porte quantique du circuit a une probabilité p d'être mutée.

Les différentes mutations utilisées sont décrites ci-dessous :

1. **Mutation par substitution** : Une porte remplacée par une autre porte choisie aléatoirement. Ce mécanisme vise à diversifier la population sans altérer excessivement la structure du circuit.
2. **Mutation par suppression** : Il s'agit d'une mutation structurelle qui modifie directement l'architecture du circuit. Une porte est supprimée de manière aléatoire, ce qui contribue souvent à réduire la profondeur du circuit et à simplifier sa structure.
3. **Mutation par insertion** : C'est une autre mutation structurelle qui modifie directement la configuration du circuit. Une porte est insérée à une position choisie aléatoirement. Cette opération est utilisée de manière plus modérée, car elle tend à augmenter la profondeur du circuit, bien qu'elle puisse parfois introduire des structures fonctionnellement avantageuses.

Les mutations sont appliquées à tous les chromosomes de la population selon une pondération asymétrique entre les types d'opérations, tel que :

- p_1 : Probabilité associée à la mutation par substitution.
- p_2 : Probabilité associée à la mutation par suppression.
- p_3 : Probabilité associée à la mutation par insertion.

Avec : $p_1 > p_2 > p_3$

Cette répartition permet de mieux contrôler l'effet structurel des mutations, en favorisant certains types selon les objectifs visés (comme la diversification ou la simplification).

L'algorithme 5 illustre concrètement le mécanisme de mutation appliqué aux chromosomes.

Algorithm 5: Algorithme de mutation d'un chromosome

procédure *Mutate*(*chromosome*, *num_qubits*, *mutation_rate*) :**Entrée :***chromosome* : liste de gènes représentant un individu*num_qubits* : nombre de qubits utilisés pour générer un gène*mutation_rate* : probabilité de mutation d'un gène**Sortie :** Le chromosome potentiellement muté**begin** **if** *rand()* < *mutation_rate* **et** $|chromosome| > 0$ **then** Sélectionner un indice aléatoire $idx \in [0, |chromosome| - 1]$ $chromosome[idx] \leftarrow generate_gene(num_qubits)$ **if** *rand()* < 0.1 **then** **if** *rand()* < 0.5 **et** $|chromosome| > 1$ **then** Sélectionner un indice aléatoire $idx \in [0, |chromosome| - 1]$ Supprimer le gène à l'indice *idx* du chromosome **else**

Générer un nouveau gène :

 $new_gene \leftarrow generate_gene(num_qubits)$ Sélectionner une position aléatoire $idx \in [0, |chromosome|]$ Insérer *new_gene* à l'indice *idx* dans le chromosome Retourner le chromosome

4.2.5 Croisement (Crossover)

Dans notre approche, nous utilisons une méthode de **croisement classique à un point**, bien connue dans les algorithmes évolutionnaires.

Nous effectuons le croisement en sélectionnant deux parents, P1 et P2, puis en déterminant la longueur minimale entre leurs séquences respectives : $L = \min(\text{len}(P1), \text{len}(P2))$.

— Si $L < 2$, le croisement est ignoré (cas dégénéré).

— Sinon, on choisit un point de coupe *k* uniformément entre 1 et $L - 1$.

Le croisement de deux parents génère deux fils :

— **fils 1** : prend les *k* premiers gènes de P_1 et les complète avec les gènes restants de P_2 .

— **fils 2** : prend les *k* premiers gènes de P_2 et les complète avec les gènes restants de P_1 .

L'algorithme 6 illustre le déroulement du croisement à un point, permettant la génération de deux nouveaux individus à partir de deux parents.

Algorithm 6: Algorithme de croisement entre deux chromosomes**procédure** *Crossover*(*parent1*, *parent2*) :**Entrée :***parent1* : premier chromosome parent*parent2* : second chromosome parent**Sortie :** Deux enfants résultant du croisement**begin**Calculer la longueur minimale : $min_length \leftarrow \min(|parent1|, |parent2|)$ **if** $min_length < 2$ **then**└ Retourner *copy*(*parent1*), *copy*(*parent2*)**else**

Choisir un point de coupure aléatoire :

 $cut_point \sim \mathcal{U}(1, min_length - 1)$

Construire les enfants :

 $child1 \leftarrow parent1[0 : cut_point] + parent2[cut_point :]$ $child2 \leftarrow parent2[0 : cut_point] + parent1[cut_point :]$ └ Retourner *child1*, *child2***4.2.6 Recherche locale des angles de rotation**

Dans les circuits quantiques paramétrés, certaines portes — comme $R_X(\theta)$, $R_Y(\theta)$ ou $R_Z(\theta)$ — dépendent d'un angle de rotation θ . L'ajustement fin de ces paramètres peut améliorer significativement la fidélité du circuit généré.

L'objectif est d'explorer localement différentes valeurs des angles θ des portes de rotation dans un chromosome donné, afin de maximiser la fonction de fitness. Cette étape agit comme une descente de gradient centrée sur les paramètres continus du circuit.

Méthodologie : Pour chaque gène du type $R_X(\theta)$, $R_Y(\theta)$ ou $R_Z(\theta)$, on procède comme suit :

- Nous estimons le gradient de la fonction de fitness $\mathcal{F}(\theta)$ autour de l'angle courant à l'aide de la formule de différences finies centrée :

$$\nabla_{\theta}\mathcal{F} \approx \frac{\mathcal{F}(\theta + \delta) - \mathcal{F}(\theta - \delta)}{2\delta} \quad (4.2.3)$$

Cette formule est une approximation numérique de la dérivée première $\frac{d\mathcal{F}}{d\theta}$ en θ .

Autrement dit, cette méthode permet de calculer une **dérivée approchée** en évaluant la fonction de fitness en deux points symétriques autour de l'angle courant.

- Ensuite, on met à jour l'angle selon la règle :

$$\theta_{\text{new}} = \theta + \eta \cdot \nabla_{\theta}\mathcal{F} \quad (4.2.4)$$

où :

- $\nabla_{\theta}\mathcal{F}$: le gradient de la fonction de fitness $\mathcal{F}$ par rapport à θ .
- θ : l'angle actuel (paramètre du circuit quantique),

- θ_{new} : l'angle mis à jour après optimisation,
- η : le taux d'apprentissage, choisi parmi un ensemble de valeurs à tester "eta_range".
- δ : petit pas utilisé pour l'estimation numérique du gradient (typiquement $\delta = 10^{-3}$).

L'algorithme 7 montre le déroulement général de la procédure de recherche locale des paramètres de rotation, appliquée aux portes paramétrées du circuit afin d'améliorer la fidélité.

Algorithm 7: Recherche locale des paramètres de rotation (schéma général)

```

procédure Recherche_Paramètres_Rotation( $\mathcal{C}, \mathcal{U}_{\text{cible}}, \mathcal{E}_\eta, \delta, K$ ) :
  Entrée :
   $\mathcal{C}$  : chromosome (circuit) contenant des portes paramétrées  $R_X(\theta), R_Y(\theta), R_Z(\theta)$ 
   $\mathcal{U}_{\text{cible}}$  : unitaire cible
   $\mathcal{E}_\eta$  : ensemble de taux d'apprentissage candidats (p. ex.  $\{0.01, 0.1, 0.5\}$ )
   $\delta$  : pas pour l'estimation du gradient
   $K$  : nombre de passes de raffinement (itérations locales)
  Sortie : chromosome  $\mathcal{C}$  mis à jour
  begin
    for  $k \leftarrow 1$  to  $K$  do
      for chaque porte paramétrée  $R_\alpha(\theta)$  dans  $\mathcal{C}$  do
        Évaluer la fitness  $\mathcal{F}(\theta + \delta)$  et  $\mathcal{F}(\theta - \delta)$ 
         $\nabla_\theta \mathcal{F} \leftarrow \frac{\mathcal{F}(\theta + \delta) - \mathcal{F}(\theta - \delta)}{2\delta}$ 
         $F^* \leftarrow -\infty, \quad \theta^* \leftarrow \theta$ 
        for chaque  $\eta \in \mathcal{E}_\eta$  do
           $\theta' \leftarrow \theta + \eta \cdot \nabla_\theta \mathcal{F}$ 
          Évaluer  $\mathcal{F}(\theta')$ 
          if  $\mathcal{F}(\theta') > F^*$  then
             $F^* \leftarrow \mathcal{F}(\theta'), \quad \theta^* \leftarrow \theta'$ 
        Mettre à jour la porte  $R_\alpha$  avec  $\theta^*$ 
      if gain de fitness global  $< \varepsilon$  then
        break
    return  $\mathcal{C}$ 

```

Nous appliquons la recherche locale *uniquement* aux $k = 15$ meilleurs chromosomes (classés par fitness) **après** les étapes de croisement et de mutation. Pour chacun de ces chromosomes, **toutes** les portes paramétrées présentes dans le circuit sont traitées, c'est-à-dire l'ensemble des $R_X(\theta), R_Y(\theta), R_Z(\theta)$ et toute autre porte unitaire dépendant d'angles (par exemple $P(\lambda), U_3(\theta, \phi, \lambda)$ ou leurs versions contrôlées). Elle peut également être utilisée de façon itérative pour effectuer plusieurs passes d'amélioration.

4.2.7 Raffinement structurel de circuit

L'optimisation structurelle d'un circuit quantique vise à réduire sa complexité sans altérer sa fonctionnalité. Cela repose notamment sur l'exploitation d'identités algébriques entre portes quantiques consécutives agissant sur les mêmes qubits. Deux transformations fondamentales sont généralement utilisées : *l'annulation de portes* et *la fusion de portes de rotation*.

1. Annulation de portes involutives

Certaines portes quantiques, dites involutives, sont égales à leur propre inverse. Autrement dit, leur application deux fois de suite équivaut à une opération identité :

$$U \cdot U = I$$

Nous appliquons cette propriété sur les portes à fort coût, notamment le CNOT :

$$\text{CNOT} \cdot \text{CNOT} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = I$$

Ainsi, deux portes CNOT consécutives peuvent être supprimées sans altérer le résultat du circuit. Cette propriété est également valable pour d'autres portes involutives usuelles, telles que :

$$X \cdot X = I \quad , \quad H \cdot H = I \quad , \quad Z \cdot Z = I \quad , \quad Y \cdot Y = I$$

Certaines portes ne sont pas involutives, mais possèdent des relations de composition utiles. C'est le cas des portes de phase discrètes, dont les compositions successives mènent à d'autres portes :

$$S \cdot S = Z, \quad T \cdot T = S, \quad T^4 = Z$$

L'algorithme 8 illustre la procédure d'annulation des portes quantiques consécutives qui se neutralisent afin de simplifier le circuit.

Algorithm 8: Annulation de portes involutives

```

procédure CANCEL_INVOLUTIVE_GATES(circuit_original) :
  Entrée : circuit_original séquence de portes quantiques
  Sortie : circuit_optimisé avec portes redondantes supprimées
  begin
    circuit_optimisé  $\leftarrow \emptyset$ 
    portes_à_ignorer  $\leftarrow \emptyset$ 
    for  $i \leftarrow 0$  to  $\|circuit\_original\| - 1$  do
      if  $i \in portes\_à\_ignorer$  then
        continuer
       $U \leftarrow$  porte à la position  $i$ 
      if  $i < \|circuit\_original\| - 1$  then
         $V \leftarrow$  porte à la position  $i + 1$ 
        if  $qubits(U) = qubits(V)$  then
          if  $type(U) = type(V)$  et  $type(U) \in \{X, Y, Z, H, CNOT\}$  then
            portes_à_ignorer  $\leftarrow portes\_à\_ignorer \cup \{i + 1\}$ 
            continuer
          if  $type(U) = S$  et  $type(V) = S$  then
            Ajouter  $Z$  sur  $qubits(U)$  à circuit_optimisé
            portes_à_ignorer  $\leftarrow portes\_à\_ignorer \cup \{i + 1\}$ 
            continuer
          if  $type(U) = T$  et  $type(V) = T$  then
            Ajouter  $S$  sur  $qubits(U)$  à circuit_optimisé
            portes_à_ignorer  $\leftarrow portes\_à\_ignorer \cup \{i + 1\}$ 
            continuer
          if  $type(U) = T$  et  $i + 3 < |circuit\_original|$  then
            Vérifier que  $U, V, porte_{i+2}, porte_{i+3}$  sont tous  $T$  appliqués au
            même qubit
            if condition vraie then
              Ajouter  $Z$  sur  $qubits(U)$  à circuit_optimisé
              portes_à_ignorer  $\leftarrow$ 
                 $portes\_à\_ignorer \cup \{i + 1, i + 2, i + 3\}$ 
              continuer
        continuer
      Ajouter  $U$  à circuit_optimisé
  return circuit_optimisé

```

2. Fusion de portes de rotation

Les portes de rotation autour d'un même axe peuvent être combinées en une seule rotation d'angle total. Cette simplification repose sur les propriétés exponentielles des opérateurs unitaires. On considère notamment les rotations autour des axes x , y et z :

$$R_Z(\theta) = e^{-i\theta Z/2}, \quad R_X(\theta) = e^{-i\theta X/2}, \quad R_Y(\theta) = e^{-i\theta Y/2}$$

Puisque les matrices de Pauli X , Y et Z commutent avec elles-mêmes, deux rotations successives sur un même axe peuvent être fusionnées :

$$R_Z(\theta_1) \cdot R_Z(\theta_2) = R_Z(\theta_1 + \theta_2)$$

Des règles analogues s'appliquent aux rotations R_X et R_Y .

L'algorithme 9 illustre la fusion des rotations successives sur un même qubit.

Algorithm 9: Fusion des rotations successives sur le même qubit

procédure *Merge_rotations*(*circuit*) :

Entrée :

circuit : circuit quantique (objet `QuantumCircuit`)

Sortie : Nouveau circuit avec les rotations successives fusionnées

begin

num_qubits $\leftarrow$ *circuit.num_qubits*

new_circuit $\leftarrow$ `QuantumCircuit`(*num_qubits*)

i $\leftarrow$ 0

while *i* < |*circuit.data*| **do**

 (*gate*, *qargs*) $\leftarrow$ *circuit.data*[*i*]

if *gate.name* $\in$ {*rx*, *ry*, *rz*} **then**

angle $\leftarrow$ *gate.params*[0]

j $\leftarrow$ *i* + 1

while *j* < |*circuit.data*| **do**

 (*next_gate*, *next_qargs*) $\leftarrow$ *circuit.data*[*j*]

if *next_gate.name* = *gate.name* **et** *next_qargs* = *qargs* **then**

angle $\leftarrow$ *angle* + *next_gate.params*[0]

j $\leftarrow$ *j* + 1

else

break

qindex $\leftarrow$ index de *qargs*[0] dans *circuit.qubits*

 Appliquer *gate.name*(*angle*, *qindex*) dans *new_circuit*

i $\leftarrow$ *j*

else

qindices $\leftarrow$ indices de *qargs* dans *circuit.qubits*

 append(*gate*, *qindices*) dans *new_circuit*

i $\leftarrow$ *i* + 1

 Retourner *new_circuit*

4.2.8 Contrôle dynamique de la diversité

Dans notre algorithme, lorsqu'on applique une mutation, celle-ci n'est pas automatiquement acceptée. Elle est d'abord testée : si la mutation conduit à une augmentation de la fidélité, elle est acceptée, sinon elle est rejetée. Cependant, cette approche peut entraîner une stagnation de la valeur de fitness. Pour éviter ce problème, on a recours à d'autres techniques :

1. **Acceptation via recuit simulé** : Certaines mutations qui dégradent temporairement la fitness peuvent être acceptées avec une probabilité donnée par le schéma du *recuit simulé* :

$$P = \exp\left(\frac{\Delta}{T}\right) \quad (4.2.5)$$

où Δ est la perte de fitness, et T une température qui décroît au fil des générations.

Cela permet de sortir des minima locaux et d’explorer des solutions moins évidentes mais potentiellement meilleures à long terme.

2. **Ajustement dynamique du taux de mutation** : Afin d’équilibrer entre exploration et convergence, le taux de mutation est ajusté dynamiquement en fonction de la stagnation observée dans l’évolution de la fitness.

En cas de stagnation prolongée (c’est-à-dire, aucune amélioration significative), le taux de mutation est augmenté selon la règle suivante :

- Si la valeur de fitness reste identique ou n’évolue que très légèrement pendant plus de 15 itérations, le taux de mutation est mis à jour comme suit :

$$\text{mutation_rate} = \text{mutation_rate} + (\text{stagnation_count} \times 0.05) \quad (4.2.6)$$

avec :

$$\text{mutation_rate} = \min(\text{mutation_rate}, 1.0) \quad (4.2.7)$$

et

`stagnation_count` : c’est le nombre d’itérations de suite sans amélioration de la fitness.

Autrement dit, le taux de mutation augmente progressivement en fonction du nombre d’itérations sans amélioration, sans jamais dépasser 100%.

3. **Remplacement aléatoire de $\alpha\%$ des individus** : Si l’ajustement dynamique du taux de mutation ne parvient pas à restaurer une diversité suffisante au sein de la population, et que la stagnation persiste au-delà de 30 itérations, une stratégie de remplacement est appliquée. Dans ce cas, un pourcentage $\alpha\%$ d’individus est remplacé aléatoirement, à l’exception des individus élites, qui sont conservés pour préserver les meilleures solutions trouvées.

Bien que l’algorithme mono-objectif centré sur la fidélité fournisse un socle expérimental solide, il montre ses limites sur matériel NISQ : la maximisation de la seule fidélité tend à accroître la profondeur et le nombre de portes multi-qubits (p. ex. CNOT), ce qui dégrade la réalisabilité physique et la robustesse au bruit. Ces constats motivent une extension *multi-objectifs* qui optimise simultanément la fidélité, la profondeur et le coût en portes — via NSGA-II. Dans la section suivante, nous présentons l’implémentation de cette approche.

4.3 Vers une approche multi-objectifs

L’approche mono-objectif, bien qu’efficace pour des tâches de synthèse ciblées, présente plusieurs limitations dès que la complexité du circuit augmente. Dans les travaux initiaux, l’optimisation consistait principalement à minimiser une fonction unique, souvent basée sur la fidélité unitaire entre le circuit candidat C et une matrice cible G.

Cependant, cette stratégie montre rapidement ses limites dans le contexte des dispositifs quantiques NISQ, pour les raisons suivantes :

- Un circuit peut atteindre une excellente fidélité théorique (proche de 1) par rapport à une matrice cible, mais présenter une **profondeur excessive** ou un **nombre élevé de portes multi-qubits**, en particulier les portes CNOT. Ces dernières sont connues pour leur **taux d’erreur élevé** et leur **coût physique important** sur

les dispositifs NISQ. Une telle complexité rend le circuit **difficilement exécutable en pratique**, car il dépasse souvent les limites de cohérence et de connectivité des machines réelles.

- La fidélité unitaire utilisée comme critère d’optimisation repose sur une comparaison mathématique avec une cible idéale. Or, elle **néglige les effets du bruit, des erreurs accumulées, et de la topologie du matériel**.
- Chercher à maximiser la fidélité conduit fréquemment à ajouter des portes ou à ajuster finement les angles de rotation, augmentant ainsi la **taille et la complexité du circuit**. Cela entre en conflit avec les contraintes NISQ, où il est crucial de maintenir les circuits **compacts, peu profonds**, et adaptés aux **architectures physiques** sous-jacentes. Ce compromis difficile entre précision logique et réalisabilité physique justifie l’introduction d’une approche plus équilibrée.

Ainsi, une généralisation naturelle de notre approche consiste à adopter une **optimisation multi-objectifs**, dans laquelle plusieurs critères sont simultanément pris en compte.

4.3.1 Architecture globale de l’algorithme

La figure ci-dessous 4.6 montre les différentes étapes de l’extension vers une optimisation à objectifs multiples, où la fidélité, la profondeur et le coût structurel sont évalués conjointement.

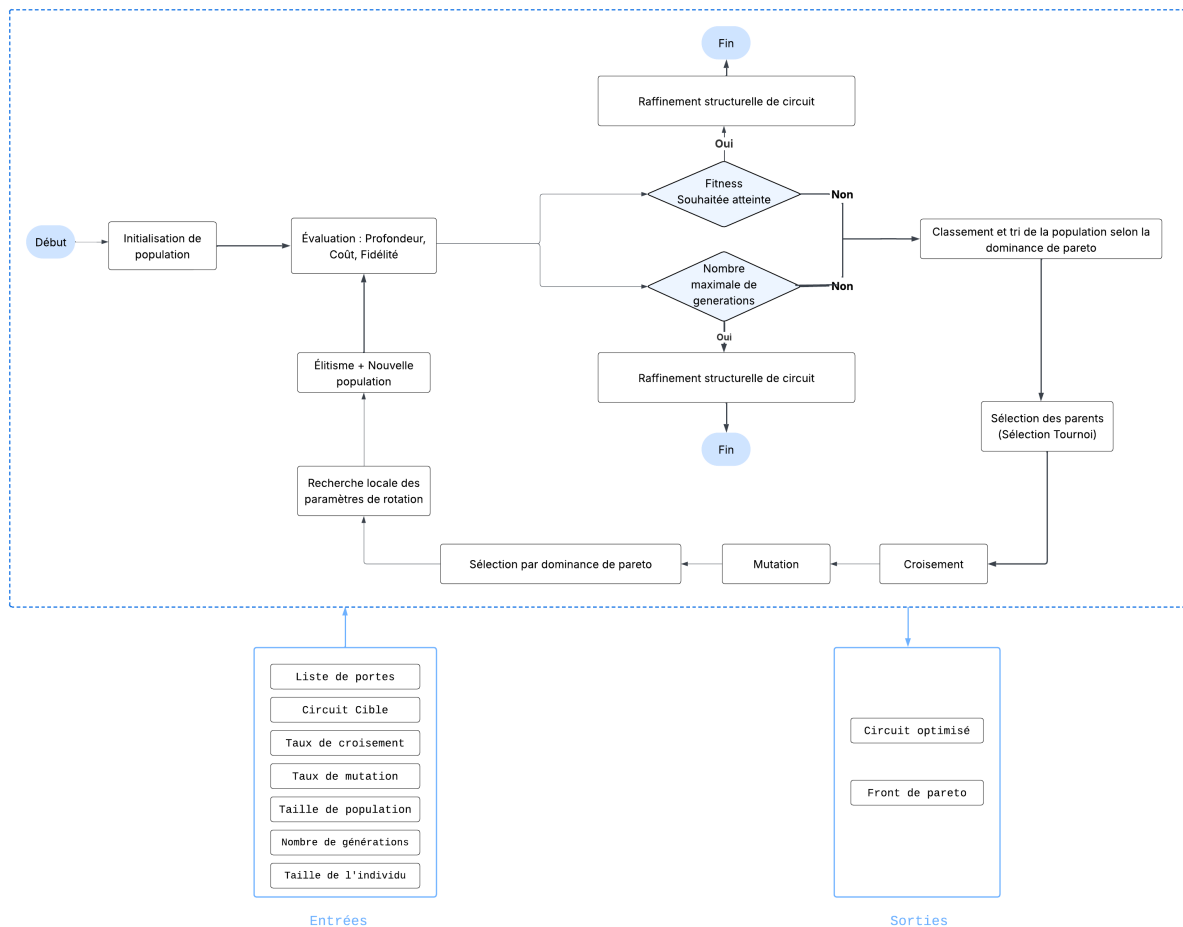


FIGURE 4.6 – Architecture de l’algorithme génétique multi-objectifs.

4.3.2 Définition des objectifs

Notre modèle multi-objectifs repose sur l'évaluation simultanée de trois dimensions fondamentales. Ce trio d'objectifs vise à **minimiser simultanément trois critères antagonistes** : (i) l'erreur de fidélité $(1 - F)$, (ii) la profondeur du circuit $D(C)$, (iii) le coût structurel en portes $\text{Cost}(C)$.

$$\min_C \left[1 - F(C), D(C), \text{Cost}(C) \right] \quad (4.3.1)$$

1. **Fidélité unitaire $\mathcal{F}$** : nous utilisons ici la même métrique que celle définie dans l'approche mono-objective (voir équation 4.2.2), qui évalue la proximité entre le circuit candidat $\lambda(C)$ et la cible G . L'erreur associée est notée $\varepsilon = 1 - \mathcal{F}$, et sert de critère d'optimisation.
2. **Profondeur du circuit D** : correspond à la longueur du chemin critique du circuit, c'est-à-dire le nombre de niveaux logiques successifs nécessaires pour exécuter toutes les portes sans conflit sur les qubits. Une profondeur plus faible est recherchée pour limiter la décohérence et rendre le circuit plus robuste au bruit temporel.

Dans le circuit de la figure 5.12 :

- Les portes monoqubit agissant sur des lignes différentes peuvent être exécutées en parallèle au même niveau logique.
- Les portes multi-qubits (comme CNOT) réservent temporairement les qubits impliqués et forcent une exécution séquentielle.
- La profondeur est obtenue en identifiant la plus longue séquence de portes successives non parallélisables sur un qubit donné.

La figure 4.7 illustre le calcul de la profondeur pour le circuit de la figure 5.12 qui est égale à 6.

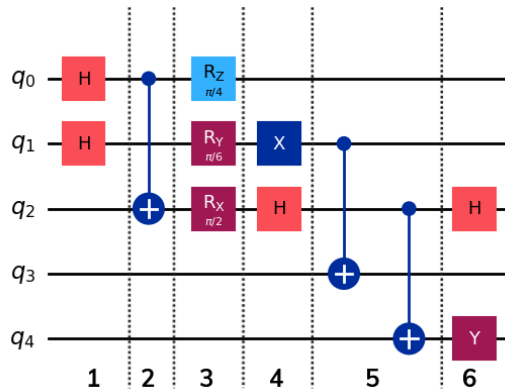


FIGURE 4.7 – Illustration du calcul de la profondeur logique : ce circuit requiert 6 niveaux pour être exécuté sans collision sur les qubits.

3. **Coût structurel C_{gate}** : nous définissons une fonction de coût basée sur la nature et la fréquence des portes utilisées dans le circuit. Ce coût reflète la complexité physique et la sensibilité au bruit de chaque opération quantique.

Les portes multi-qubits (comme CX, CZ, CCX) sont fortement pénalisées, car elles nécessitent des interactions complexes entre qubits et sont plus sujettes aux erreurs expérimentales. Le coût total est exprimé par :

$$C_{\text{gate}} = \sum_{ng \in \text{portes}} \gamma_g \cdot g \quad (4.3.2)$$

où γ_g est le poids attribué à la porte g selon sa criticité, et ng est le nombre d'occurrences de cette porte dans le circuit. Les coefficients utilisés proviennent du tableau résumé présenté dans le tableau 4.1, inspiré de (LEE et al., 2006). Par exemple, une porte **X** coûte 1 unité, tandis qu'une **CCX** en coûte 13.

Gate	Pulse Sequence	Cost
NOT	$IR_x(\pi)$	1
Phase	$e^{i\frac{\pi}{8}}R_z(\phi)$	1
Hadamard	$iR_y(\frac{\pi}{2})R_z(\pi)$	2
CNOT	$e^{-i\frac{\pi}{4}}R_{2y}(\frac{\pi}{2})R_{1z}(-\frac{\pi}{2})R_{2z}(-\frac{\pi}{2})J_{12}(\frac{\pi}{2})R_{2y}(-\frac{\pi}{2})$	5
SWAP	$e^{-i\frac{3\pi}{4}}R_{2y}(\frac{\pi}{2})R_{2z}(-\frac{3\pi}{2})R_{1z}(-\frac{3\pi}{4})J_{12}(\frac{\pi}{2})R_{2y}(\frac{\pi}{2})$ $R_{1y}(-\frac{\pi}{2})J_{12}(\frac{\pi}{2})R_{1x}(\frac{\pi}{2})R_{2x}(-\frac{\pi}{2})J_{12}(\frac{\pi}{2})$ $R_{2y}(-\frac{\pi}{2})$	11
Toffoli	$e^{-i\frac{\pi}{8}}R_{1z}(-\frac{\pi}{8})R_{2z}(-\frac{\pi}{8})J_{12}(\frac{\pi}{8})R_{3y}(-\frac{\pi}{2})$ $R_{3z}(\frac{\pi}{4})J_{31}(-\frac{\pi}{8})J_{23}(-\frac{\pi}{8})R_{3y}(\frac{\pi}{2})J_{31}(\frac{\pi}{8})$ $R_{3x}(\frac{\pi}{2})J_{23}(-\frac{\pi}{4})R_{3z}(-\frac{\pi}{2})J_{31}J_{31}(-\frac{\pi}{8})$	13

TABLE 4.1 – Résumé des coûts associés aux principales portes quantiques (LEE et al., 2006).

Le **coût structurel** d'un circuit est obtenu en associant un poids à chaque type de porte, comme présenté dans le tableau 4.1. Ainsi, pour le circuit de la figure 5.12, on calcule le coût total C_{gate} en sommant les contributions de chaque porte selon leur type.

Ce trio d'objectifs est utilisé dans le cadre d'un algorithme évolutionnaire multi-objectifs de type NSGA-II, permettant d'explorer le compromis entre ces métriques via un *front de Pareto*. Chaque individu de la population est ainsi évalué selon ces trois axes, sans les combiner en une unique valeur scalaire.

4.3.3 Les étapes héritées de l'algorithme mono-objectifs

Les principaux étapes du cycle évolutionnaire restent inchangés par rapport à l'approche mono-objective 4.2 et sont réutilisés dans le cadre multi-objectifs.

- Représentation des individus.
- Initialisation de la population.
- Opérateurs de mutation.
- Croisement (crossover).
- Recherche locale des paramètres de rotation.
- Contrôle dynamique de la diversité.
- Raffinement structurel de circuit

4.3.4 Sélection dans l'algorithme NSGA-II

Comme illustré dans la figure 4.6, l'étape de sélection intervient après l'évaluation multi-objectifs de la population. Elle repose sur deux mécanismes successifs :

- le classement par **domination de Pareto** et **distance de densité**,
- puis la **sélection par tournoi** pour la reproduction.

4.3.4.1 Classement selon la domination de Pareto et la densité de population

L'algorithme NSGA-II repose sur deux mécanismes complémentaires pour classer les individus les plus prometteurs à chaque génération : la **domination de Pareto** et la **distance de densité (crowding distance)**.

1. Domination de Pareto :

Chaque circuit candidat est évalué selon trois objectifs indépendants :

- Maximiser la fidélité unitaire $\mathcal{F}(C)$;
- Minimiser la profondeur du circuit $D(C)$;
- Minimiser le coût total en portes $C_{\text{gate}}(C)$.

Un individu A domine un individu B si :

$$A \prec B \iff \left(\mathcal{F}(A) \geq \mathcal{F}(B) \wedge D(A) \leq D(B) \wedge C_{\text{gate}}(A) \leq C_{\text{gate}}(B) \right) \text{ et} \\ \exists \text{ critère strict}$$

dit, A est meilleur ou égal à B sur les trois objectifs, et strictement meilleur sur au moins l'un d'entre eux.

2. Distance de densité (Crowding Distance)

Tous les individus d'un même front sont considérés comme équivalents en termes de domination. Pour départager ces individus et préserver une bonne diversité dans la population. Pour chaque individu i , cette distance est définie par :

$$\text{CD}(i) = \frac{\mathcal{F}(i+1) - \mathcal{F}(i-1)}{\max(\mathcal{F}) - \min(\mathcal{F})} + \frac{D(i-1) - D(i+1)}{\max(D) - \min(D)} + \frac{C_{\text{gate}}(i-1) - C_{\text{gate}}(i+1)}{\max(C_{\text{gate}}) - \min(C_{\text{gate}})} \quad (4.3.3)$$

où :

- $\mathcal{F}(i)$ est la fidélité de l'individu i ,
- $D(i)$ est sa profondeur,
- $C_{\text{gate}}(i)$ est son coût en portes,
- les indices $i-1$ et $i+1$ désignent les voisins de i après tri des individus selon chaque objectif.

Cette distance est utilisée pour sélectionner les individus les plus éloignés de leurs voisins, c'est-à-dire ceux apportant le plus de diversité. En cas de dépassement de capacité dans un front, les individus sont triés par distance de densité décroissante afin de conserver un ensemble varié de solutions.

Remarque : Le dépassement de capacité dans un front survient lorsque l'on ne peut pas insérer tous les individus d'un même front sans excéder la taille de population. Dans ce cas, un tri selon la distance de densité est effectué pour sélectionner les individus les plus diversifiés.

4.3.4.2 Sélection finale de la nouvelle population

La construction de la nouvelle génération s'effectue comme suit :

1. Les individus sont ajoutés *front par front*, en commençant par le premier ;
2. Si un front dépasse la capacité restante, on y applique un tri décroissant selon la *distance de densité* ;
3. Les individus avec la plus grande distance sont sélectionnés en priorité.

Ce mécanisme garantit un bon équilibre entre **qualité** (via la domination) et **diversité** (via la densité), ce qui est essentiel dans une recherche multi-objectifs.

Comme illustré dans l'algorithme 10, notre sélection repose à la fois sur la domination de Pareto et la dispersion locale (crowding distance) (DEB et al., 2002).

Algorithm 10: Sélection multi-objectifs avec NSGA-II (Pareto + distance de densité)

```

procédure NSGA2_Selection(population, fitness_values, pop_size) :
  Entrées :
  population : liste des individus (circuits candidats)
  fitness_values : liste de triplets ( $\mathcal{F}$ ,  $D$ ,  $C_{gate}$ ) pour chaque individu
  pop_size : taille souhaitée de la nouvelle population
  Sortie :
  liste des individus sélectionnés (taille pop_size)

  Trier la population en fronts de Pareto ordonnés :  $F_1, F_2, \dots$  ;
  selected  $\leftarrow []$  ; // Nouvelle population à construire

  foreach front dans l'ordre croissant du rang de Pareto do
    Initialisation des distances de densité foreach  $i \in \text{front}$  do
       $CD[i] \leftarrow 0$ 

    Calcul des distances de densité sur chaque objectif foreach objectif
       $m \in \{\mathcal{F}, D, C_{gate}\}$  do
        Trier front selon  $m$ ;
        Mettre  $CD[\text{first}] \leftarrow \infty$ ,  $CD[\text{last}] \leftarrow \infty$ ;
        for  $i = 2$  to  $|\text{front}| - 1$  do
           $CD[i] += \frac{m(i+1) - m(i-1)}{\max(m) - \min(m)}$ ;

    if  $|\text{selected}| + |\text{front}| \leq \text{pop\_size}$  then
      Ajouter tous les individus de front à selected;
    else
      Dépassement de capacité : sélection par diversité Trier front par  $CD$ 
      décroissante;
      Ajouter les meilleurs individus (selon  $CD$ ) jusqu'à atteindre pop_size;
      return selected;

  return selected;

```

4.3.4.3 Sélection des parents (tournoi binaire) pour le croisement

Une fois la population de la génération $t + 1$ construite, les **parents** servant à produire la génération suivante sont sélectionnés par une procédure de *tournoi binaire*, compatible avec le classement NSGA-II :

- Deux individus sont tirés aléatoirement de la population ;
- Celui ayant le **meilleur rang de Pareto** est sélectionné ;
- En cas d'égalité, on choisit celui ayant la **plus grande distance de densité**, afin de favoriser la diversité.

Les individus sélectionnés par tournoi servent ensuite à produire les enfants via les opérateurs de **croisement** et de **mutation**, comme illustré dans le schéma de l'algorithme (figure 4.6).

Malgré les apports de l'approche multi-objectifs, une difficulté majeure persiste : le calcul exact de la fonction de fitness basée sur la fidélité unitaire (équation 4.2.2). En effet, l'évaluation nécessite la construction explicite de la matrice unitaire $\lambda(C)$ de dimension $2^n \times 2^n$, ce qui entraîne une complexité en temps et en mémoire de l'ordre de $\mathcal{O}(2^{2n})$.

Pour des circuits de petite taille ($n \leq 10$), ce calcul reste envisageable et fournit une mesure fiable. Cependant, dès que le nombre de qubits dépasse une douzaine, le coût devient prohibitif et ralentit considérablement le processus évolutionnaire. Cette limitation compromet la scalabilité de l'algorithme, et restreint sa capacité à traiter des circuits plus réalistes correspondant aux dispositifs NISQ.

Afin de dépasser cette barrière, nous introduisons dans la section suivante des **stratégies d'estimation évolutives de la fidélité**. Ces méthodes reposent sur des approximations locales — telles que la décomposition en blocs indépendants ou l'évaluation par fenêtres glissantes — et permettent de conserver un compromis entre précision et efficacité, rendant possible l'optimisation de circuits de plus grande taille tout en maintenant la pertinence des résultats.

4.4 Stratégies d'estimation de fidélité évolutives

Dans le cadre du NSGA-II présenté précédemment, nous intégrons deux méthodes d'estimation de la fidélité permettant de contourner la complexité exponentielle du calcul matriciel global. Ces deux approches offrent un compromis entre précision et scalabilité, et s'adaptent aux architectures à grand nombre de qubits :

1. Évaluation de la fidélité par blocs indépendants :

cette méthode repose sur une partition du circuit en sous-blocs de petite taille (typiquement $k \leq 6$ qubits) via un graphe d'interaction. Chaque bloc est évalué indépendamment à l'aide de la métrique de trace (*trace fidelity*). Les scores locaux sont ensuite agrégés pour reconstruire une estimation globale. Cette méthode est particulièrement efficace pour les circuits faiblement dense et à connectivité locale.

2. Évaluation par blocs séquentiels chevauchants (fenêtres glissantes) :

cette méthode consiste à découper temporairement le circuit en petites fenêtres locales (3–5 qubits) se chevauchant, uniquement lors de l'évaluation de la fitness. Pour chaque paire fenêtre candidat–cible, la fidélité locale est calculée via la métrique de trace overlap. Les scores sont ensuite agrégés par produit géométrique, ce qui

pénalise les blocs mal alignés et préserve les corrélations entre fenêtres. Intégrée dans NSGA-II, elle offre une estimation scalable et robuste, particulièrement adaptée aux circuits denses et fortement denses.

4.4.1 Méthode 1 : Évaluation de la fidélité par blocs indépendants

L'objectif de cette stratégie est d'éviter le calcul de la matrice unitaire globale d'un circuit quantique, dont la complexité mémoire est $\mathcal{O}(2^{2n})$, en décomposant le circuit en sous-circuits localement évaluable. Elle repose sur une partition du circuit initial en blocs faiblement intriqués, puis sur une évaluation individuelle des sous-circuits correspondants. L'objectif est de regrouper des qubits fortement intriqués afin d'évaluer ou d'optimiser localement chaque bloc, tout en limitant les interactions inter-blocs. La figure 4.8 illustre ce processus global de partitionnement, d'optimisation locale et de réassemblage final du circuit.

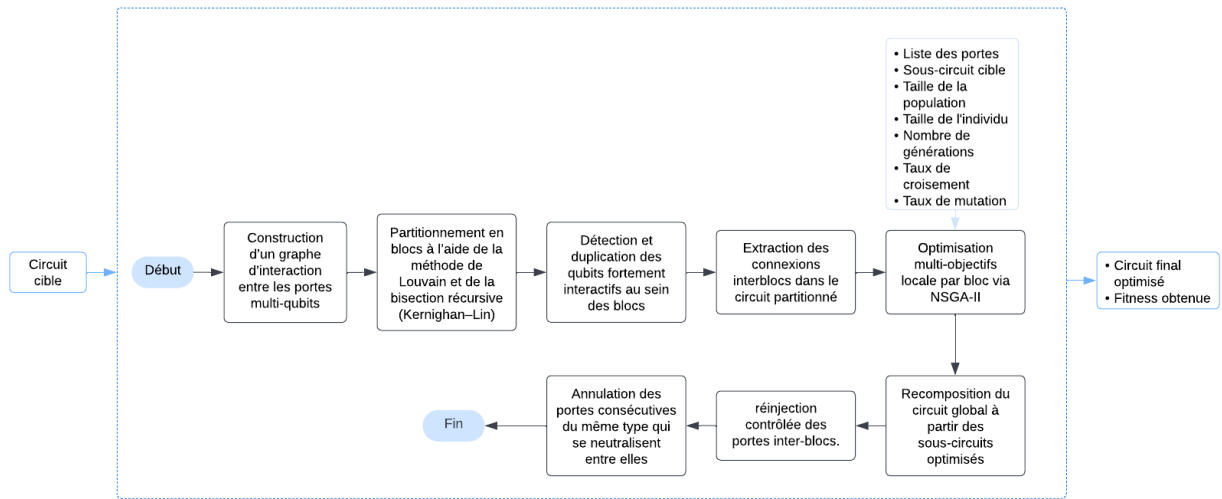


FIGURE 4.8 – Pipeline d'optimisation d'un circuit quantique reposant sur une évaluation locale de la fidélité par blocs.

Les étapes principales de ce pipeline sont présentées ci-dessous :

Étape 1 : Graphe d'interaction quantique Nous modélisons un circuit quantique C par un graphe non orienté pondéré $G = (V, E)$, appelé **graphe d'interaction**, où :

- chaque sommet $v \in V$ correspond à un qubit,
- chaque arête $(i, j) \in E$ est présente s'il existe une porte bi-qubit (par ex. CX, CZ, RZZ) entre les qubits q_i et q_j dans le circuit,
- le poids w_{ij} d'une arête est le nombre total de telles interactions.

Ce graphe est formellement défini par la matrice d'adjacence A :

$$A_{ij} = \begin{cases} w_{ij}, & \text{si } \exists \text{ interaction entre } q_i \text{ et } q_j \\ 0, & \text{sinon} \end{cases} \quad (4.4.1)$$

Étape 2 : Partitionnement en blocs Le partitionnement du circuit est réalisé en deux phases complémentaires : une détection de communautés pour identifier la structure modulaire du graphe, suivie, si nécessaire, d'un découpage plus fin pour respecter une contrainte de taille maximale. Ce processus vise à équilibrer fidélité locale, cohérence topologique, et efficacité de recomposition.

1. **Détection initiale par la méthode de Louvain** Dans un circuit quantique complexe, les qubits ne sont pas uniformément connectés : certains interagissent beaucoup plus fortement entre eux que d'autres. Pour exploiter cette structure, nous utilisons la méthode de Louvain (BLONDEL et al., 2008), un algorithme de détection de communautés appliqué au graphe d'interaction des qubits. L'idée est de regrouper naturellement les qubits fortement intriqués dans un même bloc, de manière à révéler une hiérarchie de sous-ensembles cohérents.

Le processus suit deux étapes principales :

- **Phase locale** : Chaque qubit est évalué par rapport à ses voisins immédiats et peut être déplacé vers la communauté qui maximise son « bénéfice d'interaction » (c'est-à-dire la densité de ses connexions internes par rapport aux connexions externes). Cette étape favorise des regroupements où les portes à deux qubits sont concentrées à l'intérieur d'un même bloc.
- **Phase d'agrégation** : Une fois les communautés locales stabilisées, celles-ci sont fusionnées en « super-qubits ». Le graphe réduit ainsi formé est ensuite soumis au même processus, ce qui permet de construire une structure hiérarchique multi-niveaux.

Cette approche génère une partition naturelle des qubits en blocs fortement liés. L'algorithme correspondant est présenté ci-dessous.

Algorithm 11: Détection de communautés (Louvain adaptée aux qubits)

Input: Graphe d'interaction $G = (Q, E)$ des qubits

Output: Partition hiérarchique $\mathcal{P}$ en blocs de qubits

while *amélioration possible* **do**

foreach *qubit* $q \in Q$ **do**

 Évaluer le gain de déplacer q vers chaque communauté voisine;

 Déplacer q vers la communauté offrant le plus fort gain;

 Agréger les communautés en « super-qubits » pour former un graphe réduit;

return *Partition hiérarchique* $\mathcal{P}$

2. **Affinage par bisection récursive (Hybrid METIS/KL)**

Une fois les blocs initiaux détectés par Louvain, certains peuvent rester trop volumineux pour être exploités efficacement dans un simulateur quantique ou pour évaluer localement la fidélité d'un sous-circuit. Afin de résoudre ce problème, nous appliquons une stratégie de partitionnement récursif. L'idée est de décomposer progressivement chaque bloc en sous-blocs de taille maîtrisée, tout en limitant la perte d'interactions intra-bloc. Ce processus est résumé par les algorithmes ci-dessous. L'algorithme 12 décrit le mécanisme général de la partition récursive hybride appliquée au graphe d'interaction.

Algorithm 12: Partition récursive hybride METIS/KL

Input: Graphe d'interaction $G = (Q, E)$; taille maximale $k_{\max}$

Output: Liste de sous-blocs $\mathcal{B}$

Initialiser une pile $\mathcal{S} \leftarrow [Q]$;

while $\mathcal{S}$ *non vide* **do**

 Extraire un ensemble Q' de $\mathcal{S}$;

if $|Q'| \leq k_{\max}$ **then**

 Ajouter Q' à $\mathcal{B}$;

else

$(Q_1, Q_2) \leftarrow \text{HYBRIDBIPARTITION}(Q')$;

 Empiler Q_1 et Q_2 dans $\mathcal{S}$;

return $\mathcal{B}$

L'algorithme 13 précise la logique de la sous-routine hybride, qui combine METIS et KL.

Algorithm 13: HYBRIDBIPARTITION(Q')

$(Q_1, Q_2) \leftarrow \text{Partitionner } Q' \text{ avec METIS};$

if $\max(|Q_1|, |Q_2|) > k_{\max}$ **then**

$(Q_1, Q_2) \leftarrow \text{Raffiner } (Q_1, Q_2) \text{ avec KL};$

return (Q_1, Q_2)

Étape 3 : duplication de qubits fortement interactifs Avant l'optimisation locale intra-bloc, une analyse est menée pour détecter les qubits fortement impliqués dans des portes inter-blocs. Ces qubits sont candidats à la duplication dans d'autres blocs pour préserver leur influence sans alourdir la phase d'injection.

Soit q_i un qubit appartenant à un bloc B_k . On note :

- $g_{\text{tot}}(q_i)$: nombre total de portes (multi-qubits) appliquées à q_i ,
- $g_{\text{inter}}(q_i, B_j)$: nombre de portes entre q_i et le bloc $B_j \neq B_k$,
- $\rho(q_i, B_j) = \frac{g_{\text{inter}}(q_i, B_j)}{g_{\text{tot}}(q_i)}$: **ratio d'interaction inter-bloc**.

Un qubit q_i est considéré comme fortement interactif avec le bloc B_j s'il vérifie :

$$\rho(q_i, B_j) \geq \tau \quad (\text{ex. } \tau = 0,6) \quad (4.4.2)$$

Dans ce cas, q_i est dupliqué dans le bloc B_j (si la taille du bloc le permet).

Cette duplication permet de rendre le qubit accessible localement dans plusieurs blocs tout en limitant les futurs échanges inter-blocs.

L'algorithme 14 résume le déroulement de duplication des qubits fortement interactifs.

Algorithm 14: DUPLICATE HIGH INTERACTION QUBITS($C, B, \mathcal{G}_{\text{inter}}, \tau, k_{\text{max}}$)

Input: Circuit C ; blocs $B = \{B_1, \dots, B_s\}$; portes inter-blocs $\mathcal{G}_{\text{inter}}$; seuil de duplication τ ; taille max de bloc k_{max} .

Output: Blocs mis à jour B et mapping des duplications $\mathcal{M}_{\text{dup}}$.

$\mathcal{M}_{\text{dup}} \leftarrow \emptyset$ // Initialisation

Mémorise les qubits dupliqués, dans quels blocs, et avec quels identifiants.

```

foreach qubit  $q_i$  do
    Calculer  $g_{\text{tot}}(q_i) \leftarrow$  nombre total de portes sur  $q_i$ 
    foreach bloc  $B_j \neq B_k$  où  $q_i \in B_k$  do
         $g_{\text{inter}}(q_i, B_j) \leftarrow$  nombre de portes entre  $q_i$  et  $B_j$ 
         $\rho(q_i, B_j) \leftarrow \frac{g_{\text{inter}}(q_i, B_j)}{g_{\text{tot}}(q_i)}$  // Comptage des interactions
    end
end

foreach qubit  $q_i$  do
    foreach bloc  $B_j \neq B_k$  où  $\rho(q_i, B_j) \geq \tau$  et  $|B_j| + 1 \leq k_{\text{max}}$  do
        if non déjà dupliqué vers  $B_j$  then
            Créer un nouveau qubit  $q_i^{\text{dup}}$  dans  $B_j$ 
            Ajouter  $(q_i \mapsto q_i^{\text{dup}}, B_j)$  à  $\mathcal{M}_{\text{dup}}$  // Sélection des duplications
        end
    end
end

return  $B, \mathcal{M}_{\text{dup}}$ 

```

Étape 4 : Optimisation locale multi objectives par bloc avec NSGA-II Chaque bloc extrait est optimisé de manière indépendante à l’aide de NSGA-II décrit précédemment dans la section 4.3, en tenant compte les trois objectifs comme la fidélité locale, la profondeur, et le coût topologique. Cela permet de réduire le coût global tout en respectant les contraintes de chaque sous-circuit.

Étape 5 : réassemblage global Une fois tous les sous-circuits optimisés, ils sont recombinaés en un seul circuit global via `recompose_from_blocks()`, qui respecte l’ordre des opérations du circuit d’origine.

Étape 6 : réinjection contrôlée des portes inter-blocs Une fois le circuit recomposé, les portes inter-blocs extraites à l’étape 2 sont réintroduites de manière contrôlée. On distingue ici deux cas :

- Si les qubits impliqués dans une porte inter-bloc ont été dupliqués, la version dupliquée est utilisée pour exécuter la porte de manière purement intra-bloc.
- Sinon, la porte est injectée entre un qubit du bloc source et un qubit du bloc cible, en s’assurant que les conditions de connectivité et de fidélité soient satisfaites.

la stratégie d’injection est :

- **Stratégie stochastique guidée par la fidélité** : injecte une porte aléatoire, et l’accepte uniquement si la fidélité globale augmente :

$$\text{gain} = \mathcal{F}_{\text{après}} - \mathcal{F}_{\text{avant}} > 0 \quad (4.4.3)$$

Chaque porte inter-bloc injectée est donc validée par sa contribution réelle à l’amélioration de la fidélité du circuit global.

Étape 7 : compression finale Une dernière étape d’optimisation structurelle est appliquée au circuit global recomposé, en utilisant les méthodes décrites dans la section 4.2.7 (*Raffinement structurel de circuit*). À l’issue de cette étape, la fidélité globale du circuit est calculée en appliquant la relation présentée à l’équation 4.4.6, ce qui permet d’agréger les contributions de chaque bloc tout en tenant compte des approximations introduites lors de la décomposition et de l’optimisation.

4.4.2 Méthode 2 : Évaluation de la fidélité par blocs séquentiels chevauchants (fenêtres glissantes)

L’objectif de cette seconde stratégie est de contourner la complexité exponentielle liée au calcul de la matrice unitaire globale d’un circuit quantique, tout en évitant le recours au partitionnement explicite en sous-circuits optimisés séparément. Elle adopte une approche plus adaptée aux circuits fortement denses et intriqués, en s’appuyant sur un schéma de fenêtre glissante. Celui-ci consiste à évaluer la fidélité du circuit par portions successives de portes, avec un léger chevauchement entre fenêtres afin de préserver les corrélations locales. Cette approximation séquentielle permet d’obtenir une estimation fiable de la fitness globale, sans nécessiter la construction de l’unitaire complet. La méthode se révèle ainsi particulièrement efficace pour les circuits peu modulaires, où les techniques de partitionnement atteignent rapidement leurs limites.

La figure 4.9 illustre ce processus global de notre NSGA-II après l’intégration de l’approche des blocs séquentiels chevauchants dans l’évaluation de la fidélité.

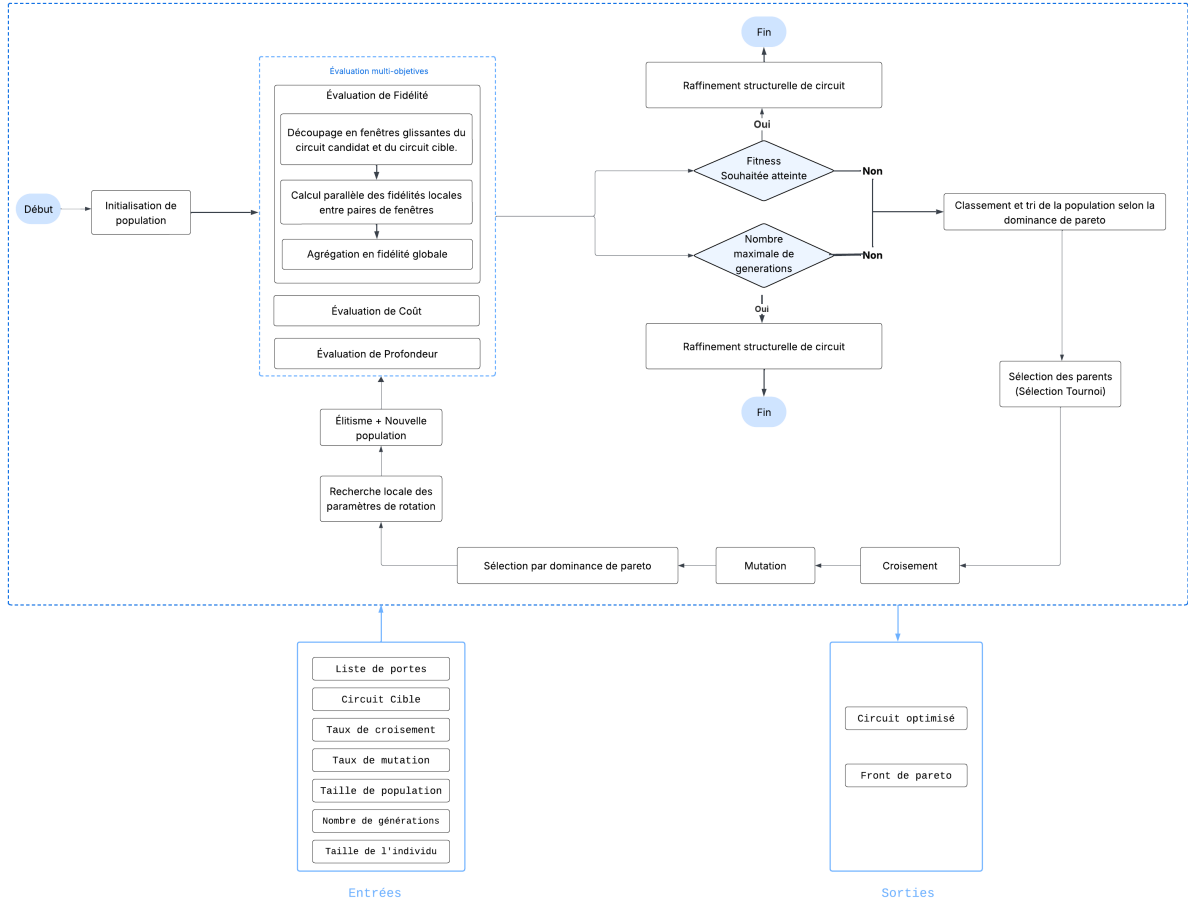


FIGURE 4.9 – Pipeline d’optimisation d’un circuit quantique reposant sur une évaluation de la fidélité par blocs séquentiels chevauchants.

Étape 1 : Découpage séquentiel en fenêtres glissantes Le circuit candidat et le circuit cible sont découpés en une série de sous-circuits locaux appelés *fenêtres*.

Chaque fenêtre regroupe un ensemble contigu de k qubits ($k = 3 \sim 6$ typiquement), avec un chevauchement de o qubits ($o = 1 \sim 2$) entre fenêtres successives.

Ainsi, une portion du circuit de n qubits est couverte par environ :

$$\frac{n}{k - o} \text{ fenêtres.} \quad (4.4.4)$$

Ce schéma de fenêtres glissantes permet de conserver une continuité de l’intrication entre blocs, tout en maintenant chaque sous-circuit dans une dimension calculable.

Étape 2 : Évaluation locale de la fidélité Pour chaque fenêtre, on extrait le sous-circuit candidat U_i et le sous-circuit cible V_i , puis on calcule la fidélité locale via la métrique de *trace overlap* :

$$F(U, V) = \frac{|\text{Tr}(UV^\dagger)|}{d}, \quad d = 2^n \quad (4.4.5)$$

où U est l’unitaire du bloc courant et V la portion correspondante du circuit cible.

L’ensemble des fidélités locales est ensuite agrégé pour obtenir une mesure globale.

Étape 3 : Agrégation en fidélité globale La fidélité globale du circuit candidat est obtenue par *produit géométrique* des fidélités locales :

$$F_{\text{global}} = \prod_{i=1}^m F_i \quad (4.4.6)$$

Cette agrégation présente deux avantages :

- Elle pénalise fortement les blocs où la correspondance est faible, évitant ainsi qu'un sous-circuit mal aligné ne soit masqué par d'autres.
- Elle fournit une métrique robuste même en présence d'un grand nombre d'interactions inter-fenêtres.

Étape 4 : Estimation multi-objectifs locale Contrairement à la méthode 1, il n'y a pas d'optimisation indépendante par fenêtre. Le NSGA-II agit directement sur le circuit global, mais il utilise la fitness issue de l'agrégation par fenêtres comme signal d'évaluation. Outre la fidélité, chaque circuit candidat est évalué selon :

- sa profondeur $d(C)$,
- son coût topologique $Cost(C)$ (pondération des portes utilisées).

Le vecteur de fitness associé à un circuit C est donc défini par :

$$\text{Fitness}(C) = [F_{\text{global}}(C), -d(C), -Cost(C)] \quad (4.4.7)$$

Ce vecteur est ensuite utilisé dans l'algorithme multi-objectifs **NSGA-II** pour guider l'évolution de la population.

Étape 5 : Validation et correction Étant donné que la fidélité issue de la fenêtre glissante constitue une approximation, une phase de validation est introduite afin d'assurer la fiabilité du processus d'optimisation.

Lorsque c'est possible, la fidélité exacte du circuit obtenu est calculée. Cette valeur sert alors de référence pour comparer la fidélité approchée obtenue via l'évaluation par blocs chevauchants. L'objectif est de mesurer l'écart entre la valeur réelle et son approximation locale.

- **Si l'écart reste faible** : l'approximation est jugée suffisamment représentative et le circuit est transmis directement à l'algorithme évolutionnaire.
- **Si l'écart est significatif** : cela indique que certaines parties du circuit candidat ne reproduisent pas correctement le comportement attendu.

Dans ce **second cas**, des corrections locales ciblées sont appliquées sur les blocs identifiés comme fautifs. Ces ajustements peuvent prendre plusieurs formes :

- réglage fin des angles de rotation des portes,
- suppression ou remplacement de portes redondantes,
- injection contrôlée de portes multi-qubits aléatoires afin de réintroduire de l'intrication manquante.

Ces corrections permettent de réaligner progressivement la dynamique locale du circuit candidat avec celle du circuit cible, sans nécessiter un recalcul complet de la matrice unitaire globale.

Enfin, après cette étape de correction initiale, le circuit est systématiquement soumis aux méthodes de raffinement structurel décrites dans la section 4.2.7. Ces méthodes (telles que l’annulation de portes inverses consécutives ou la fusion de rotations) permettent de réduire la profondeur, de simplifier la topologie du circuit et de fiabiliser davantage la fidélité obtenue.

Ce mécanisme garantit une convergence progressive vers une fidélité plus fiable, malgré l’approximation initiale. Les étapes de cette approche sont simplifiées via l’algorithme 15 ci-dessous :

Algorithm 15: Évaluation par fenêtres glissantes

procédure *Évaluer_Par_Fenêtres_Glissantes*(U, V, k, o) :

Entrée :

U : circuit candidat

V : circuit cible

k : taille de la fenêtre (3 à 6 qubits)

o : chevauchement entre fenêtres successives (1 à 2 qubits)

Sortie : F_{global} : fidélité globale du circuit candidat

begin

 Découper U et V en une suite de m fenêtres $\{U_i, V_i\}_{i=1..m}$

 Chaque fenêtre couvre k qubits avec un chevauchement de o qubits

 Initialiser $F_{\text{global}} \leftarrow 1$

for $i \leftarrow 1$ **to** m **do**

 Extraire le sous-circuit candidat U_i et le sous-circuit cible V_i

 Calculer la fidélité locale :

$$F_i = \frac{|\text{Tr}(U_i V_i^\dagger)|}{2^k}$$

 Mettre à jour la fidélité globale :

$$F_{\text{global}} \leftarrow F_{\text{global}} \times F_i$$

 Retourner F_{global}

Conclusion

Ce chapitre a présenté notre approche fondée sur un algorithme génétique pour la conception automatique de circuits quantiques optimisés. Après avoir mis en évidence les limites d’une formulation mono-objectif, nous avons proposé une extension multi-objectifs s’appuyant sur l’algorithme NSGA-II. Cette extension permet d’optimiser simultanément la fidélité, la profondeur et le coût structurel des circuits, répondant ainsi aux contraintes spécifiques des dispositifs NISQ. Nous avons également introduit des méthodes évolutives d’estimation de la fidélité, basées sur une décomposition en blocs, garantissant une scalabilité adaptée aux circuits quantiques de grande taille. Les détails de cette mise en œuvre seront présentés dans le chapitre suivant.

Chapitre 5

Réalisation

Introduction

Après avoir défini dans le chapitre précédent notre approche basée sur un algorithme génétique pour la conception automatique de circuits quantiques optimisés, nous abordons à présent sa mise en œuvre concrète. Ce chapitre est consacré à la description technique et pratique de la réalisation de notre solution, depuis le choix des outils jusqu'à l'évaluation expérimentale. Nous détaillons dans un premier temps l'environnement de développement ainsi que les bibliothèques et technologies utilisées. Nous expliquons ensuite la structure du code, l'implémentation des principales composantes de l'algorithme génétique (représentation des individus, opérateurs génétiques, fonctions de coût, etc.) ainsi que les étapes d'adaptation spécifiques au contexte quantique. Une attention particulière est portée au traitement des contraintes matérielles des dispositifs NISQ, ainsi qu'à l'intégration des critères multi-objectifs dans le processus d'optimisation. Enfin, nous présentons les jeux de tests utilisés, les scénarios expérimentaux mis en place, et les résultats intermédiaires obtenus, afin de valider le bon fonctionnement et l'efficacité de notre approche.

5.1 Technologies utilisées

Qiskit¹ est un framework open-source développé par IBM Research pour la programmation quantique. Il permet d'écrire des programmes en Python et de les exécuter sur des simulateurs quantiques ou des ordinateurs quantiques réels. Qiskit offre des outils pour concevoir, optimiser et exécuter des circuits quantiques, facilitant ainsi l'exploration de l'informatique quantique. Dans notre projet, Qiskit est utilisé pour construire les circuits quantiques, extraire leur matrice unitaire via `Operator`, et évaluer leur profondeur réelle grâce à la fonction `transpile()` combinée à une architecture cible réaliste.

PennyLane² est une bibliothèque Python open-source conçue pour la programmation quantique et l'apprentissage automatique quantique. Développée par Xanadu, elle permet d'exécuter et d'entraîner des programmes quantiques sur divers simulateurs et matériels quantiques. PennyLane facilite l'intégration des circuits quantiques avec des frameworks de machine learning comme PyTorch et TensorFlow, permettant ainsi des calculs hybrides quantiques-classiques. Son approche repose sur la programmation différentiable, ce qui la rend particulièrement utile pour l'optimisation de modèles quantiques.

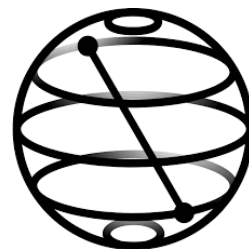


FIGURE 5.1 – Logo de Qiskit.



FIGURE 5.2 – Logo de PennyLane.

1. <https://qiskit.org>

Atos QLM (Quantum Learning Machine) est un serveur de la plateforme ROMEO³ est un environnement de simulation quantique haute performance. Il permet d'exécuter des circuits sur des simulateurs classiques capables de simuler jusqu'à 40 qubits avec une grande précision, ce qui est crucial pour les études à grande échelle.

Dans notre projet, nous avons utilisé le simulateur QLM pour évaluer la fidélité de circuits de taille intermédiaire, comparer la performance de l'algorithme mono-objectif vs multi-objectifs, et observer l'impact du raffinement structurel sur la profondeur et le coût. Ce serveur a été particulièrement utile pour contourner les limites de Qiskit Aer en local (max 39 qubits).

NumPy est une bibliothèque fondamentale pour le calcul numérique en Python. Elle permet de manipuler efficacement des tableaux multidimensionnels, de réaliser des opérations linéaires et matricielles complexes, et de générer des variables aléatoires suivant des lois spécifiques. Dans notre implémentation, elle intervient dans le calcul de la fidélité ($\text{trace}(U^\dagger V)$), la normalisation des résultats, et la génération aléatoire des paramètres de rotation des portes quantiques.

Matplotlib est une bibliothèque de visualisation scientifique en 2D. Elle est utilisée ici pour tracer l'évolution de l'erreur de fidélité à chaque génération (`plot_convergence`) ainsi que pour représenter le front de Pareto selon les dimensions profondeur, coût et fidélité (`plot_pareto`). Ces graphiques facilitent l'analyse de la performance de l'algorithme et la comparaison des solutions obtenues.

NetworkX un outil Python spécialisé dans la modélisation et l'analyse de graphes. Elle nous a permis de représenter un circuit quantique sous forme de graphe d'interaction pondéré.

Multiprocessing est un module standard de Python permet d'exploiter plusieurs cœurs de processeur pour accélérer les tâches parallélisables. Dans notre cas, il est utilisé pour évaluer en parallèle les individus d'une population à chaque génération. Cette parallélisation est cruciale car le calcul de la fidélité (basé sur la matrice unitaire complète) est coûteux, notamment au-delà de 4–5 qubits.

IBM Quantum Composer C'est un outil de programmation quantique graphique qui vous permet de créer des circuits quantiques en faisant glisser et déposer des opérations. Il vous offre la possibilité d'exécuter ces circuits sur du matériel quantique réel ou des simulateurs, de visualiser l'état des qubits et de générer automatiquement du code OpenQASM ou Python.

La figure figure 5.3 met en évidence l'interface graphique proposée par IBM Quantum Composer, pour l'implémentation de circuits quantiques sur simulateur.

2. <https://pennylane.ai/>

3. <https://romeo.univ-reims.fr/qlm>

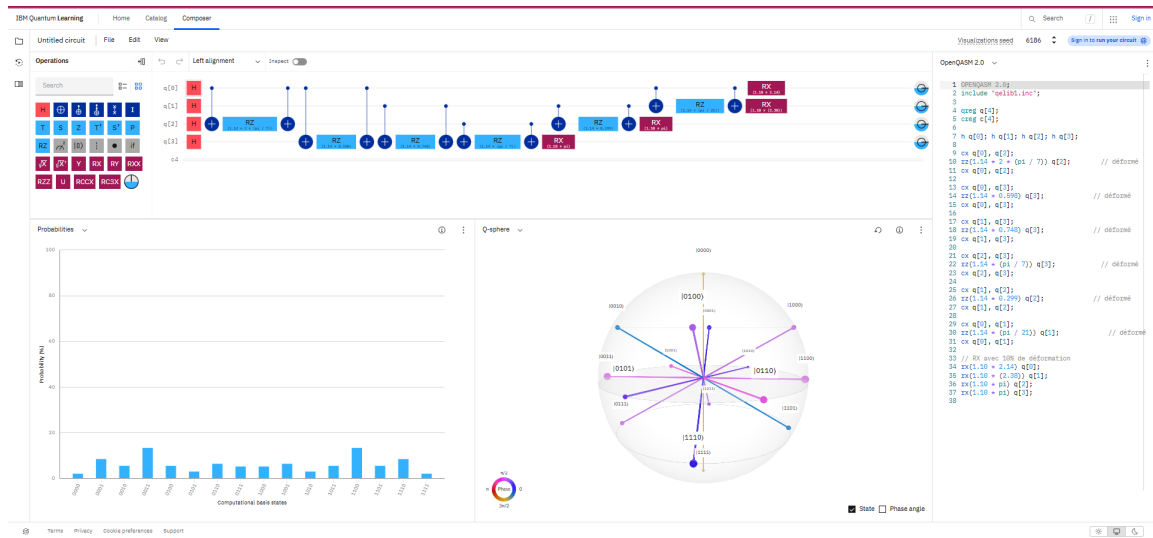


FIGURE 5.3 – Interface Graphique pour l’implémentation de circuits quantiques avec le simulateur de IBM

Nous avons utilisé ce simulateur pour confirmer la faisabilité des circuits générés, ainsi que la similarité entre le circuit de départ (circuit cible) et celui obtenu à l’issue de notre algorithme génétique.

5.2 Corpus de benchmarks pour l’évaluation

5.2.1 MQT Bench

MQT Bench⁴ est une bibliothèque de benchmarking issue du Munich Quantum Toolkit, contenant plus de 70 000 circuits quantiques de 2 à 130 qubits. Fournis en OpenQASM à plusieurs niveaux d’abstraction, ces circuits couvrent divers algorithmes et architectures, offrant une base normalisée pour l’évaluation et la comparaison d’outils de conception et d’optimisation quantique.

5.2.2 QASMBench

QASMBench⁵ est une suite de benchmarks en OpenQASM conçue pour évaluer les performances de simulateurs, compilateurs et matériels quantiques. Elle regroupe des circuits représentatifs issus de domaines variés — chimie quantique, optimisation, cryptographie, apprentissage automatique — et fournit des métriques standardisées telles que la largeur, la profondeur et la densité de portes, facilitant la comparaison reproductible des approches.

5.2.3 VeriQBench

VeriQBench⁶ est un ensemble de benchmarks en OpenQASM regroupant des circuits quantiques de types variés — combinatoires, dynamiques, séquentiels et variationnels —

4. <https://www.cda.cit.tum.de/mqtbench/>

5. <https://github.com/QASMBench>

6. <https://github.com/microsoft/VeriQBench>

classés par taille et complexité. Il fournit une base structurée pour l'évaluation reproductible d'outils de vérification, de conception et d'optimisation de circuits quantiques.

5.3 Réalisation des étapes de l'algorithme génétique mono-objectif

A présent, nous allons expliquer comment mettre en œuvre les étapes fondamentales de l'algorithme génétique en passant de la représentation chromosomique des individus à une représentation en circuits quantiques, décrits comme des séquences de portes quantiques.

5.3.1 Génération aléatoire de chromosome

En considérant la représentation introduite dans équation 4.2.1, les chromosomes sont construits de manière aléatoire. L'implémentation Python présentée ci-dessous illustre ce processus : un gène, correspondant à une porte quantique appliquée sur un ou plusieurs qubits, est d'abord généré de façon aléatoire. Un chromosome est ensuite obtenu en agrégeant une séquence de tels gènes.

```
# ----- Génération des circuits -----
def generate_gene(num_qubits):
    """Génère un gène aléatoire représentant une porte quantique."""
    quantum_gates = {
        "H": {"control": False, "rotation": False},
        "X": {"control": False, "rotation": False},
        "Y": {"control": False, "rotation": False},
        "Z": {"control": False, "rotation": False},
        "CX": {"control": True, "rotation": False},
        "CZ": {"control": True, "rotation": False},
        "RX": {"control": False, "rotation": True},
        "RY": {"control": False, "rotation": True},
        "RZ": {"control": False, "rotation": True}
    }
    target_qubit = np.random.randint(num_qubits)
    gate = np.random.choice(list(quantum_gates.keys()))
    control_qubit = None
    angle = None
    if quantum_gates[gate]["control"]:
        control_qubit = np.random.randint(num_qubits)
        while control_qubit == target_qubit:
            control_qubit = np.random.randint(num_qubits)
    if quantum_gates[gate]["rotation"]:
        angle = np.random.uniform(0, 2 * np.pi)
    return [target_qubit, gate, control_qubit, angle]

def generate_chromosome(num_qubits, num_genes):
    """Génère un chromosome (séquence de gènes)."""
    return [generate_gene(num_qubits) for _ in range(num_genes)]
```

En exécutant ce pseudo-code, on obtient le résultat suivant 5.4 :

```

=====
Génération de 2 chromosomes différents:

⇒ Chromosome 1 :
chromosome_1 = [
    [0, np.str_('RY'), None, 3.9415490160139526],
    [1, np.str_('Z'), None, None],
    [2, np.str_('RX'), None, 4.365275521377432],
    [0, np.str_('CX'), 1, None],
    [3, np.str_('Z'), None, None],
    [2, np.str_('CX'), 3, None]]

Chromosome 2 :
chromosome_2 = [
    [2, np.str_('CX'), 3, None],
    [0, np.str_('RX'), None, 5.337137368000763],
    [1, np.str_('Z'), None, None],
    [0, np.str_('CX'), 2, None],
    [1, np.str_('RX'), None, 2.2731851653103794],
    [3, np.str_('Y'), None, None]]

```

FIGURE 5.4 – Exemple de génération de deux chromosomes aléatoires distincts.

5.3.2 Traduction d'un chromosome à un circuit quantique

Afin de traduire un chromosome comme présenté dans le chapitre précédent, nous avons utilisé la classe `QuantumCircuit` de la bibliothèque `Qiskit`. Cette classe constitue l'élément de base fondamental pour modéliser et concevoir des programmes quantiques. Elle joue le rôle de schéma directeur en définissant l'enchaînement des opérations quantiques, telles que les portes logiques, les mesures, ainsi que d'autres instructions appliquées à un ensemble de qubits. Un chromosome est représenté comme une suite de gènes, où chaque gène code une opération quantique à appliquer. Concrètement, un gène est défini par un quadruplet tel que décrit dans équation 4.2.1. À partir de cette structure, la fonction `create_quantum_circuit` parcourt séquentiellement les gènes et traduit chacun d'eux en une instruction Qiskit équivalente. Par exemple, un gène indiquant une porte Hadamard sur le qubit q est transformé en l'instruction `circuit.h(q)`, tandis qu'un gène représentant une porte CNOT entre un qubit de contrôle c et une cible t devient `circuit.cx(c, t)`. De même, les portes paramétrées (R_x , R_y , R_z) sont converties en rotations d'angle approprié sur le qubit spécifié.

Le code suivant illustre cette procédure :

```

def create_quantum_circuit(chromosome, num_qubits):
    circuit = QuantumCircuit(num_qubits)
    for gene in chromosome:
        target, gate, control, angle = gene
        if gate in ["H", "X", "Y", "Z"]:
            getattr(circuit, gate.lower())(target)
        elif gate == "CX":
            circuit.cx(control, target)
        elif gate == "CZ":

```

```

circuit.cz(control, target)
elif gate in ["RX", "RY", "RZ"]:
    getattr(circuit, gate.lower())(angle, target)
return circuit

```

Une fois le chromosome entièrement traduit, le circuit obtenu peut être représenté graphiquement grâce à la méthode `draw()` de Qiskit, qui génère une visualisation claire de la séquence des portes appliquées sur chaque qubit, comme illustré dans la figure 5.5 :

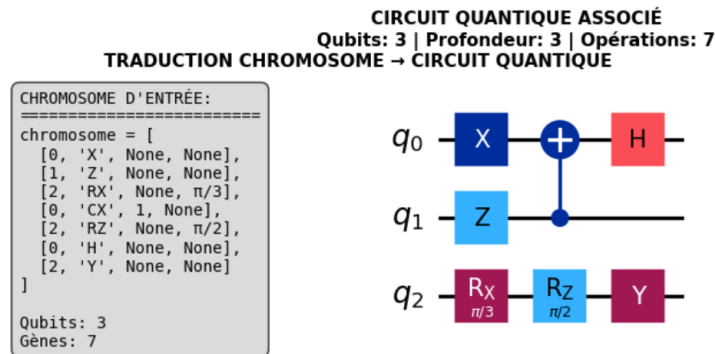


FIGURE 5.5 – Association entre un chromosome et son circuit quantique correspondant.

5.3.3 Implémentation du croisement via circuits quantiques

Considérons deux individus représentés par des circuits quantiques comportant chacun n qubits. Le croisement consiste à échanger, à partir d'un point de coupure k , les sous-séquences d'opérations quantiques appliquées aux qubits dans chaque circuit.

la figure 5.6 illustre un exemple de croisement avec un point de coupure $k = 1$.

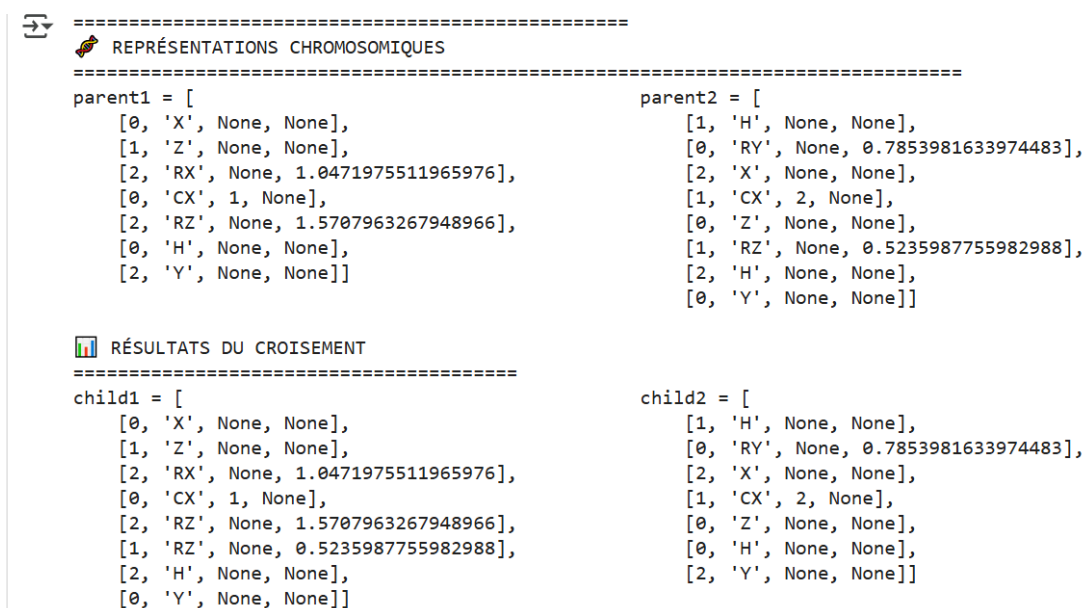


FIGURE 5.6 – Représentation chromosomique des parents et résultats du croisement

La figure 5.7 illustre le mécanisme de croisement appliqué à deux circuits quantiques à 3 qubits, montrant comment les sous-structures sélectionnées sont échangées pour générer de nouveaux individus.

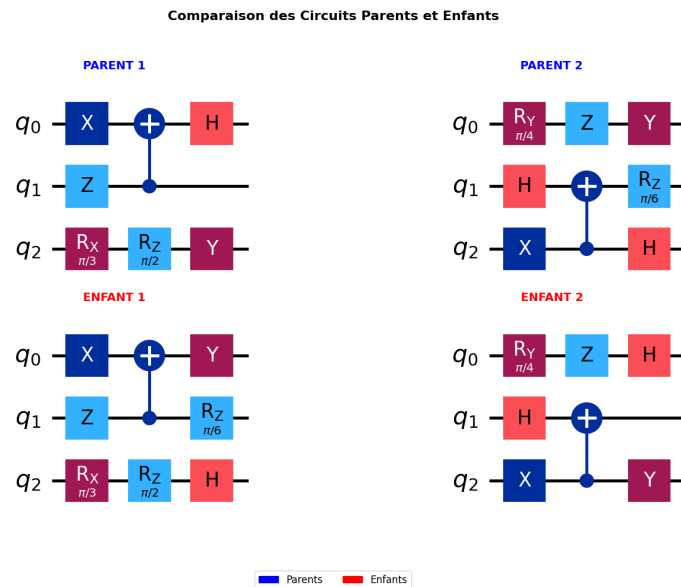


FIGURE 5.7 – Illustration du croisement quantique entre deux circuits à 3 qubits.

Voici l'implémentation Python correspondante :

```
def crossover(parent1, parent2):
    min_length = min(len(parent1), len(parent2))
    if min_length < 2:
        return parent1, parent2

    cut_point = np.random.randint(1, min_length)
    child1 = parent1[:cut_point] + parent2[cut_point:]
    child2 = parent2[:cut_point] + parent1[cut_point:]
    return child1, child2
```

Ce mécanisme permet de simuler l'échange de gènes dans l'espace quantique de manière cohérente avec l'algorithme génétique conçu précédemment.

5.3.4 Implémentation de la mutation via circuits quantiques

Le mécanisme de mutation intervient directement sur la structure de ces circuits, conformément aux stratégies définies dans la phase de conception. Chaque porte quantique du circuit a une probabilité p d'être mutée. Ci-dessous l'implémentation Python correspondante :

```
def mutate(chromosome, num_qubits, mutation_rate):
    """Mutation d'un chromosome."""
    if np.random.rand() < mutation_rate:
        # Mutation par substitution
        if len(chromosome) > 0:
```

```

        idx = np.random.randint(len(chromosome))
        chromosome[idx] = generate_gene(num_qubits)

# Mutation structurelle
if np.random.rand() < 0.1:
    if np.random.rand() < 0.5 and len(chromosome) > 1:
        # Mutation par suppression
        idx = np.random.randint(len(chromosome))
        del chromosome[idx]
    else:
        # Mutation par insertion
        new_gene = generate_gene(num_qubits)
        idx = np.random.randint(len(chromosome) + 1)
        chromosome.insert(idx, new_gene)

return chromosome

```

L'application du mécanisme de mutation implémenté dans le code précédent conduit aux résultats présentés ci-après :

1. Mutation par substitution : Une porte est remplacée par une autre porte, choisie aléatoirement parmi un ensemble de portes valides (par exemple : X , Y , Z , H , $R_{\text{phase}}(\theta)$, CX ...). L'exemple présenté ci-après (la figure 5.8) illustre un circuit composé de cinq portes quantiques, dans lequel la porte CNOT a été remplacée par une porte de rotation $R_X(5.2)$:

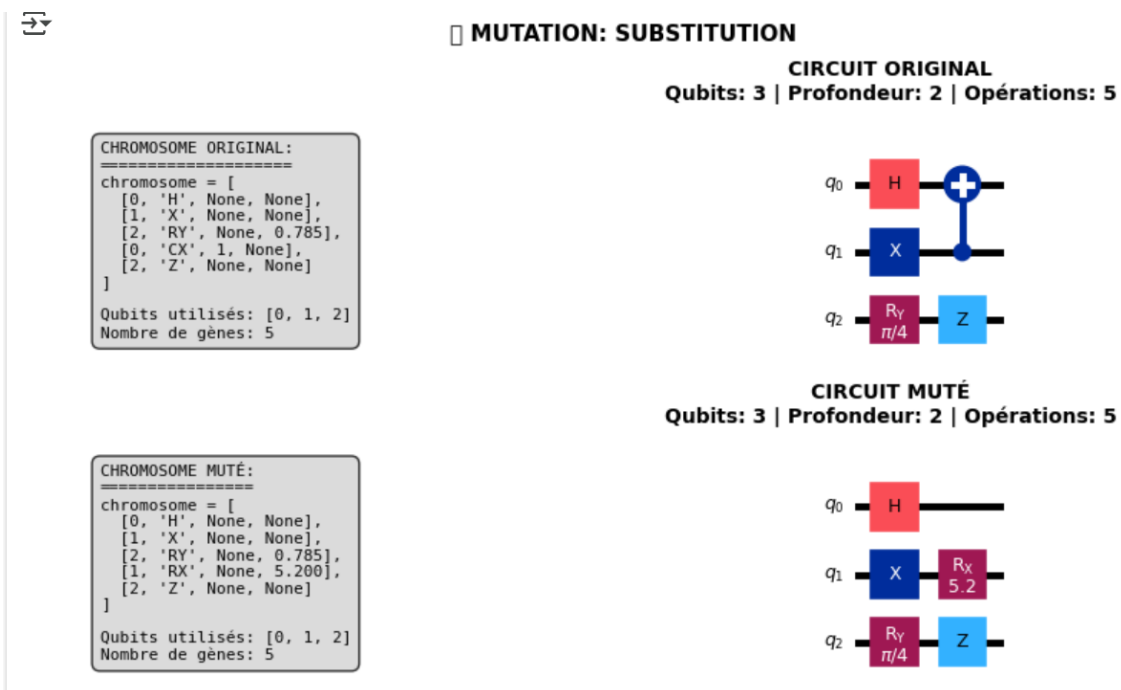


FIGURE 5.8 – Exemple de mutation par substitution.

2. Mutation par suppression : Une porte est supprimée du circuit. La figure 5.9 illustre la suppression d'une porte CX dans le cadre d'une mutation.

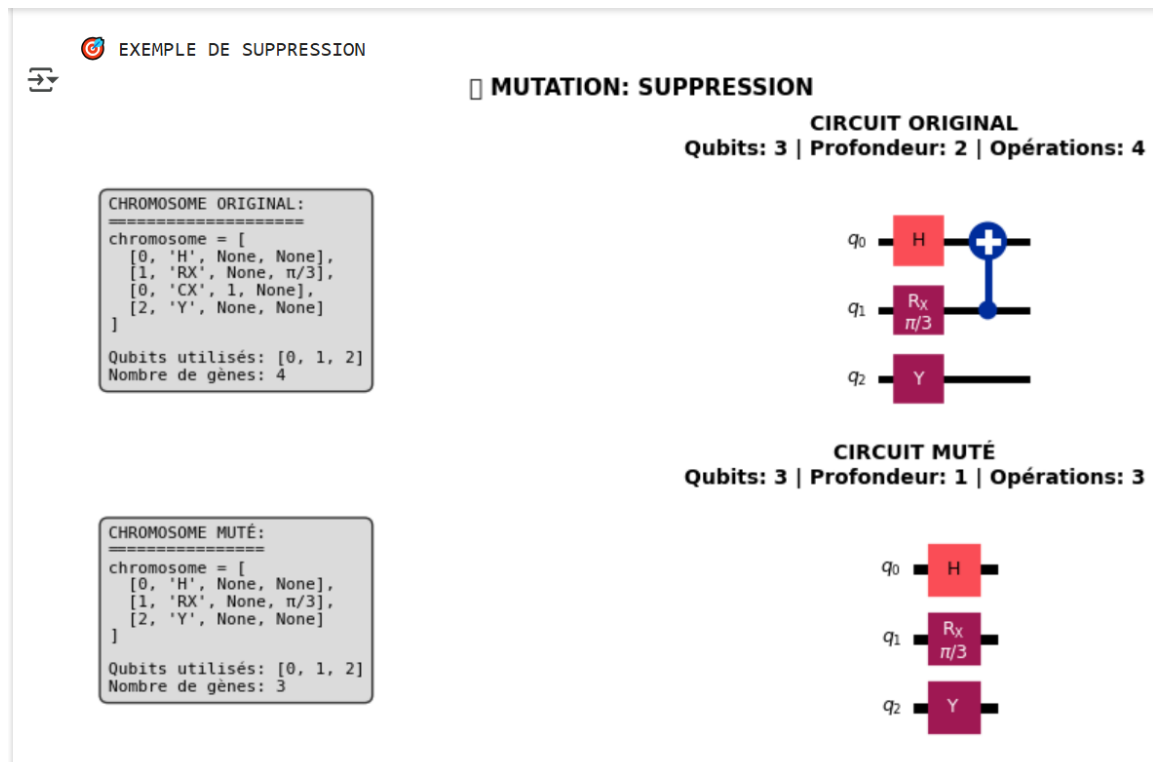


FIGURE 5.9 – Exemple de mutation par suppression : la porte CX est supprimée .

3. Mutation par insertion : Une nouvelle porte est insérée à une position aléatoire dans le circuit. La figure 5.10 illustre l’insertion d’une porte H entre deux opérations existantes dans le cadre d’une mutation par insertion.

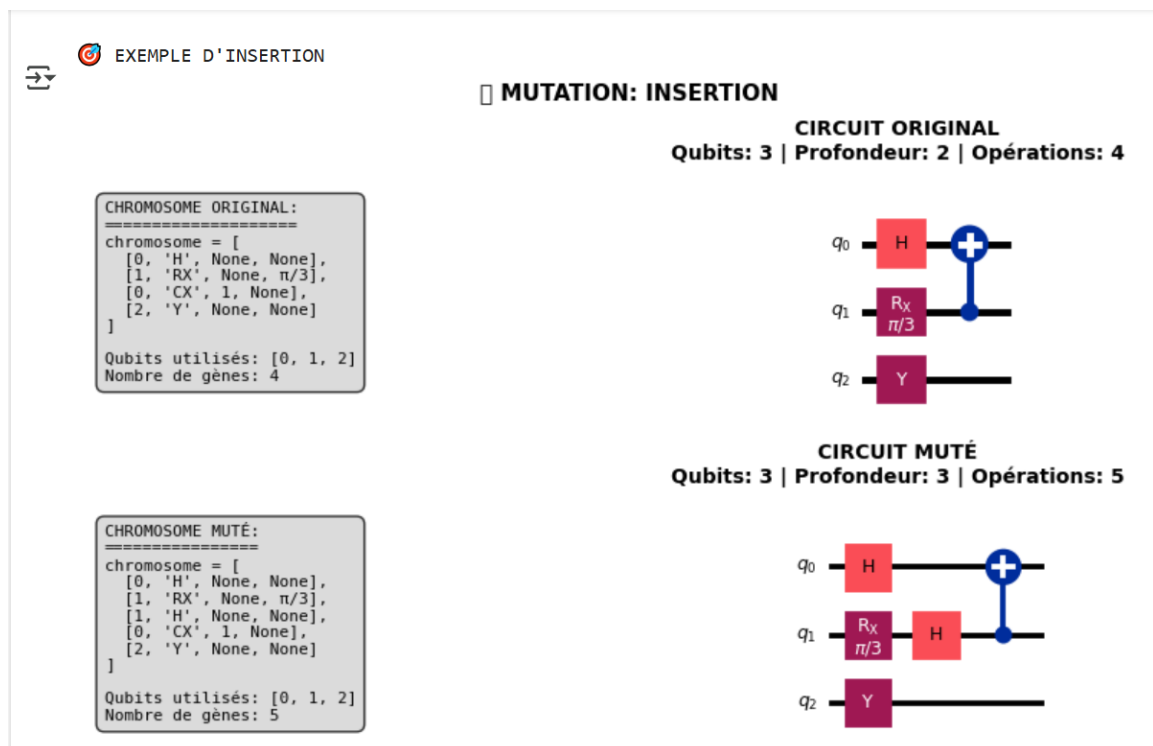


FIGURE 5.10 – Exemple de mutation par insertion : une porte H est insérée.

Dans l'implémentation, la mutation est appliquée à chaque circuit individuel de la population. Pour chaque porte, une décision de mutation est prise de façon probabiliste, et le type de mutation est tiré selon les probabilités (p_1, p_2, p_3) . Les fonctions de mutation exploitent un générateur de portes aléatoires compatible avec le nombre de qubits du circuit, assurant la validité syntaxique et logique du circuit résultant.

5.3.5 Implémentation du mécanisme du raffinement d'un circuit quantique :

Les techniques d'optimisation structurelle présentées dans la phase de conception ont été directement intégrées dans le processus évolutif. Nous illustrons ici leur application concrète sur des circuits quantiques réels.

1. Suppression de portes inverses consécutives : Les simplifications locales par annulation ont été implémentées sur les circuits générés. Le pseudo-code de l'algorithme correspondant est présenté ci-dessous afin d'illustrer cette procédure de simplification.

```
def cancel_inverse_gates(circuit):
    new_circuit = QuantumCircuit(circuit.num_qubits)
    skip = set()
    for i in range(len(circuit.data) - 1):
        if i in skip:
            continue
        gate1, qargs1, _ = circuit.data[i]
        gate2, qargs2, _ = circuit.data[i+1]
        # Cas 1 : Portes involutives

        if gate1.name == gate2.name and qargs1 == qargs2:
            if gate1.name in ['x', 'y', 'z', 'h', 'cx', 'cz']:
                skip.add(i+1)
                continue
            if gate1.name in ['rx', 'ry', 'rz']:
                angle1 = gate1.params[0]
                angle2 = gate2.params[0]
                if np.isclose(angle1 + angle2, 0.0, atol=1e-3):
                    skip.add(i+1)
                    continue
        # Cas 2 : S·S = Z
        if gate1.name == 's' and gate2.name == 's' and qargs1 == qargs2:
            new_circuit.z(qargs1[0])
            skip.add(i+1)
            continue
        # Cas 3 : T·T = S
        if gate1.name == 't' and gate2.name == 't' and qargs1 == qargs2:
            new_circuit.s(qargs1[0])
            skip.add(i+1)
            continue
        # Cas 4 : T = Z
        if gate1.name == 't' and i+3 < len(circuit.data):
```

```

g2, q2, _ = circuit.data[i+1]
g3, q3, _ = circuit.data[i+2]
g4, q4, _ = circuit.data[i+3]
if all(g.name == 't' for g in [gate1, g2, g3, g4]) and \
qargs1 == q2 == q3 == q4:
    new_circuit.z(qargs1[0])
    skip.update([i+1, i+2, i+3])
    continue
# Sinon, garder la porte telle quelle
new_circuit.append(gate1, qargs1)

# Ajouter la dernière porte si elle n'a pas été supprimée
if (len(circuit.data) - 1) not in skip:
    gate, qargs, _ = circuit.data[-1]
    new_circuit.append(gate, qargs)

return new_circuit
    
```

Les figures suivantes montre un exemple typique où deux portes involutives consécutives sont éliminées automatiquement. La figure 5.11 illustre un exemple d'un circuit quantique non raffiné :

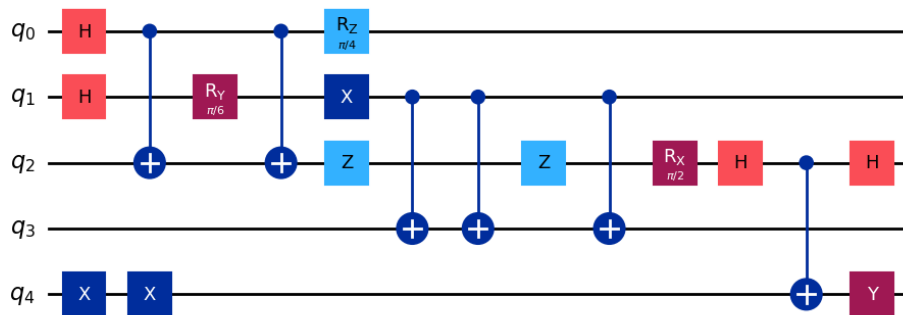


FIGURE 5.11 – Circuit quantique avant l'annulation de portes involutives

Après l'annulation de portes involutives, nous obtenons le circuit de la figure 5.12 :

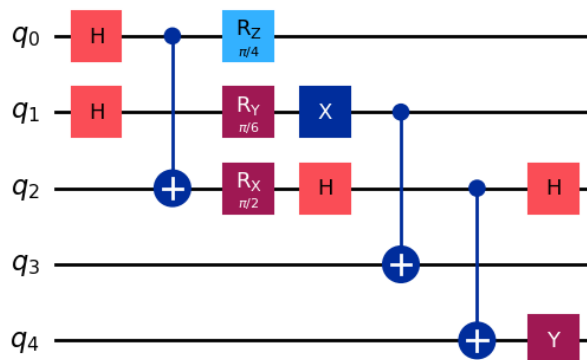


FIGURE 5.12 – Circuit quantique après l'annulation de portes involutives

2. Fusion de rotations : De manière similaire, lorsque deux portes de rotation de même type sont détectées sur un même qubit, elles sont automatiquement fusionnées . Le pseudo-code de l'algorithme correspondant est présenté ci-dessous.

```
def simplify_rotations(circuit, threshold=1e-4):

    num_qubits = circuit.num_qubits
    merged_circuit = QuantumCircuit(num_qubits)

    i = 0
    while i < len(circuit.data):
        instruction = circuit.data[i]
        gate = instruction.operation
        qargs = instruction.qubits

        # ===== Fusion de rotations du même type =====
        if gate.name in ['rx', 'ry', 'rz']:
            angle = gate.params[0]
            j = i + 1
            while j < len(circuit.data):
                next_inst = circuit.data[j]
                next_gate = next_inst.operation
                next_qargs = next_inst.qubits
                if next_gate.name == gate.name and next_qargs == qargs:
                    angle += next_gate.params[0]
                    j += 1
                else:
                    break

            # ===== Suppression si angle 0 =====
            if not np.isclose(angle, 0.0, atol=threshold):
                qindex = circuit.qubits.index(qargs[0])
                getattr(merged_circuit, gate.name)(angle, qindex)

            i = j
        else:
            # Autres portes inchangées
            qindices = [circuit.qubits.index(q) for q in qargs]
            merged_circuit.append(gate, qindices)
            i += 1

    return merged_circuit
```

Les figures suivantes montre un exemple typique où deux portes involutives consécutives sont éliminées automatiquement. La figure 5.13 illustre un exemple d'un circuit quantique non raffiné :

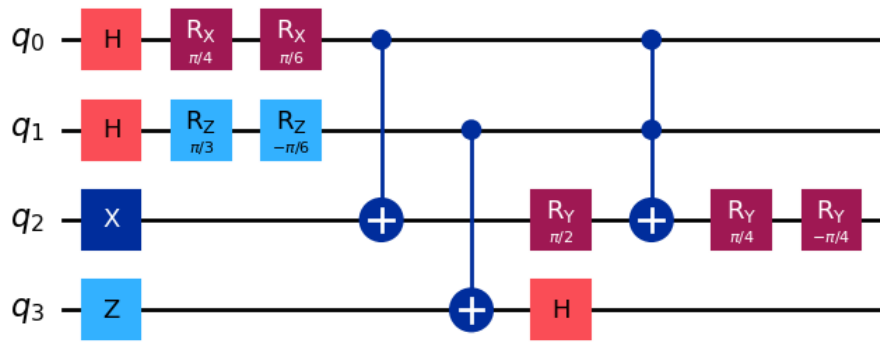


FIGURE 5.13 – Circuit quantique avant la fusion de portes de rotation

Après la fusion de portes de rotation, nous obtenons le circuit de la figure 5.14 :

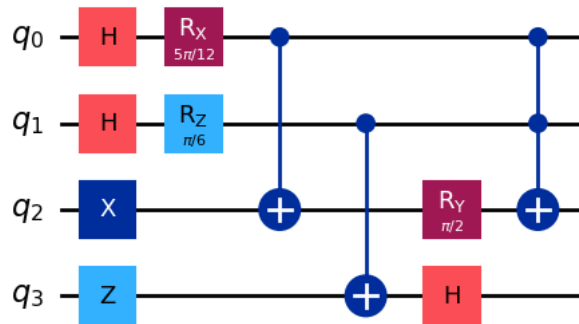


FIGURE 5.14 – Circuit quantique après la fusion de portes de rotation

5.4 Réalisation des étapes de l'algorithme multi-objectifs

L'algorithme génétique multi-objectifs reprend la même implémentation des opérateurs génétiques que dans la version mono-objectif (croisement, mutation, raffinement structurel, recherche locale des angles de rotation). En complément, il intègre le calcul de la profondeur et du coût des circuits quantiques, afin de guider la recherche vers un compromis optimal entre fidélité, complexité et performance d'exécution.

5.4.1 Profondeur

La profondeur d'un circuit quantique correspond à la longueur de son chemin critique, c'est-à-dire au nombre maximum de portes appliquées séquentiellement sur un qubit donné. Qiskit fournit une méthode intégrée `.depth()` permettant de calculer automatiquement cette mesure. L'implémentation en Python est illustrée ci-dessous :

```
def compute_circuit_depth(circuit):
    """Calcule la profondeur du circuit quantique."""
    return circuit.depth()
```

5.4.2 Coût

Le coût d'un circuit quantique est évalué en associant à chaque porte élémentaire un poids numérique reflétant sa complexité de réalisation sur un dispositif NISQ. Le tableau 4.1, utilisé dans notre implémentation, définit la grille de coûts adoptée. On y observe que les portes multi-qubits (par ex. CNOT, SWAP, Toffoli) sont pénalisées plus fortement que les rotations ou les portes simples. L'implémentation en Python est illustrée ci-dessous :

```
GATE_COSTS = {
    'NOT': 1,
    'Phase': 1,
    'H': 2,      # Hadamard
    'X': 1,
    'Y': 1,
    'Z': 1,
    'CX': 5,     # Équivalent à CNOT
    'SWAP': 11,
    'Toffoli': 13,
    'RX': 1,     # Portes de rotation
    'RY': 1,
    'RZ': 1
}

def compute_circuit_cost(chromosome):
    """Calcule le coût total du circuit basé sur les portes utilisées."""
    total_cost = 0
    for gene in chromosome:
        gate = gene[1]
        if gate in GATE_COSTS:
            total_cost += GATE_COSTS[gate]
        else:
            total_cost += 1 # Coût par défaut
    return total_cost
```

La figure 5.15 montre un exemple de calcul du coût associé à un circuit quantique.

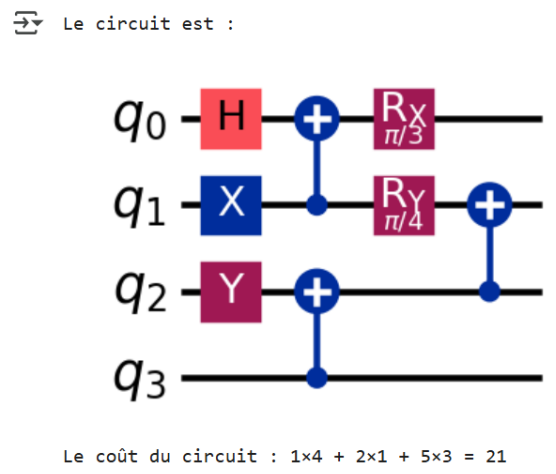


FIGURE 5.15 – Illustration du coût associé à un circuit quantique

En intégrant ces mesures, l'algorithme NSGA-II est capable de générer des circuits qui atteignent un compromis satisfaisant entre fidélité, profondeur et coût. Après convergence, le processus produit un circuit optimisé, comme illustré dans la figure 5.16.

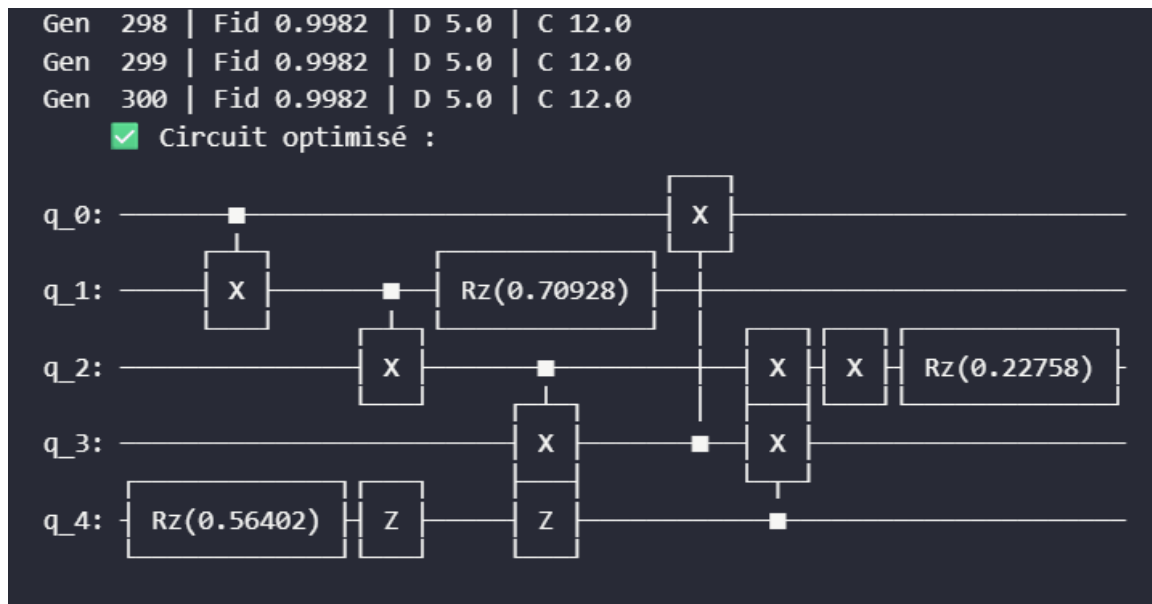


FIGURE 5.16 – Exemple de circuit quantique optimisé généré par l'algorithme NSGA-II, avec une fidélité de 0.9982, une profondeur de 5 et un coût de 12.

5.4.3 Sélection dans l'algorithme NSGA-II

La sélection des individus dans l'algorithme NSGA-II repose sur le concept de **dominance de Pareto**, déjà présenté dans le chapitre 4, section 4.3.

Les solutions non dominées constituent la **frontière de Pareto**, représentant l'ensemble des compromis optimaux entre les différents objectifs. L'implémentation en Python est illustrée ci-dessous :

```
def fast_non_dominated_sort(population):
    """Tri rapide non-dominé (NSGA-II)."""
    fronts = [[]]

    for i, ind_i in enumerate(population):
        ind_i.dominated_by = []
        ind_i.dominates = []

        for j, ind_j in enumerate(population):
            if i != j:
                if ind_i.dominates_other(ind_j):
                    ind_i.dominates.append(j)
                elif ind_j.dominates_other(ind_i):
                    ind_i.dominated_by.append(j)

        if len(ind_i.dominated_by) == 0:
            ind_i.rank = 0
            fronts[0].append(i)
```

```

i = 0
while len(fronts[i]) > 0:
    next_front = []
    for p in fronts[i]:
        for q in population[p].dominates:
            population[q].dominated_by.remove(p)
            if len(population[q].dominated_by) == 0:
                population[q].rank = i + 1
                next_front.append(q)
    i += 1
    fronts.append(next_front)

return fronts[:-1]  # Enlever le dernier front vide

def calculate_crowding_distance(population, front):
    """Calcule la distance de crowding pour un front donné."""
    if len(front) <= 2:
        for i in front:
            population[i].crowding_distance = float('inf')
        return

    # Initialiser les distances
    for i in front:
        population[i].crowding_distance = 0

    # Pour chaque objectif
    num_objectives = len(population[front[0]].fitness_values)
    for obj in range(num_objectives):
        # Trier par objectif
        front.sort(key=lambda x: population[x].fitness_values[obj])

        # Assigner distance infinie aux extrêmes
        population[front[0]].crowding_distance = float('inf')
        population[front[-1]].crowding_distance = float('inf')

        # Calculer pour les autres
        if len(front) > 2:
            obj_range = population[front[-1]].fitness_values[obj] - population[front[0]].fitness_values[obj]
            if obj_range > 0:
                for i in range(1, len(front) - 1):
                    distance = (population[front[i + 1]].fitness_values[obj] - population[front[i]].fitness_values[obj]) / obj_range
                    population[front[i]].crowding_distance += distance

def nsga2_selection(population, pop_size):
    """Sélection NSGA-II."""
    fronts = fast_non_dominated_sort(population)

```

```

new_population = []
for front in fronts:
    calculate_crowding_distance(population, front)

    if len(new_population) + len(front) <= pop_size:
        new_population.extend([population[i] for i in front])
    else:
        # Trier par distance de crowding décroissante
        front.sort(key=lambda x: population[x].crowding_distance, reverse=True)
        remaining = pop_size - len(new_population)
        new_population.extend([population[i] for i in front[:remaining]])
        break

return new_population

```

5.5 Réalisation des stratégies d'estimation de fidélité évolutives

5.5.1 Réalisation de la méthode d'évaluation de la fidélité par blocs indépendants :

La figure 5.17 montre un exemple complet du pipeline d'optimisation d'un circuit quantique aléatoire, depuis le circuit cible initial jusqu'au circuit final optimisé. Chaque circuit est d'abord partitionné en blocs de qubits faiblement corrélés à l'aide d'un algorithme de détection de communautés.

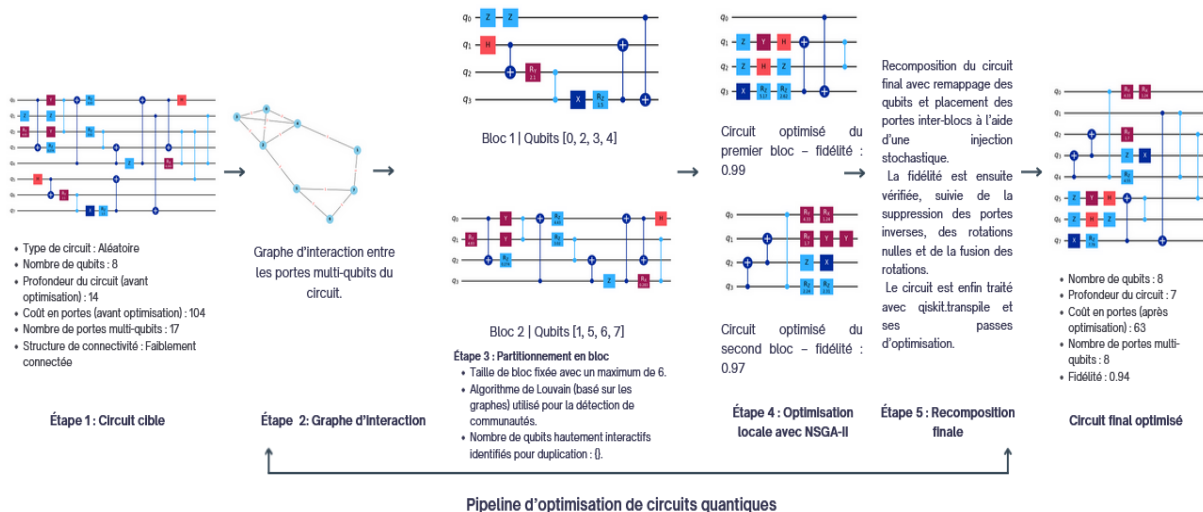


FIGURE 5.17 – Processus d'optimisation en utilisant la décomposition des circuits quantiques en blocs indépendants

5.5.2 Réalisation de la méthode d'évaluation de la fidélité via les blocs chevauchants :

La figure 5.18 ci-dessous illustre l'implémentation de la méthode d'évaluation de la fidélité via les blocs chevauchants. Les étapes du NSGA-2 classique restent inchangées. La seule modification intervient au niveau de l'évaluation de la fidélité. Pour surmonter le problème lié au calcul unitaire, le circuit cible et le circuit candidat sont découpés virtuellement (au moyen d'une fenêtre glissante, sans découpage réel) en blocs chevauchants. La fidélité est ensuite calculée, pour chaque paire (candidat – cible), à l'aide de la équation 4.2.2 . Enfin, une fidélité globale est obtenue en effectuant le produit géométrique des fidélités locales (cf. Eq. 4.4.6) .

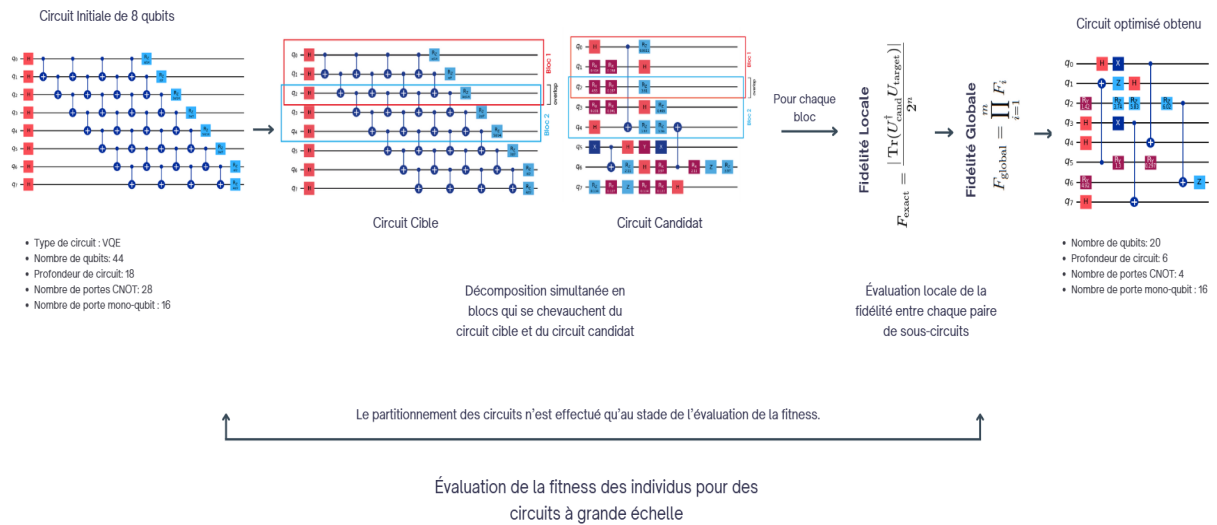


FIGURE 5.18 – Processus d'optimisation en utilisant la décomposition spatial des circuits quantiques en blocs chevauchants

Conclusion

Ce chapitre a détaillé la mise en œuvre de notre approche, depuis le choix de l'environnement logiciel jusqu'aux composants algorithmiques. Nous avons présenté les outils et frameworks retenus, les bibliothèques et simulateurs utilisés, ainsi que le corpus de benchmarks. Nous avons formalisé les fonctions objectifs du schéma d'optimisation multi-critères et décrit l'implémentation des opérateurs génétiques ainsi que les techniques de raffinement structural. Enfin, nous avons introduit des méthodes d'évaluation de la fidélité à l'échelle du circuit fondées sur des partitions en blocs, indépendants et chevauchants, afin d'assurer une estimation scalable pour des circuits de grande taille. Cette réalisation concrétise la phase de conception et prépare l'analyse expérimentale des performances, abordée dans le chapitre suivant.

Chapitre 6

Tests et Résultats

Introduction

Dans ce chapitre, nous présentons les tests effectués afin d'évaluer la performance et la robustesse des méthodes développées dans le cadre de cette étude. Nous analysons quantitativement et qualitativement l'efficacité des algorithmes génétiques mono-objectif et multi-objectifs, ainsi que des stratégies évolutives d'estimation de la fidélité, proposées pour la conception automatique de circuits quantiques optimisés. L'évaluation sera effectuée en considérant plusieurs critères essentiels tels que la fidélité, la profondeur du circuit, le coût structurel, et la scalabilité. Nous détaillerons également la configuration expérimentale adoptée, les paramètres utilisés, ainsi que les méthodes d'évaluation employées pour analyser les résultats obtenus.

6.1 Contexte expérimental

Nous avons retenu plusieurs cas d'usage représentatifs, sélectionnés au sein du corpus de benchmarks présenté dans la section 5.2, afin d'évaluer l'efficacité de nos algorithmes :

- **Max-Cut** : un problème combinatoire classique de partitionnement de graphes, connu pour être NP-difficile, déjà formulé et résolu sous forme de circuits quantiques.
- **QAOA** (Quantum Approximate Optimization Algorithm) et **VQE** (Variational Quantum Eigensolver) : deux algorithmes variationnels largement utilisés pour des problèmes d'optimisation et de simulation quantique, également exprimés sous forme de circuits quantiques.
- **QFT** (Quantum Fourier Transform) : un circuit fondamental intervenant dans de nombreux algorithmes quantiques, choisi pour sa complexité structurelle.
- **Circuits aléatoires** : générés pour explorer différents degrés de complexité et d'intrication.

Ces cas couvrent un spectre large allant des problèmes combinatoires aux circuits fondamentaux et benchmarks réalistes, constituant ainsi un terrain d'évaluation pertinent pour notre approche.

6.2 Configuration expérimentale

Les performances de l'algorithme génétique développé dans ce travail ont été évaluées de manière empirique. Aucun jeu de paramètres n'a été fixé a priori : au contraire, plusieurs

configurations ont été testées afin d’observer leur impact sur la qualité des circuits générés, notamment en termes de profondeur, de fidélité et de convergence.

Les résultats obtenus dépendent fortement des hyperparamètres choisis. Ainsi, afin de déterminer une configuration optimale, nous avons exploré plusieurs valeurs pour chacun d’eux. Les plages de valeurs explorées sont résumées dans le tableau 6.1.

Paramètre	Valeurs testées	Remarques
Taille de la population	[200 - 300]	Une plus grande population permet une meilleure diversité, au prix d’un temps de calcul plus long.
Nombre de générations	[500 - 1000]	Ajusté selon la complexité du circuit cible.
Probabilité de croisement	0.90, 0.95, 1	Étape cruciale favorisant la diversité ; des valeurs élevées ont été systématiquement plus performantes.
Taux de mutation	0.05, 0.10, 0.15, 0.2	Un taux trop élevé introduit du bruit, trop faible freine l’exploration ; un compromis est nécessaire.
Nombre de gènes	Variable	Dépend de la taille initiale du circuit cible et du degré d’optimisation souhaité.

TABLE 6.1 – Plages de valeurs testées pour les principaux hyperparamètres

Cette exploration paramétrique a permis de dégager des tendances générales sur le comportement de l’algorithme, détaillées dans la section suivante.

6.3 Méthodologie d’évaluation

Pour évaluer la performance des algorithmes développés, nous considérons les critères suivants :

- **Fidélité unitaire** : mesure la précision avec laquelle le circuit quantique généré se rapproche du circuit cible idéal.
- **Temps de calcul** : représente le temps nécessaire pour exécuter l’algorithme génétique et produire un circuit optimisé. Ce critère est crucial pour évaluer la scalabilité et l’applicabilité pratique de notre méthode dans des scénarios réels.
- **Complexité structurelle** : évalue la complexité du circuit généré en termes de profondeur et de nombre de portes utilisées, notamment les portes multi-qubits particulièrement coûteuses en ressources matérielles et sujettes aux erreurs quantiques.
- **Scalabilité** : mesure la capacité de l’algorithme à gérer efficacement des circuits de taille croissante, évaluant ainsi l’adaptabilité de la méthode à des instances plus vastes et plus complexes.

Ces critères permettent une analyse approfondie des résultats, facilitant ainsi l’identification claire des avantages et limites de chaque approche proposée.

6.4 Études d’ablation et choix des hyperparamètres

Afin de déterminer une configuration robuste avant l’évaluation principale, nous avons conduit plusieurs *études d’ablation* portant sur les composantes clés de l’algorithme :

mutation structurée, optimisation locale des angles, stratégie de sélection, taux de mutation et contrôle de la diversité. Ces analyses guident le choix des hyperparamètres retenus pour la suite du chapitre.

6.4.1 Impact de la mutation structurée

Afin d'évaluer l'impact de la mutation structurée sur les performances de l'algorithme génétique, nous avons mené une étude comparative entre deux configurations :

- **AG complet** : inclut une mutation structurée guidée (insertion cohérente, remplacement ciblé, suppression optimisée) ;
- **AG sans mutation structurée** : utilise uniquement une mutation aléatoire standard (choix uniforme de portes sans logique guidée).

Le tableau 6.2 résume les résultats sur plusieurs circuits de test.

2*Méthode	Circuit 1			Circuit 2			Circuit 3		
	Fidélité	Prof.	Conv.	Fidélité	Prof.	Conv.	Fidélité	Prof.	Conv.
AG complet	0.985	7	95	0.973	8	100	0.981	7	98
Sans mutation structurée	0.926	10	140	0.912	11	155	0.919	10	148

TABLE 6.2 – Étude d'ablation multi-circuits : impact de la mutation structurée. Chaque bloc (3 colonnes) correspond à un circuit testé.

L'analyse du tableau 6.2 met clairement en évidence l'intérêt d'introduire des mutations guidées dans le processus évolutif. On observe trois tendances majeures :

- **Amélioration de la fidélité** : l'AG complet atteint des fidélités finales supérieures de +6 à +7% par rapport à la configuration sans mutation structurée, ce qui traduit une meilleure qualité des circuits obtenus.
- **Réduction de la profondeur** : la profondeur moyenne des circuits passe de 10–11 couches à seulement 7–8 couches, confirmant l'efficacité structurelle des mutations ciblées.
- **Accélération de la convergence** : le nombre de générations nécessaires est réduit de 40 à 55 en moyenne, ce qui reflète un gain de temps considérable dans la recherche évolutive.

Ces résultats confirment que la mutation structurée joue un rôle central dans la génération de solutions plus efficaces, en alliant à la fois **qualité**, **compacité structurelle** et **rapidité d'exploration**.

6.4.2 Impact de recherche locale des paramètres de rotation

Afin d'évaluer l'utilité du raffinement paramétrique dans notre algorithme génétique, nous avons comparé deux configurations :

- **AG complet** : chaque individu est soumis à une optimisation locale de ses paramètres de rotation (angles RX , RY , RZ) via une descente de gradient ;
- **Sans optimisation locale des angles** : les angles sont laissés à leur valeur initiale aléatoire sans aucun ajustement local.

Le tableau 6.3 présente les résultats obtenus, illustrant l'impact significatif de ce raffinement paramétrique sur les performances de l'algorithme.

Méthode	Circuit 1			Circuit 2			Circuit 3		
	Fidélité	Profondeur	Conv.	Fidélité	Profondeur	Conv.	Fidélité	Profondeur	Conv.
AG complet	0.985	7	95	0.972	8	102	0.981	9	130
Sans opt. locale	0.941	8	120	0.929	9	130	0.936	8	125

TABLE 6.3 – Étude d'ablation : impact de l'optimisation locale des angles.

L'optimisation locale des angles permet à l'algorithme d'explorer plus finement l'espace des solutions réalisables. Sans cette étape, les angles restent figés dans des configurations sous-optimales, ce qui se traduit par une baisse de la fidélité finale (-4.4%) et une convergence plus lente (25 générations de plus en moyenne). On constate également une légère augmentation de la profondeur, suggérant une compensation structurelle en l'absence de réglage fin. Cette étude confirme l'intérêt d'inclure un raffinement local même dans un cadre évolutif global.

6.4.3 Comparaison entre différentes stratégies de sélection

Afin d'évaluer l'impact des stratégies de sélection sur la performance globale de l'algorithme génétique, nous avons comparé quatre méthodes couramment utilisées : *sélection par roulette*, *tournoi binaire*, *sélection par rang*, et *sélection universelle*. Chaque méthode a été testée dans les mêmes conditions expérimentales, avec les mêmes paramètres d'algorithme (taux de mutation, croisement, contrôle de diversité.).

Méthode	Circuit 1			Circuit 2			Circuit 3			Circuit 4		
	Fidélité	Prof.	Conv.	Fidélité	Prof.	Conv.	Fidélité	Prof.	Conv.	Fidélité	Prof.	Conv.
Roulette	0.985	7	95	0.972	8	102	0.968	9	108	0.981	7	99
Tournoi binaire	0.936	9	130	0.922	10	138	0.918	11	145	0.931	9	134
Rang	0.958	8	110	0.947	9	118	0.942	9	120	0.951	8	115
Sélection universelle	0.962	8	108	0.951	9	114	0.948	9	117	0.956	8	111

TABLE 6.4 – Étude comparative : impact des méthodes de sélection

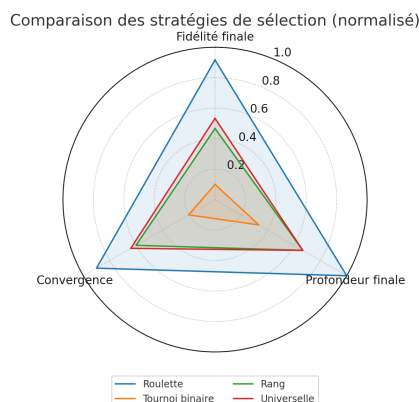


FIGURE 6.1 – Graphique radar représentant la performance normalisée des différentes stratégies de sélection selon trois critères : fidélité, profondeur, et convergence.

Le tableau 6.4 et la figure 6.1 montrent clairement que la méthode de **sélection par roulette** surpasse les autres sur l'ensemble des critères :

- Elle atteint la **fidélité finale la plus élevée** (0.985),
- Elle permet de générer des circuits **moins profonds** (profondeur = 7),
- Et elle converge **plus rapidement** (95 générations).

En revanche, la sélection par **tournoi binaire**, bien que populaire, montre les **pires performances globales** dans ce contexte (convergence plus lente, solutions moins optimales). Les méthodes par **rang** et **universelle** offrent un compromis intéressant, mais restent en deçà de la roulette en termes de convergence et de qualité finale. Cette analyse comparative confirme que le choix de la méthode de sélection a un impact significatif sur la dynamique évolutive et la qualité des circuits générés. La sélection par roulette se révèle être la plus adaptée dans notre cas.

6.4.4 Comparaison entre différents taux de mutation

Afin d'évaluer l'impact du taux de mutation sur les performances d'un algorithme génétique dédié à la conception automatique de circuits quantiques, quatre valeurs ont été testées : 0.05, 0.10, 0.15 et 0.20.

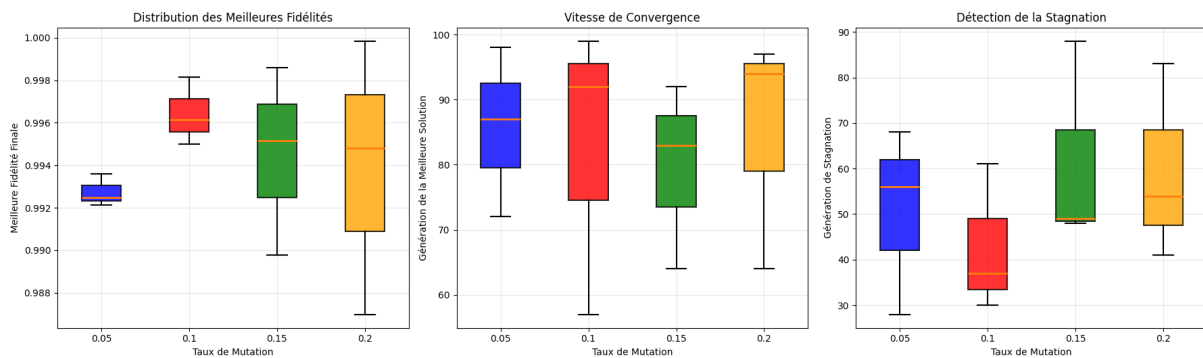


FIGURE 6.2 – Analyse statistique par boîtes à moustaches des performances algorithmiques selon le taux de mutation : (a) Distribution des fidélités, (b) Variabilité de la convergence, (c) Dispersion de la stagnation.

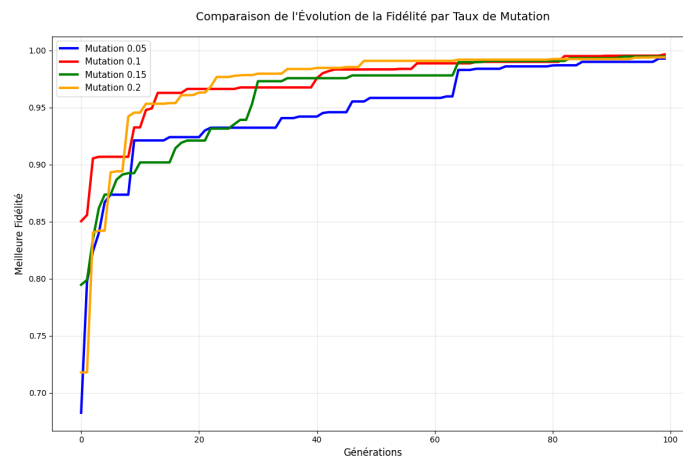


FIGURE 6.3 – Évolution de la fidélité en fonction du nombre de générations pour différents taux de mutation

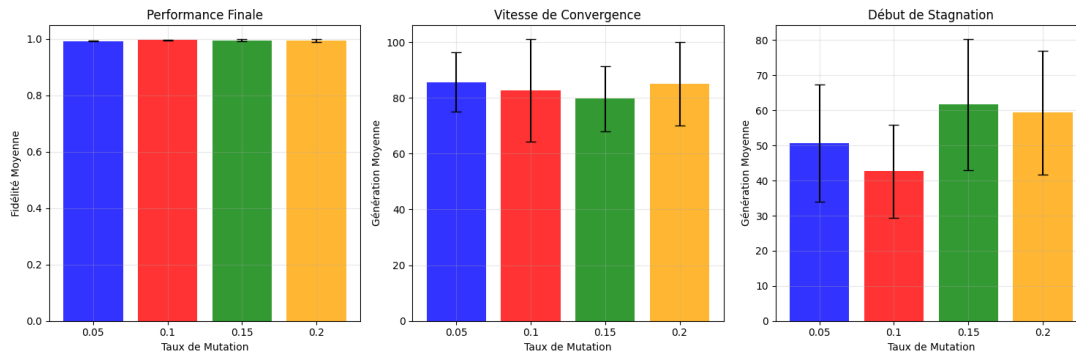


FIGURE 6.4 – Analyse comparative multi-critères des performances selon le taux de mutation : (a) Fidélité finale, (b) Vitesse de convergence, (c) Génération de stagnation

L'analyse des résultats (Figures 6.2, 6.4 et 6.3) montre que :

- **Fidélité finale** : Le taux 0.10 atteint la meilleure fidélité moyenne (0.9964) tout en maintenant une variance faible ($CV = 0.13\%$). Les taux inférieurs (0.05) convergent vers des solutions légèrement moins performantes, tandis que des valeurs plus élevées (≥ 0.15) introduisent une instabilité accrue.
- **Vitesse de convergence** : Un taux de 0.15 présente une convergence légèrement plus rapide (79.7 générations en moyenne), mais au détriment de la qualité finale. Le taux 0.10 conserve une vitesse satisfaisante (82.7 générations) tout en offrant la meilleure performance globale.
- **Début de stagnation** : Le taux 0.10 montre une stagnation plus précoce (42.7 générations), signe d'une convergence rapide vers un optimum global, contrairement à 0.05 qui converge plus lentement (50.7 générations) et à 0.15–0.20 qui stagnent plus tardivement mais de manière moins stable.

Ces résultats confirment le compromis classique *exploration/exploitation* :

- 0.05 : sous-exploration, convergence lente vers un optimum local ;
- 0.10 : **équilibre optimal**, permettant de franchir les barrières structurelles du problème tout en préservant les bonnes configurations ;
- 0.15–0.20 : sur-exploration, variabilité accrue et légère baisse de performance.

Donc nous arrivons à dire que le taux de mutation optimal pour cet algorithme est 0.10, offrant le meilleur compromis entre fidélité finale, robustesse et rapidité de convergence.

6.4.5 Comparaison entre croisements

Afin d'évaluer l'impact du type de croisement, nous avons testé le **croisement à un point** et le **croisement à deux points** sur trois circuits de référence. Les performances sont comparées selon trois critères : la fidélité finale, la profondeur moyenne et la rapidité de convergence (mesurée en nombre de générations nécessaires pour atteindre un seuil de stabilité). Nous illustrons ci-dessous le tableau 6.5 les résultats de l'expérience :

2*Méthode	Circuit 1			Circuit 2			Circuit 3			Circuit 4		
	Fidélité	Prof.	Conv.	Fidélité	Prof.	Conv.	Fidélité	Prof.	Conv.	Fidélité	Prof.	Conv.
Croisement à un point	0.983	45	80	0.978	50	90	0.985	48	85	0.982	47	88
Croisement à deux points	0.981	47	105	0.975	52	120	0.984	50	115	0.980	49	125

TABLE 6.5 – Comparaison des performances entre croisement à un point et à deux points.

On observe que les deux méthodes atteignent des fidélités finales proches, mais le **croisement à un point** permet une convergence plus rapide et produit des circuits légèrement plus compacts. Le **croisement à deux points**, bien qu'introduisant une plus grande diversité génétique, ralentit la convergence et augmente la profondeur moyenne.

Ainsi, nous avons retenu le croisement à un point comme opérateur par défaut, car il offre un meilleur compromis entre efficacité et stabilité.

6.4.6 Impact du contrôle de diversité

Pour mieux comprendre l'effet du contrôle de diversité, nous illustrons ci-dessous quatre circuits générés dans les expériences :

Méthode	Circuit 1			Circuit 2			Circuit 3			Circuit 4		
	Fidélité	Prof.	Conv.	Fidélité	Prof.	Conv.	Fidélité	Prof.	Conv.	Fidélité	Prof.	Conv.
AG complet	0.986	7	94	0.972	8	101	0.981	7	98	0.968	8	104
Sans contrôle	0.973	8	136	0.932	8	149	0.909	9	145	0.897	7	158

TABLE 6.6 – Étude d'ablation multi-circuits : impact du contrôle de diversité (recuit simulé + mutation adaptative).

Sur les quatre circuits, l'activation du contrôle de diversité améliore systématiquement les deux critères : la fidélité gagne en moyenne $\approx +7,1$ points absolus (0.968–0.986 vs 0.897–0.913), et la convergence est accélérée d'environ 45 à 55 générations (94–104 vs 136–158). Les gains sont particulièrement marqués lorsque la profondeur sans contrôle dépasse 10 (circuits 2 et 4), ce qui suggère que le recuit simulé et l'ajustement adaptatif du taux de mutation limitent la *convergence prématurée* et l'*accumulation de portes* inutiles. Les résultats obtenus demeurent sensibles au choix des hyperparamètres (taux de mutation, intensité du recuit) et à la taille restreinte des circuits testés. Une évaluation plus robuste nécessiterait d'étendre l'analyse à des architectures plus complexes et à un ensemble plus large de benchmarks. Néanmoins, les tendances observées rejoignent celles rapportées dans des travaux antérieurs, qui soulignent déjà le rôle central du maintien de la diversité dans la prévention de la convergence prématurée. Cela renforce la décision d'activer par défaut le contrôle de diversité dans la suite des expériences.

6.4.7 Paramètres retenus pour l'évaluation

Sur la base des résultats précédents, nous fixons les hyperparamètres suivants (tableau 6.7) pour toutes les expériences de la suite :

Hyperparamètre	Valeur retenue
Stratégie de sélection	Roulette (roulette wheel)
Taux de croisement	0.90
Taux de mutation	0.10
Mutation structurée	Activée
Croisement	à un point
Optimisation locale des angles	Activée
Contrôle de diversité (recuit + adaptation)	Activé
Taille de population / Générations	cf. tableau 6.1 selon cas

TABLE 6.7 – Hyperparamètres fixés à partir des études d’ablation.

Ces choix équilibrent fidélité finale, profondeur, coût et rapidité de convergence, tout en assurant la reproductibilité des expériences ultérieures.

6.5 Résultats expérimentaux et tests comparatifs

Cette section présente les résultats obtenus pour chaque stratégie testée, accompagnés d’illustrations sous forme de graphiques et tableaux. Nous comparons les performances obtenues avec les objectifs initiaux, notamment l’amélioration de la fidélité, la réduction de la profondeur et la simplification structurelle des circuits générés.

Le nombre total de tests et leur répartition ont été déterminés selon un compromis couverture statistique et coût de calcul, de manière à garantir la validité interne des résultats et leur généralisation à des topologies et objectifs variés. Le plan d’expérimentation a été structuré autour de quatre axes complémentaires : la validation fonctionnelle sur des cas unitaires canoniques (CNOT, Toffoli, Fredkin) , la robustesse via des circuits aléatoires de tailles croissantes et la pertinence applicative au moyen de circuits orientés problème (Max-Cut, VQE, QFT/ansatz structurés) .

6.5.1 Tests de l’algorithme mono-objectif

1. Tests sur portes classiques complexes :

Dans un premier temps, nous avons évalué l’algorithme sur des cas simples et bien établis dans la littérature, afin de valider son bon fonctionnement de manière progressive. Ces tests initiaux ont consisté à retrouver des circuits équivalents pour certaines portes non unitaires, telles que la porte CNOT, la porte Toffoli, et d’autres. La figure 6.5 représente le circuit quantique équivalent à la porte quantique CNOT.

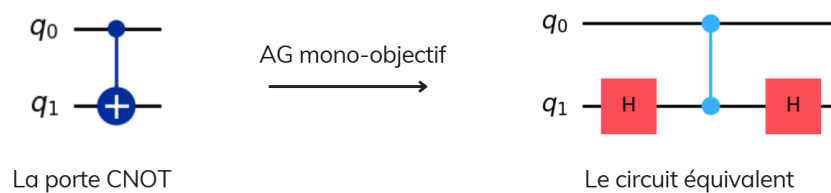


FIGURE 6.5 – Reconstruction de la porte CNOT via l’algorithme génétique mono-objectif

La courbe présentée dans figure 6.6 ci-dessous, illustrant l'évolution de la fidélité (fitness) du meilleur individu au fil des générations. On observe que le circuit équivalent à la porte CNOT est découvert dès la 13^e génération, où la fidélité atteint la valeur maximale de 1.0. Cette convergence rapide témoigne de l'efficacité de l'algorithme génétique mono-objectif à identifier des circuits équivalents en un nombre réduit d'itérations, même à partir d'une population initialement aléatoire.

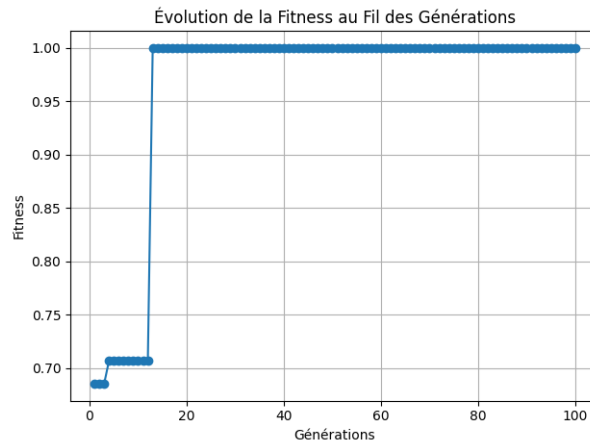


FIGURE 6.6 – L'évolution de la fidélité au fil des générations de la porte CNOT via l'algorithme génétique mono-objectif

Le tableau 6.8 illustre la capacité de notre algorithme génétique mono-objectif à reproduire et simplifier des circuits cibles complexes.

Test	Qubits	Fidélité	Profondeur		Portes / CNOTs		Convergence
			Avant	Après	Avant	Après	
CNOT	2	1.00	1	3	1 / 1	3 / 0	10 générations
Toffoli	3	1.00	18	7	18 / 8	14 / 6	30 générations
Fredkin	3	1.00	1	3	1/0	3/2	20 générations

TABLE 6.8 – Résultats expérimentaux sur des circuits quantiques complexes (avant/après optimisation) à l'aide de l'algorithme génétique mono-objectif

Le tableau 6.8 met en évidence la capacité de l'algorithme génétique mono-objectif à reconstruire fidèlement des portes logiques quantiques classiques complexes, tout en proposant des circuits équivalents souvent plus compacts. On constate que la fidélité atteint systématiquement la valeur maximale de 1.0, confirmant la validité des circuits générés.

2. Tests sur des circuits aléatoires :

Pour évaluer la capacité de généralisation de l'algorithme génétique, nous avons mené des tests sur des circuits aléatoires générés à l'aide de différentes graines pseudo-aléatoires. Ces circuits permettent de simuler des cas d'usage non déterministes, où la topologie du circuit initial n'est pas optimisée. L'objectif est d'analyser le comportement du processus évolutif face à une complexité non dirigée, en mesurant la fidélité atteinte et la profondeur finale.

La figure 6.7 illustre le circuit quantique optimisé obtenu à partir d'un circuit initial aléatoire composé de 5 qubits.

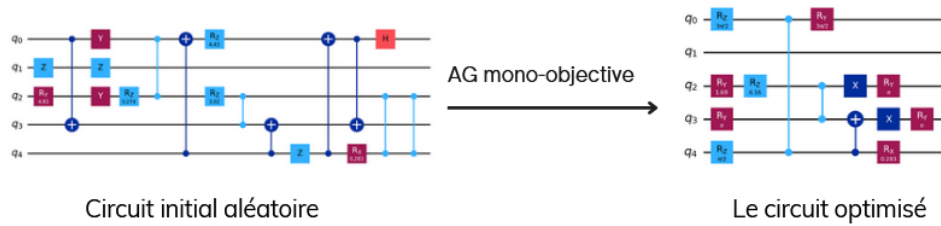


FIGURE 6.7 – L’optimisation d’un circuit quantique aléatoire à l’aide d’un algorithme génétique mono-objectif.

La courbe présentée dans la figure 6.8 illustre une convergence rapide de l’algorithme génétique : la fidélité augmente de manière significative dès les premières générations et atteint un plateau proche de 1.0 avant la 100^e génération. Cela met en évidence l’efficacité du processus évolutif appliqué au circuit quantique aléatoire.

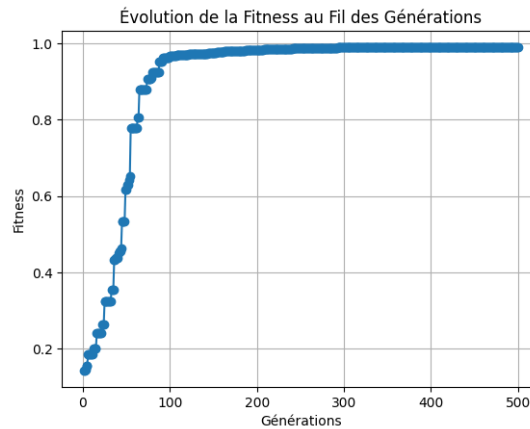


FIGURE 6.8 – L’évolution de la fidélité au fil des générations d’un circuit quantique aléatoire à l’aide d’un algorithme génétique mono-objectif.

Le tableau 6.9 ci-dessous montre que notre algorithme parvient à réduire significativement la complexité du circuit.

Test	Qubits	Fidélité	Profondeur		Portes / CNOTs		Convergence
			Avant	Après	Avant	Après	
Circuit aléatoire 1	5	0.92	15	9	25 / 6	15 / 5	96 générations
Circuit aléatoire 2	5	0.98	11	12	20 / 6	14 / 5	113 générations
Circuit aléatoire 3	6	0.951	15	11	28 / 8	21 / 6	135 générations
Circuit aléatoire 4	7	0.957	18	12	33 / 10	24 / 7	115 générations
Circuit aléatoire 5	8	0.907	22	13	38 / 12	29 / 9	134 générations
Circuit aléatoire 6	8	0.891	24	15	40 / 14	30 / 10	156 générations
Circuit aléatoire 7	10	0.90	14	17	35 / 15	32 / 16	523 générations

TABLE 6.9 – Résultats expérimentaux sur les circuits aléatoires (avant/après optimisation) à l’aide de l’algorithme génétique mono-objectif.

Le tableau 6.9 met en évidence la capacité de l’algorithme génétique mono-objectif à réduire la complexité des circuits aléatoires, aussi bien en termes de profondeur que de nombre total de portes. On observe en effet une diminution systématique

de la profondeur et du nombre de portes après optimisation, tout en maintenant une fidélité globalement satisfaisante (souvent supérieure à 0.9). Cela confirme la robustesse de l'approche face à des topologies initiales non structurées.

Toutefois, plusieurs limites apparaissent :

- La réduction de la profondeur n'est pas toujours garantie. Par exemple, pour certains circuits (cas des 10 qubits), et l'amélioration est moins marquée. Cela montre que l'optimisation mono-objectif centrée principalement sur la fidélité peut négliger la minimisation de la profondeur.
- La taille de l'individu est contrainte par la borne maximale fixée pour le nombre de portes. Ainsi, même si l'algorithme parvient à réduire certaines redondances, il ne peut pas adapter dynamiquement la longueur de la représentation au besoin réel du circuit. Cela limite la capacité de l'approche à explorer des solutions beaucoup plus compactes, puisqu'on opère toujours dans un espace borné de tailles prédéfinies.
- La convergence demande un nombre significatif de générations, ce qui reflète une complexité d'exploration élevée, notamment pour des circuits de taille plus importante.

En résumé, si l'approche mono-objectif garantit une amélioration de la compacité globale, elle ne fournit pas toujours une réduction optimale de la profondeur et reste contrainte par la structure statique des individus. Ces limites justifient l'introduction d'approches multi-objectifs ou hiérarchiques permettant de mieux équilibrer fidélité, profondeur et taille du circuit.

3. Tests sur des circuits quantiques orientés problème :

dans cette partie, nous avons évalué notre approche sur des circuits représentatifs de cas d'usage concrets et réalistes. L'objectif est de vérifier si notre méthode est capable de réduire la complexité structurelle du circuit (profondeur, nombre total de portes, en particulier les portes multi-qubits) tout en maintenant une fidélité élevée. La figure 6.9 représente le processus d'optimisation d'un circuit quantique issu du problème MaxCut, où l'algorithme génétique mono-objectif parvient à simplifier la structure du circuit tout en préservant son comportement fonctionnel.

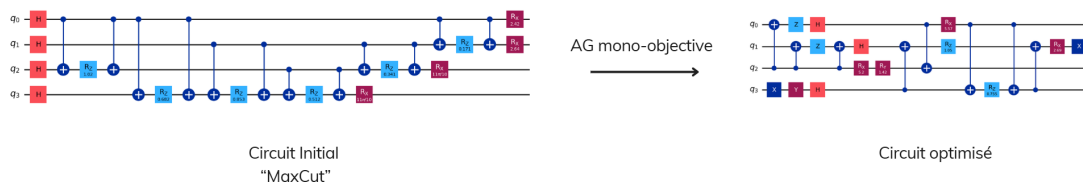


FIGURE 6.9 – L'optimisation d'un circuit quantique de maxcut à l'aide d'un algorithme génétique mono-objectif.

Le tableau 6.10 ci-dessous résume les résultats obtenus.

Test	Qubits	Fidélité	Profondeur		Portes / CNOTs		Convergence
			Avant	Après	Avant	Après	
VQE Ansatz	4	0.9997	6	14	15 / 3	25 / 2	24 générations
L'état W_5	5	0.980	7	8	15 / 5	12 / 4	160 générations
Add-Rotate	5	0.958	14	9	22 / 7	16 / 5	108 générations
MaxCut (g1)	6	0.870	13	7	26 / 8	20 / 4	120 générations
MaxCut (g2)	6	0.890	14	9	27 / 9	22 / 8	115 générations

TABLE 6.10 – Résultats expérimentaux sur circuits applicatifs (avant/après optimisation) à l'aide d'un algorithme génétique mono-objectif.

Comme déjà mentionné lors des tests sur circuits aléatoires, l'algorithme mono-objectif parvient souvent à réduire la taille et le nombre de portes, tout en maintenant une fidélité élevée. Le tableau 6.10 confirme cette tendance sur des circuits applicatifs : par exemple, dans les cas Add-Rotate et MaxCut, la profondeur et le nombre de CNOTs sont significativement réduits, ce qui illustre la capacité de l'approche à simplifier des structures fonctionnelles complexes.

Toutefois, les limites déjà relevées se manifestent également ici. Pour certains circuits, comme le VQE Ansatz ou l'état $|W_5\rangle$, l'optimisation n'entraîne pas une réduction nette de la profondeur, voire conduit à une augmentation. De plus, la convergence reste très variable selon la nature du problème, allant de quelques dizaines à plus d'une centaine de générations. Cela montre que, bien que la fidélité soit bien préservée, le critère mono-objectif ne garantit pas une réduction systématique de la complexité structurelle, ce qui motive l'exploration d'approches multi-objectifs plus équilibrées.

4. Évolution du temps d'exécution selon le nombre de qubits

La figure 6.10 illustre l'évolution du temps d'exécution de l'algorithme génétique mono-objectif en fonction du nombre de qubits, pour des circuits générés aléatoirement. Chaque expérience a été conduite avec la configuration suivante :

- **Méthode de sélection** : roulette (roulette wheel selection) ;
- **Optimisation locale** des angles de rotation activée ;
- **Taux de mutation** : 0,1 ;
- **Taux de croisement** : 0,9 ;
- **Contrôle de diversité** et **élitisme** activés ;
- **Nombre de générations** : 200 ;
- **Taille de la population** : 200 ;
- **Nombre de gènes par individu** : 12.

On observe une croissance quasi-exponentielle du temps d'exécution avec l'augmentation du nombre de qubits, principalement due au coût de calcul de la fidélité unitaire (via la matrice globale) qui devient intraitable au-delà de 10–12 qubits.

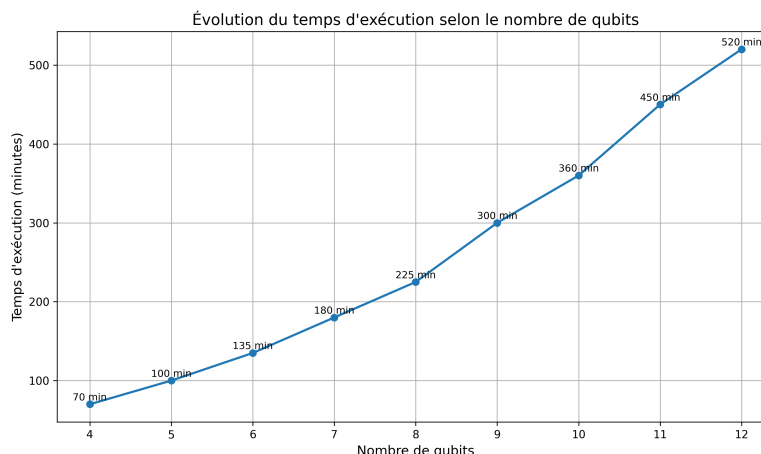


FIGURE 6.10 – Évolution du temps d’exécution (en minutes) de l’algorithme génétique mono-objectif selon le nombre de qubits (4 à 12), sous configuration complète.

5. Analyse de stabilité inter-instance

Nous avons réalisé une série d’expériences sur un circuit généré aléatoirement, évalué sur **10 instances différentes**. L’objectif est de mesurer la stabilité de l’algorithme génétique en analysant les variations de la fidélité et du nombre de générations nécessaires à la convergence.

Les résultats obtenus sont présentés dans le tableau 6.11. On y observe une **fidélité finale très homogène**, avec une moyenne de $\mu = 0.9708$ et un écart-type $\sigma = 0.0051$, ce qui témoigne de la robustesse du processus évolutif.

Instance	Fidélité finale	Convergence (générations)
1	0.978	102
2	0.965	98
3	0.973	108
4	0.968	99
5	0.976	101
6	0.974	97
7	0.962	106
8	0.970	95
9	0.975	100
10	0.967	103
Moyenne $\pm$ Écart-type	0.971 $\pm$ 0.0051	100.9 $\pm$ 3.7

TABLE 6.11 – Fidélité finale obtenue sur 10 circuits aléatoires distincts – Évaluation de la stabilité

Formule mathématique : L’écart-type σ est une mesure statistique de la dispersion des résultats autour de la moyenne μ , et est défini par la formule suivante :

$$\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2} \quad (6.5.1)$$

où :

- x_i est la fidélité atteinte pour l'instance i ,
- $n = 10$ est le nombre total d'expériences,
- μ est la moyenne des fidélités.

Observation : Malgré des topologies différentes, le modèle converge systématiquement vers des solutions de haute qualité avec une dispersion faible ($\sigma = 0.0051$). Cette stabilité prouve que l'approche reste robuste, même face à la variabilité aléatoire des instances.

Bien que la fidélité finale reste stable sur l'ensemble des expériences, une variation modérée est constatée sur le nombre de générations nécessaires à la convergence ($\sigma = 3.7$). Cela s'explique par la nature stochastique de l'algorithme génétique, combinée aux différences structurelles entre circuits, qui peuvent affecter la rapidité de l'évolution.

6.5.2 Tests de l'algorithme multi-objectifs

1. Tests sur des circuits aléatoires

Pour évaluer l'efficacité de l'algorithme NSGA-II dans un contexte non structuré, nous avons appliqué notre méthode d'optimisation à des circuits quantiques aléatoires. Ces circuits initiaux sont générés aléatoirement. L'objectif est de vérifier la capacité de l'algorithme à améliorer simultanément trois critères clés : la fidélité unitaire, la profondeur du circuit, et le coût.

Les figures suivantes illustrent les résultats obtenus. La Figure 6.11 montre un exemple de circuit optimisé. La Figure 6.12 présente le front de Pareto associé, illustrant la diversité des solutions non dominées. Enfin, la Figure 6.13 trace l'évolution des trois métriques au fil des générations.

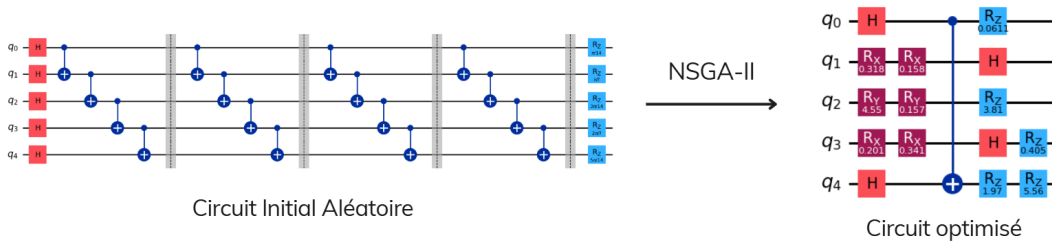


FIGURE 6.11 – Optimisation d'un circuit quantique aléatoire de 5 qubits à l'aide de NSGA-II

L'illustration de la figure 6.11 met en évidence l'impact de l'optimisation par **NSGA-II** sur la qualité d'un circuit quantique. Le circuit initial, généré aléatoirement, présente une forte densité de portes **CNOT**, dont l'utilisation excessive augmente la profondeur et introduit un bruit significatif, ces opérations étant parmi les plus coûteuses dans les architectures quantiques actuelles. Après optimisation, le circuit obtenu est plus compact, mieux structuré et se caractérise par une **réduction notable du nombre de CNOT** au profit de portes à un seul qubit (telles que les rotations et Hadamard), qui sont moins bruitées et plus fiables. Cette transformation se traduit par une diminution simultanée de la profondeur et du coût du circuit, améliorant ainsi sa faisabilité et sa robustesse lorsqu'il est exécuté sur un processeur

quantique physique. De plus, la **fidélité** entre le circuit initial et le circuit optimisé atteint **0.9843**, ce qui confirme que, malgré la simplification et la réduction du bruit, le circuit final préserve le même comportement fonctionnel que le circuit d'origine.

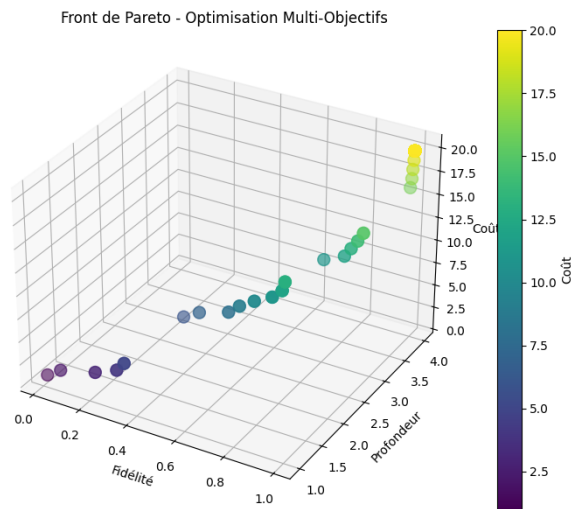


FIGURE 6.12 – Front de Pareto obtenu lors de l'optimisation d'un circuit quantique aléatoire à l'aide de NSGA-II

La figure 6.12 illustre le front de Pareto en 3D qui met en évidence l'ensemble des solutions non dominées obtenu à l'aide de l'algorithme NSGA-II appliqué à un circuit quantique aléatoire. On observe un compromis clair entre la fidélité, le coût structurel et la profondeur du circuit. En particulier, on obtient des solutions qui atteignent une fidélité maximale trop proche de 1 en assurant une réduction remarquable au niveau de coût et de profondeur.

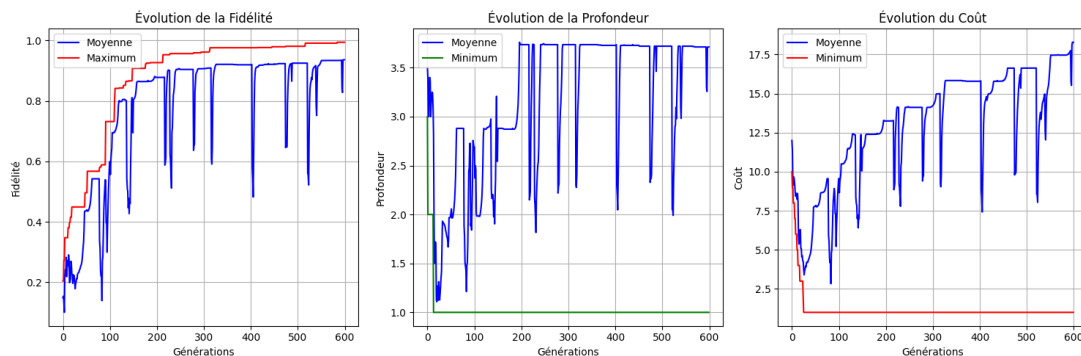


FIGURE 6.13 – Évolution de la fidélité, du coût structurel et de la profondeur au cours de l'optimisation d'un circuit quantique aléatoire à l'aide de NSGA-II

La figure 6.13 contient trois sous-courbes qui montrent l'évolution des métriques de performance au fil des générations. Trois sous-graphes suivent respectivement : la fidélité, la profondeur et le coût.

1. Fidélité La fidélité moyenne grimpe rapidement dès les premières générations (passant de 0.2 à 0.8 vers génération 100) et la fidélité maximale atteint presque 1 vers génération 300 et reste stable ensuite, ce qui montre que l'algorithme a efficacement exploré l'espace et trouvé des candidats presque parfaits.

2. Profondeur La profondeur moyenne oscille mais reste globalement faible (2 à 3), donc le NSGA-II parvient à maintenir une diversité structurelle, incluant à la fois des circuits complexes et des circuits très compacts.

3. Coût Le coût moyen augmente progressivement avec les générations (de 5 à 17), cela confirme encore que l'algorithme maintient des solutions de faible coût tout en générant des solutions plus coûteuses mais à haute fidélité.

Le tableau 6.12 récapitule des mesures obtenues avant et après l'optimisation de plusieurs circuits aléatoires à l'aide de NSGA-II.

Test	Qubits	Fidélité	Profondeur		Portes / multi-qubits		Coût	
			Avant	Après	Avant	Après	Avant	Après
Circuit aléatoire 1	5	0.9944	18	4	26 / 16	16 / 1	95	26
Circuit aléatoire 2	5	0.9747	12	6	19 / 10	12 / 4	55	27
Circuit aléatoire 3	4	0.9891	11	6	22/7	15/1	54	19
Circuit aléatoire 4	5	0.9378	11	4	26/8	9/2	63	17
Circuit aléatoire 5	6	0.9773	16	4	32/20	13/2	118	22
Circuit aléatoire 6	7	0.9881	17	5	38/24	20/3	141	32
Circuit aléatoire 7	8	0.9896	18	6	44/28	20/4	164	41

TABLE 6.12 – Résultats expérimentaux sur les circuits aléatoires (avant/après optimisation) à l'aide de NSGA-II

L'ensemble des résultats présentés dans le tableau 6.12 met en lumière l'efficacité de l'algorithme NSGA-II pour l'optimisation multi-objectifs des circuits quantiques aléatoires. On observe systématiquement une **réduction significative** de la profondeur, du coût et du nombre de portes, en particulier des multi-qubits, qui sont les plus bruitées. Dans certains cas, la profondeur est divisée par plus de quatre et le coût global réduit de plus de 70 %. Malgré cette simplification, la **fidélité** reste élevée (≥ 0.93 , souvent proche de 0.99), confirmant que les circuits optimisés conservent le même comportement fonctionnel tout en étant mieux adaptés à une exécution physique.

2. Tests sur des circuits quantiques orientés problème

Dans ce cas, notre approche basée sur NSGA-II est testée sur des circuits orientés problème, c'est-à-dire des circuits conçus spécifiquement pour résoudre une tâche quantique donnée. La figure 6.14 représente l'optimisation d'un circuit quantique VQE de type *Structured Ansatz* à l'aide de NSGA-II.

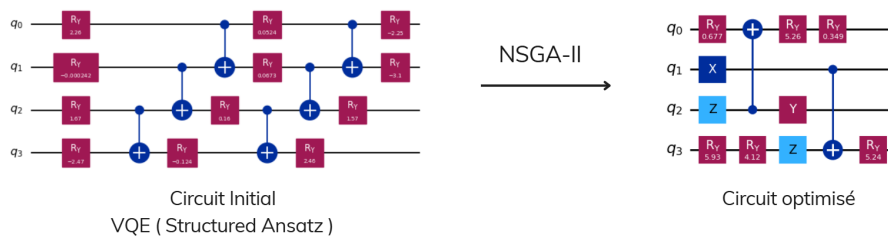


FIGURE 6.14 – L'optimisation d'un circuit quantique vqe de type "Structured Ansatz" à l'aide de NSGA-II

L'illustration de la figure 6.14 illustre l'optimisation d'un circuit quantique VQE de type Structured Ansatz par l'algorithme NSGA-II. Le circuit initial, caractérisé par une profondeur relativement élevée et certaines redondances, est transformé en une version plus compacte et structurée. L'optimisation aboutit à une réduction notable du nombre de portes et de la profondeur, tout en préservant la fidélité fonctionnelle du circuit, ce qui améliore sa robustesse et sa faisabilité sur processeur quantique physique.

La figure 6.15 montre le front de Pareto obtenu lors de cette optimisation, mettant en évidence les compromis entre fidélité, profondeur et coût structurel.

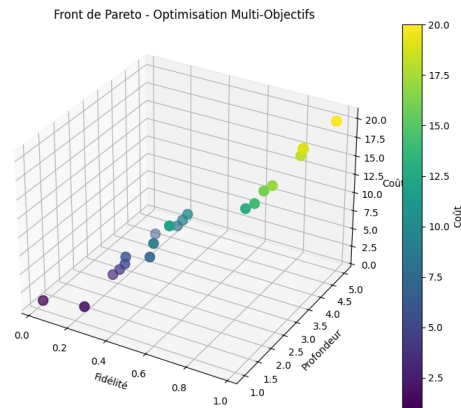


FIGURE 6.15 – Front de Pareto obtenu lors de l'optimisation d'un circuit quantique VQE à l'aide de NSGA-II

La figure 6.16 illustre l'évolution simultanée de la fidélité, du coût structurel et de la profondeur au cours des générations, ce qui permet d'analyser la dynamique de convergence de l'algorithme.

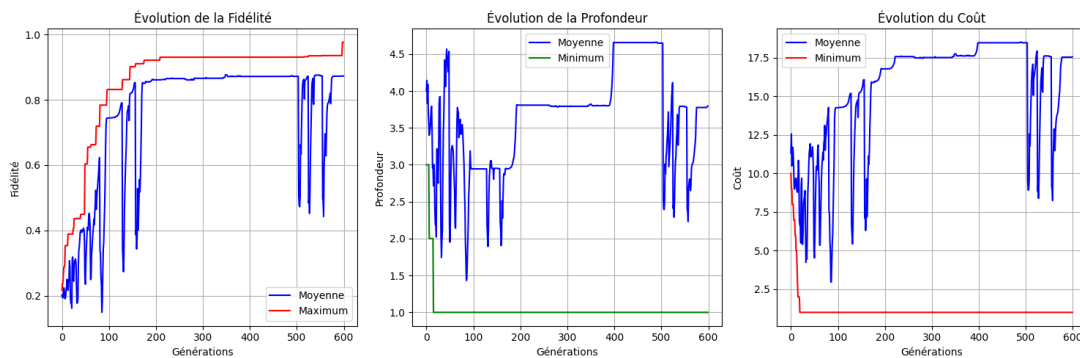


FIGURE 6.16 – Évolution de la fidélité, du coût structurel et de la profondeur au cours de l'optimisation d'un circuit quantique VQE à l'aide de NSGA-II

Le tableau 6.13 illustre les gains obtenus après optimisation, avec une réduction marquée de la profondeur et du nombre de portes, tout en maintenant une fidélité élevée sur l'ensemble des circuits étudiés.

Test	Qubits	Fidélité	Profondeur		Portes / CNOTs		Coût	
			Avant	Après	Avant	Après	Avant	Après
Grover_n2	2	0.9996	11	5	16 / 2	5 / 2	34	14
Deutsch_n2	2	1.0000	4	3	5 / 1	3 / 1	12	8
Linearsolver_n3	3	0.9578	11	4	19 / 4	7 / 0	47	7
Bell_n4	4	0.8604	13	3	33/7	8/1	64	14
VQE (Ansatz)	4	0.9763	8	5	18 / 6	12 / 2	42	20

TABLE 6.13 – Résultats expérimentaux sur les circuits quantiques orientés problème à l’aide de NSGA-II

L’ensemble des résultats présentés dans le tableau 6.13 confirme, à l’instar des circuits aléatoires, l’efficacité de l’algorithme NSGA-II pour l’optimisation multi-objectifs des circuits orientés problème. On observe une réduction systématique de la profondeur, du nombre de portes et du coût global, avec, dans certains cas, une division par plus de trois (par exemple, le circuit *Linearsolver_n3* dont le coût passe de 47 à 7). Cette simplification se traduit également par une forte diminution du nombre de portes multi-qubits, les plus sensibles au bruit, ce qui améliore la robustesse des circuits optimisés. Malgré ces réductions, la fidélité reste globalement élevée (≥ 0.86 et souvent proche de 0.99), ce qui confirme que les transformations préservent le comportement fonctionnel des circuits d’origine tout en les rendant plus adaptés à une exécution sur processeur quantique physique.

3. Évolution du temps d’exécution selon le nombre de qubits

La figure 6.17 illustre l’évolution du temps d’exécution de l’algorithme génétique multi-objectifs en fonction du nombre de qubits, pour des circuits générés de manière aléatoire. Deux représentations complémentaires sont proposées : une courbe linéaire (en haut) et une courbe en échelle logarithmique (en bas). Cette dernière permet de mieux mettre en évidence la nature de la croissance, en particulier lorsque le temps d’exécution augmente rapidement avec le nombre de qubits, rendant la comparaison difficile sur une échelle linéaire. Les expériences ont été réalisées sous la configuration suivante :

- **Optimisation locale** des angles de rotation activée ;
- **Taux de mutation** : 0,1 ;
- **Taux de croisement** : 0,9 ;
- **Contrôle de diversité** et **élitisme** activés ;
- **Nombre de générations** : 150 ;
- **Taille de la population** : 150 ;
- **Nombre de gènes par individu** : 13.

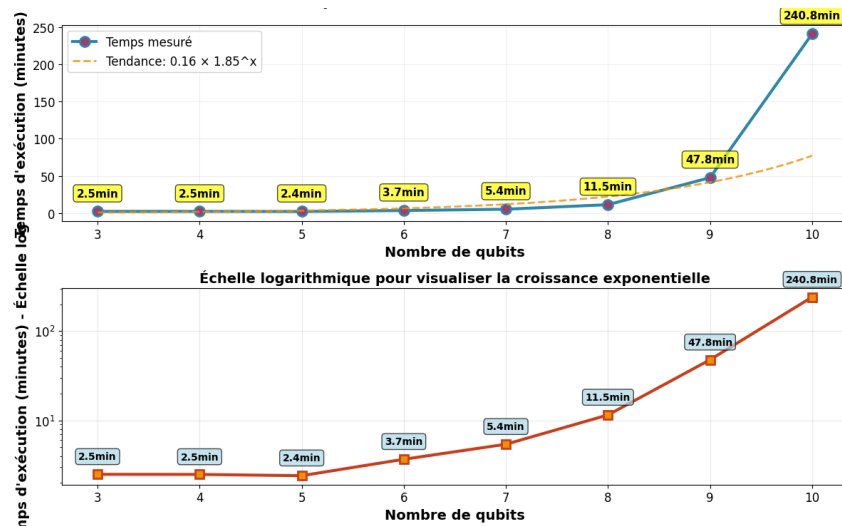


FIGURE 6.17 – Évolution du temps d'exécution de l'algorithme génétique multi-objectifs en minutes et sur l'échelle logarithmique en fonction de nombre de qubits (3 à 10), sous configuration complète.

Les résultats obtenus montrent que pour 3 à 5 qubits, le temps d'exécution reste quasi constant autour de 2,5 minutes. À partir de 6 qubits, la croissance devient perceptible puis exponentielle : 11,5 min pour 8 qubits, 47,8 min pour 9 qubits et plus de 4 heures pour 10 qubits. Une régression exponentielle du type $T(n) \approx 0.16 \times 1.85^n$ décrit bien la tendance, liée à la croissance en $O(4^n)$ en mémoire et quasi $O(8^n)$ en temps. En pratique, l'algorithme devient difficilement exploitable au-delà de 9 qubits.

6.5.3 Tests des stratégies d'estimation de la fidélité

Afin de tester les deux méthodes d'estimation de la fidélité pour des circuits de moyenne et grande échelle, présentées dans les chapitres précédents, nous les avons appliquées à des circuits quantiques générés aléatoirement ainsi qu'à des circuits orientés problème.

6.5.3.1 Tests méthode 1 : Évaluation de la fidélité par blocs indépendants

1. Tests sur des circuits aléatoires

L'évaluation par blocs indépendants a d'abord été appliquée à des circuits aléatoires, afin de tester la robustesse de la méthode dans des configurations non orientées problème. L'idée est de partitionner un circuit initialement complexe en sous-ensembles de qubits faiblement connectés, puis d'optimiser séparément chaque bloc avant de reconstruire le circuit global. Cette approche permet de réduire la profondeur et le nombre de portes, tout en obtenant une estimation approchée de la fidélité.

La figure 6.18 montre le processus d'optimisation d'un circuit quantique aléatoire à l'aide de la méthode des blocs indépendants.

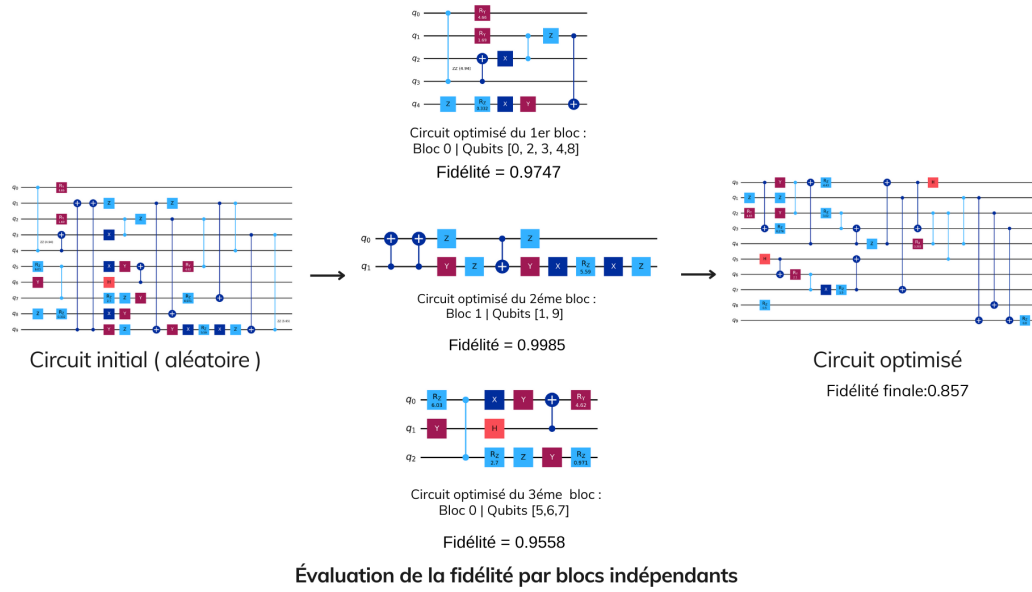


FIGURE 6.18 – L’optimisation d’un circuit quantique aléatoire avec la méthode des blocs indépendants

Des tests seront présentés dans les tableaux ci-dessous 6.14 pour évaluer la robustesse de la méthode face à des structures de circuits plus complexes.

Test	Qubits	Fid. approx	Fid. exact	Blocs	Profondeur		Portes		Coût		Portes Inter-blocs
					Avant	Après	Avant	Après	Avant	Après	
1	8	0.9460	0.8960	{0,2,3,4}, {1,5,6,7}	11	7	30	24	87	63	2
2	8	0.9600	0.9460	{0,2,4,5}, {1,3,6,7}	13	8	35	26	96	81	4
3	10	0.8977	0.8577	{0,2,3,4,8}, {1,9}, {5,6,7}	14	10	56	40	104	76	2
4	12	0.8577	0.7870	{0,2,3,4,8,9}, {1,5,6,7,10,11}	12	9	35	26	98	80	1
5	13	0.7977	0.6200	{1,5,6,7,8,12}, {0,2,3,4,9}, {10,11}	12	7	32	22	79	56	7

TABLE 6.14 – Résultats expérimentaux sur des circuits aléatoires avec la méthode des blocs indépendants

La figure 6.18 illustre clairement la réduction de la complexité du circuit initial grâce à l’approche par blocs indépendants. On constate que le partitionnement en sous-ensembles faiblement connectés permet d’obtenir un circuit global plus compact, avec une profondeur réduite et un nombre de portes moindre, ce qui est un résultat attendu et positif. Toutefois, l’image reste qualitative et ne permet pas, à elle seule, de juger de la précision de l’approximation en termes de fidélité.

Le tableau 6.14 vient quantifier ces observations. On remarque, pour tous les circuits testés, une diminution de la profondeur (par exemple de 13 à 8 dans le test 2, ou de 12 à 7 dans le test 5) ainsi qu’une réduction notable du nombre de portes et du coût structurel. Ces améliorations confirment la capacité de la méthode à simplifier efficacement des circuits aléatoires, même lorsque le nombre de qubits augmente.

En revanche, on observe une baisse systématique de la fidélité exacte après optimisation. Par exemple, dans le test 5, la fidélité passe de 0.7977 (approx.) à 0.6200, ce qui traduit une perte de précision significative. De même, dans le test 4, la fidélité chute de 0.8577 à 0.7870. Ce compromis met en évidence la principale limite de la méthode : la simplification structurelle s’accompagne d’une dégradation de la fidélité,

qui peut devenir problématique pour certaines applications sensibles à l'exactitude des résultats.

En résumé, la méthode des blocs indépendants montre une bonne efficacité pour réduire la profondeur et le coût des circuits, ce qui est essentiel dans le contexte des dispositifs NISQ. Toutefois, cette simplification se paie par une baisse de la fidélité, et son utilisation doit être envisagée en tenant compte du compromis entre performance structurelle et précision quantique. De plus, la réduction de la taille des sous-problèmes entraîne une diminution notable du temps d'exécution, ce qui constitue un avantage pratique important pour le traitement de circuits de grande taille. Par ailleurs, plus le circuit est fortement intriqué et dense, plus on observe une baisse significative de la fidélité, ce qui montre que cette méthode est surtout adaptée aux circuits faiblement intriqués.

2. Tests sur des circuits quantiques orientés problème

Dans cette section, nous appliquons la méthode d'évaluation par blocs indépendants à des circuits quantiques conçus pour résoudre des problèmes concrets à topologie faiblement connectée. Ces circuits incluent, par exemple, des circuits QAOA appliqués à des graphes en chaîne ou à faible degré. L'objectif est de démontrer que, lorsque les interactions entre qubits sont naturellement limitées par la structure du problème, la décomposition en blocs permet non seulement de **réduire la profondeur** et le **coût structurel** du circuit, mais aussi de **préserver une approximation fidèle de la fidélité globale $\mathcal{F}$** . De plus, cette approche se traduit par un **gain significatif en temps d'exécution**, grâce à l'évaluation localisée et parallélisable des sous-circuits. Le tableau 6.15 illustre ces résultats en comparant les métriques avant et après optimisation pour différents circuits faiblement connectés. On y observe des réductions notables du coût et de la profondeur.

Test	Qubits	Fid. approx	Fid. exact	Blocs	Profondeur		Portes		Coût		Portes Inter-blocs
					Avant	Après	Avant	Après	Avant	Après	
1 – QAOA	8	0.9460	0.8960	{0,2,3,4}, {1,5,6,7}	11	7	87	63	130	98	2
2 – MaxCut	10	0.8260	0.7260	{0,1,2,3,4}, {5,6,7,8,9}	12	10	30	22	40	28	3

TABLE 6.15 – Résultats expérimentaux sur des circuits quantiques orientés problème avec la méthode des blocs indépendants

L'analyse des résultats du tableau 6.15 montre que l'application de la méthode par blocs indépendants permet une réduction notable de la profondeur et du coût structurel, ce qui illustre son efficacité dans des problèmes à connectivité limitée. Toutefois, la baisse de la fidélité est plus marquée que dans le cas de QAOA, ce qui traduit le fait que les interactions nécessaires pour résoudre MaxCut sont plus denses et intriquées. Ainsi, cette méthode demeure avantageuse en termes de gain de temps d'exécution et de simplification structurelle, mais elle présente une limite en précision lorsque le circuit devient plus fortement corrélé.

3. **Comparaison qualitative avec la méthode exacte :** En comparaison avec une simulation exacte (sans partitionnement), la méthode par blocs indépendants introduit une perte de précision. Néanmoins, cette perte reste limitée (typiquement 5 à 10% de fidélité) alors que le coût en temps de calcul et en mémoire est significativement réduit. En particulier, la réduction de la profondeur (jusqu'à 40%) et du nombre de portes montre que la méthode permet de manipuler des circuits plus larges

qui seraient difficilement simulables dans leur globalité. Ainsi, le compromis temps de calcul / précision apparaît favorable pour l'exploration de circuits de taille intermédiaire. Ces résultats montrent que la méthode reste exploitable pour des circuits dépassant 10 qubits, ce qui est essentiel dans un contexte NISQ où les ressources sont limitées. L'approche par blocs permet en effet de maintenir une estimation de la fidélité cohérente, tout en rendant possible l'analyse de circuits qui, autrement, seraient inaccessibles par simulation exacte. La scalabilité est donc l'un des principaux avantages de cette méthode.

6.5.3.2 Tests méthode 2 : Évaluation de la fidélité par blocs séquentiels chevauchants (fenêtres glissantes)

1. Tests sur des circuits quantiques aléatoires

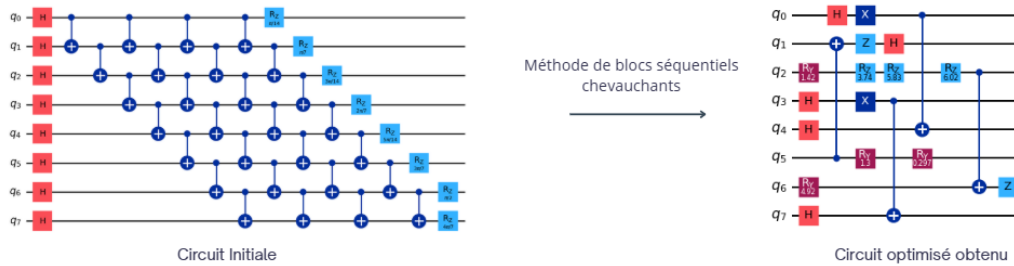


FIGURE 6.19 – Optimisation d'un circuit quantique aléatoire de 8 qubits à l'aide de la méthode d'évaluation de la fidélité par blocs séquentiels chevauchants (fenêtres glissantes)

La figure 6.19 illustre un exemple d'optimisation d'un circuit quantique aléatoire fortement dense en utilisant la méthode d'évaluation de la fidélité par blocs séquentiels chevauchants. Le circuit initial est composé d'un grand nombre de portes, notamment de nombreuses portes **CNOT** disposées en cascade. La profondeur y est élevée, ce qui entraîne une accumulation significative d'erreurs et un coût d'exécution important. La structure présente une organisation régulière mais peu optimisée : presque chaque qubit interagit avec plusieurs autres, ce qui conduit à une forte densité de connexions et accroît la complexité globale du circuit. Grâce à l'application de la méthode par fenêtres glissantes, le circuit est découpé en sous-blocs partiellement chevauchants qui sont analysés et compressés localement avant d'être réassemblés. Cette approche permet d'identifier et de supprimer les redondances, de fusionner certaines séquences de portes et de mieux paralléliser les opérations. Le circuit optimisé obtenu présente ainsi une profondeur réduite, un nombre significativement plus faible de portes **CNOT** et donc un coût global moindre. Malgré ces simplifications, la fidélité reste supérieure à 0.9, ce qui garantit que le circuit transformé conserve un comportement fonctionnel très proche de celui du circuit initial.

Instance	Qubits	Fidélité approximée	Fidélité exacte	Profondeur		Portes		Coût	
				Avant	Après	Avant	Après	Avant	Après
1	8	0.9623	0.9102	16	8	38	24	93	58
2	10	0.9031	0.8476	14	9	42	31	127	87
3	11	0.8707	0.8298	16	10	67	47	159	101

TABLE 6.16 – Résultats expérimentaux sur des circuits aléatoires avec la méthode des blocs chevauchants (fenêtres glissantes)

L’analyse des résultats du tableau 6.16, réalisé sur des circuits aléatoires, montre que l’évaluation par blocs séquentiels chevauchants, intégrée à NSGA-II, permet une réduction notable des ressources (profondeur divisée par deux, 30–40 % de portes en moins et jusqu’à 50 % de coût en moins), tout en maintenant une fidélité élevée (≥ 0.85). L’écart entre la fidélité approximée et exacte reste inférieur à 0.05, ce qui confirme la robustesse de l’approche. En plus de ces améliorations structurelles, la méthode offre un **gain significatif en temps de calcul**, rendant l’évaluation scalable pour des circuits de taille moyenne à grande, là où une estimation exacte serait intractable.

2. Tests sur des circuits quantiques orientés problème

Instance	Qubits	Fidélité approximée	Fidélité exacte	Profondeur		Portes		Coût	
				Avant	Après	Avant	Après	Avant	Après
1 – VQE	9	0.8973	0.8712	13	8	53	36	107	69
2 – QFT	10	0.8314	0.7931	15	9	75	56	162	89

TABLE 6.17 – Résultats expérimentaux sur des circuits quantiques orientés problème avec la méthode des blocs chevauchants (fenêtres glissantes)

L’analyse du tableau 6.17 confirme que, tout comme dans le cas des circuits aléatoires, la méthode d’évaluation par blocs chevauchants intégrée à NSGA-II demeure efficace sur des circuits orientés problème. Dans l’exemple du VQE à 9 qubits, on observe une réduction sensible de la profondeur (de 13 à 8), du nombre de portes (de 53 à 36) et du coût global (de 107 à 69). La fidélité approximée (0.8973) reste proche de la fidélité exacte (0.8712), avec un écart faible, ce qui confirme la robustesse de l’approximation.

3. Comparaison qualitative avec la méthode exacte :

La méthode NSGA-II classique, basée sur le calcul global de l’opérateur unitaire, garantit une fidélité exacte mais présente une complexité exponentielle $\mathcal{O}(2^n)$ en temps et $\mathcal{O}(4^n)$ en mémoire, ce qui la rend vite prohibitive au-delà de 10–12 qubits. L’approche par blocs chevauchants réduit cette complexité à $\mathcal{O}(k \times 2^b)$ et $\mathcal{O}(k \times 4^b)$, avec k le nombre de blocs et b la taille des blocs, ce qui permet de traiter des systèmes plus grands (20+ qubits). Cette approximation introduit une perte de précision limitée (5–15%), laquelle se reflète par un écart entre la valeur de fitness exacte et celle obtenue par approximation, la fidélité approchée étant souvent légèrement plus optimiste que la fidélité exacte. En contrepartie, elle s’accompagne d’une réduction significative de la profondeur des circuits (30–40%) et d’une meilleure scalabilité. En outre, elle s’adapte mieux aux contraintes NISQ (connectivité restreinte, bruits, temps de cohérence limités). Ainsi, le compromis entre précision et coût de calcul

devient favorable dès une dizaine de qubits, faisant de l'approche par blocs une alternative efficace pour l'optimisation multi-objectifs.

6.5.3.3 Comparaison des temps d'exécution

Une évaluation empirique a été menée afin de comparer le coût temporel des différentes méthodes d'estimation de la fidélité. L'objectif est de quantifier les gains offerts par les approches scalables, notamment la méthode par blocs indépendants, face au calcul classique de la fidélité basé sur la matrice unitaire complète.

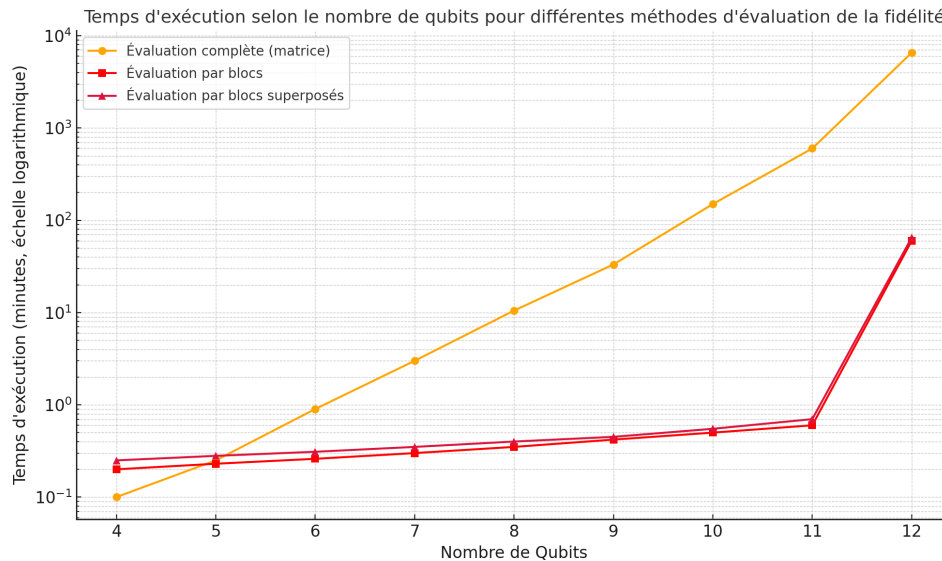


FIGURE 6.20 – Comparaison des temps d'exécution entre l'évaluation globale par matrice unitaire et les méthodes scalables (par blocs indépendants et blocs superposés).

Comme illustré dans la figure 6.20, l'évaluation classique de la fidélité via le calcul de la matrice unitaire devient rapidement intraitable au-delà de 10 qubits, avec une croissance exponentielle du temps d'exécution. À l'inverse, les méthodes proposées — fondées sur une décomposition en blocs locaux ou superposés — maintiennent une croissance quasi-linéaire du temps, grâce à une évaluation localisée et parallélisable.

Ces approches permettent un gain de plus de 10× à 14 qubits, rendant possible l'évaluation de circuits de grande taille dans un cadre raisonnable, même sur simulateur classique. Ce résultat confirme l'intérêt pratique des stratégies d'approximation proposées dans notre travail.

Conclusion

Les expérimentations menées ont permis de valider l'efficacité et la robustesse de nos approches évolutionnaires pour la synthèse automatique de circuits quantiques. L'algorithme génétique mono-objectif a montré de bonnes performances sur des circuits aléatoires et orientés problème, avec une fidélité élevée et une complexité réduite. Les tests d'ablation ont mis en évidence l'importance des composantes telles que la mutation structurée, la sélection et l'optimisation locale.

L'extension multi-objectifs avec NSGA-II a permis d'atteindre un compromis pertinent entre fidélité, profondeur et coût. Par ailleurs, les stratégies d'évaluation scalable de la fidélité ont considérablement réduit le temps de calcul, rendant notre méthode applicable à des circuits plus larges. L'ensemble des résultats confirme la pertinence et la stabilité de notre approche.

Conclusion générale et perspectives

Conclusion

Ce mémoire s’est inscrit dans un contexte marqué par l’essor de l’informatique quantique et les limites des dispositifs actuels dits NISQ. La conception manuelle de circuits quantiques se heurte en effet à de fortes contraintes : croissance exponentielle de la taille des matrices unitaires, bruit et décohérence, ainsi que la nécessité de réduire le nombre de portes et la profondeur des circuits. Face à ces défis, la conception automatique s’impose comme une alternative prometteuse, permettant de générer des circuits plus compacts et mieux adaptés aux architectures matérielles existantes.

La première partie du mémoire a été consacrée à une étude bibliographique approfondie. Nous avons présenté les concepts fondamentaux de l’informatique quantique, depuis la représentation des qubits et des portes jusqu’à la modélisation des circuits et de leur exécution bruitée. Nous avons ensuite introduit les principales familles de métaheuristiques bio-inspirées, en insistant sur les algorithmes évolutionnaires et les mécanismes qui les rendent adaptés à l’optimisation combinatoire. Enfin, un état de l’art détaillé a montré les premières applications de ces techniques à la synthèse automatique de circuits quantiques, soulignant à la fois leur potentiel et leurs limites.

Sur cette base théorique, la seconde partie a présenté notre contribution originale. Nous avons développé dans un premier temps, un algorithme génétique mono-objectif a été développé, centré sur la réduction de l’erreur de fidélité entre un circuit candidat et une matrice unitaire cible. Cette première phase a permis de poser les fondations d’une représentation génétique adaptée aux circuits quantiques, d’implémenter des opérateurs de variation (mutation, croisement) ainsi que des mécanismes de sélection cohérents avec la nature stochastique de l’optimisation. Dans un second temps, les limites rencontrées sur des architectures de type NISQ — notamment la profondeur excessive des circuits et le coût structurel des portes multi-qubits — ont conduit à la mise en place d’une extension multi-objectifs. Celle-ci, basée sur l’algorithme NSGA-II, prend simultanément en compte la fidélité unitaire, la profondeur du circuit et le coût global des opérations quantiques. Cette formulation permet d’explorer un front de Pareto de solutions, révélant des compromis pertinents entre performance théorique et contraintes physiques.

L’efficacité de cette approche a été renforcée par plusieurs techniques complémentaires : l’introduction de mutations structurelles préservant la validité fonctionnelle des circuits, une recherche locale des angles de rotation pour affiner les portes paramétriques, ainsi qu’un mécanisme de contrôle dynamique de la diversité afin d’éviter la stagnation évolutive. L’intégration d’un schéma de sélection adapté, guidé par la domination de Pareto et la distance de densité, a permis d’assurer à la fois l’élitisme et la dispersion au sein de la population.

Enfin, un enjeu majeur a concerné la scalabilité de l’évaluation de la fidélité, souvent limitée par les contraintes mémoire et temps de calcul des simulateurs classiques. Pour y répondre, plusieurs stratégies d’estimation ont été proposées : évaluation locale par blocs indépendants, projections partielles et métriques structurelles, ou encore méthodes inspirées de la physique quantique permettant une approximation sans calcul matriciel

complet. Ces solutions rendent envisageable l'optimisation de circuits dépassant la dizaine de qubits tout en respectant les contraintes imposées par le matériel quantique actuel. En résumé, ce mémoire a permis d'illustrer à la fois les fondements de la conception automatique de circuits quantiques et la pertinence d'une approche évolutionnaire adaptée aux contraintes des dispositifs actuels. L'apport de ce travail se situe à l'interface entre une étude théorique approfondie et une contribution méthodologique originale, validée expérimentalement.

Perspectives

Pour prolonger ce travail, plusieurs directions de recherche peuvent être envisagées afin d'améliorer la scalabilité, la compatibilité matérielle et l'expressivité fonctionnelle de notre approche :

- **Compilation orientée matériel** : intégrer directement dans le processus de génération les contraintes de topologie physique des dispositifs quantiques (IBM, IonQ, Rigetti...), en adaptant dynamiquement la connectivité entre qubits et en limitant les opérations coûteuses (comme les portes SWAP).
- **Modélisation du bruit dépendante du matériel** : incorporer des modèles réalistes de bruit quantique (fautes de lecture, décohérence, bruit dépendant du type de porte ou du qubit utilisé), afin de guider l'optimisation vers des circuits plus robustes. Cette perspective ouvre la voie à des tests expérimentaux sur du matériel quantique réel.
- **Hybridation avec QAOA et VQE** : combiner notre approche évolutionnaire avec des workflows variationnels (QAOA, VQE) dans lesquels la structure des couches quantiques et les paramètres sont co-optimisés. Cette hybridation permettrait d'exploiter à la fois la recherche globale des algorithmes évolutionnaires et la convergence fine des techniques variationnelles.
- **Amélioration des méthodes d'estimation de fidélité** : dépasser les limites actuelles des techniques approximatives, qui entraînent parfois une perte de précision significative. De futures avancées permettront d'obtenir des estimations plus fiables et plus stables, exploitables à grande échelle. Testées dans notre étude avec une limite imposée par les ressources matérielles disponibles, ces méthodes pourraient à terme s'intégrer dans des workflows d'optimisation adaptés au matériel quantique réel.

Ces axes d'amélioration visent à rendre l'optimisation de circuits quantiques plus efficace, plus proche du matériel, et plus adaptée aux besoins réels des applications quantiques de demain.

Bibliographie

- ABDOU, W., BLOCH, C., CHARLET, D., & SPIES, F. (2013). Algorithme génétique multi-objectifs adaptatif. *5èmes Journées Doctorales/Journées Nationales MACS*.
- AI, G. Q. (2020). Cirq : A Python Framework for Creating, Editing, and Invoking Quantum Circuits.
- BÄCK, T., & SCHWEFEL, H.-P. (1993). An Overview of Evolutionary Algorithms for Parameter Optimization. *Evolutionary Computation*, 1, 1-23. <https://doi.org/10.1162/evco.1993.1.1.1>
- BARNIER, N., & BRISSET, P. (1999). Optimisation par algorithme génétique sous contraintes. *Revue des Sciences et Technologies de l'Information-Série TSI : Technique et Science Informatiques*, 18(1), pp-1.
- BELL, J. S. (1964). On the Einstein Podolsky Rosen paradox. *Physics Physique Fizika*, 1(3), 195-200.
- BENIOFF, P. (1980). The computer as a physical system : A microscopic quantum mechanical Hamiltonian model of computers as represented by Turing machines. *Journal of Statistical Physics*, 22(5), 563-591. <https://doi.org/10.1007/BF01011339>
- BENKIRAT, A. (2019). *Vers une stratégie collaborative de foraging multi-robots* [Mémoire de fin d'études (Master)]. Université 8 Mai 1945 – Guelma.
- BENNETT, C. H., BRASSARD, G., CRÉPEAU, C., JOZSA, R., PERES, A., & WOOTTERS, W. K. (1993). Teleporting an unknown quantum state via dual classical and Einstein-Podolsky-Rosen channels. *Physical Review Letters*, 70(13), 1895-1899.
- BHANDARI, S., NICHELE, S., DENYSOV, S., & LIND, P. G. (2024). How quantum and evolutionary algorithms can help each other : two examples. *arXiv preprint arXiv :2408.00448*.
- BLONDEL, V. D., GUILLAUME, J.-L., LAMBIOTTE, R., & LEFEBVRE, E. (2008). Fast unfolding of communities in large networks. *Journal of Statistical Mechanics : Theory and Experiment*, 2008(10), P10008. <https://doi.org/10.1088/1742-5468/2008/10/P10008>
- BLUM, C. (2024). Ant colony optimization : A bibliometric review. *Physics of Life Reviews*.
- BLUM, C., & ROLI, A. (2003). Metaheuristics in combinatorial optimization : Overview and conceptual comparison. *ACM computing surveys (CSUR)*, 35(3), 268-308.
- CHUANG, I. L. (2002). Quantum computation : a new scientific paradigm. *Physics World*, 15(1), 33-38.
- COMPUTING, R. (2022). *Annonce d'un processeur quantique 128 qubits*. <https://www.rigetti.com>
- CONSTANTINESCU, E. M. (2020). *An in-depth approach to fitting probability distributions using evolutionary algorithms* [B.S. thesis]. University of Twente.
- CREEVEY, E., STANFIELD, R., WHITE, D. R., RANGAMANI, A., ABBASLOO, E., TAVAKKOLI, A., & BRADLEY, M. J. (2023). GASP : A Genetic Algorithm for State Preparation on Quantum Computers. *Scientific Reports*, 13(1), 10898. <https://doi.org/10.1038/s41598-023-37767-w>
- DAVIS, M. G., SMITH, E., TUDOR, A., SEN, K., SIDDIQI, I., & IANCU, C. (2020). Towards Optimal Topology Aware Quantum Circuit Synthesis. *IEEE Internatio-*

- nal Conference on Quantum Computing and Engineering (QCE)*, 306-317. <https://doi.org/10.1109/QCE49297.2020.00036>
- de ANDOIN, M. G., & ECHANOBÉ, J. (2022). Implementable hybrid quantum ant colony optimization algorithm. *Quantum Machine Intelligence*, 4(2), 12.
- DEB, K. (1999). Multi-objective genetic algorithms : Problem difficulties and construction of test problems. *Evolutionary computation*, 7(3), 205-230.
- DEB, K., PRATAP, A., AGARWAL, S., & MEYARIVAN, T. (2002). A fast and elitist multiobjective genetic algorithm : NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2), 182-197. <https://doi.org/10.1109/4235.996017>
- DEUTSCH, D. (1985). Quantum theory, the Church–Turing principle and the universal quantum computer. *Proceedings of the Royal Society of London. A. Mathematical and Physical Sciences*, 400(1818), 97-117. <https://doi.org/10.1098/rspa.1985.0070>
- DEUTSCH, D., & JOZSA, R. (1992). Rapid solution of problems by quantum computation. *Proceedings of the Royal Society A*, 439(1907), 553-558.
- DEVITT, S. J. (2016). Performing quantum computing experiments in the cloud. *Physical Review A*, 94(3), 032329. <https://doi.org/10.1103/PhysRevA.94.032329>
- DING, L., & SPECTOR, L. (2023). Multi-Objective Evolutionary Architecture Search for Parameterized Quantum Circuits. *Entropy*, 25(1), 93. <https://doi.org/10.3390/e25010093>
- DORIGO, M., & SOCHA, K. (2018, mai). An Introduction to Ant Colony Optimization. In T. F. GONZALEZ (Éd.), *Handbook of Approximation Algorithms and Metaheuristics, Second Edition* (1^{re} éd., p. 395-408). Chapman ; Hall/CRC. <https://doi.org/10.1201/9781351236423-23>
- DUMITRESCU, E. (2004). *Quantum Circuit Design : A Genetic Programming Approach* [mém. de mast., University of Lethbridge] [Thèse de maîtrise]. https://www.collectionscanada.gc.ca/obj/thesescanada/vol2/001/mr01267.pdf?is_thesis=1&oclc_number=62894982
- EINSTEIN, A., PODOLSKY, B., & ROSEN, N. (1935). Can Quantum-Mechanical Description of Physical Reality Be Considered Complete? *Physical Review*, 47, 777-780.
- et AL., K. M. S. (2018). Q# : Enabling Scalable Quantum Computing and Development with a High-level DSL. *Proceedings of the Real World Domain Specific Languages Workshop 2018*.
- Evolutionary Algorithms in Engineering Design Optimization*. (2022, mars). MDPI. <https://doi.org/10.3390/books978-3-0365-2715-4>
- EZRATTY, O. (2019). *Understanding Quantum Technologies* [Édition mise à jour en continu]. Self-published. <https://www.xn--quantum-bibliotheque-5yb.com>
- FARHI, E., GOLDSTONE, J., & GUTMANN, S. (2014). A Quantum Approximate Optimization Algorithm. *arXiv preprint*.
- FOWLER, A. G., MARIANTONI, M., MARTINIS, J. M., & CLELAND, A. N. (2012). Surface codes : Towards practical large-scale quantum computation. *Physical Review A*, 86, 032324.
- GAMBS, S. (2008). Introduction to Quantum Computing : A Survey for Computer Scientists. *ACM Computing Surveys*, 40(3), 1-43.
- GERSHENFELD, N. A., & CHUANG, I. L. (1997). Bulk spin-resonance quantum computation. *Science*, 275(5298), 350-356. <https://doi.org/10.1126/science.275.5298.350>
- GIRALDI, G. A., PORTUGAL, R., & THESS, R. N. (2004). Genetic Algorithms and Quantum Computation. *arXiv preprint cs/0403003*. <https://arxiv.org/abs/cs/0403003v1>

- GOLDBERG, D. E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley.
- GOLDBERG, D. E., & LINGLE, R. (1985). Alleles, loci, and the traveling salesman problem. *Proceedings of the First International Conference on Genetic Algorithms*.
- GONDRAN, M., & GONDRAN, A. (2020). *Introduction à l'informatique quantique*. Ellipses.
- GROVER, L. K. (1996). A fast quantum mechanical algorithm for database search, 212-219. <https://doi.org/10.1145/237814.237866>
- HOLAND, J. H. (1975). Adaptation in natural and artificial systems. *Ann Arbor : The University of Michigan Press*, 32.
- HOLLAND, J. H. (1984). Genetic algorithms and adaptation. *Adaptive control of ill-defined systems*, 317-333.
- IMANI, V., SEVILLA-SALCEDO, C., MORADI, E., FORTINO, V., TOHKA, J., INITIATIVE, A. D. N., et al. (2024). Multi-objective genetic algorithm for multi-view feature selection. *Applied Soft Computing*, 167, 112332.
- JAFFALI, F. (2020). Cours d'introduction à l'informatique quantique [Université de Tunis]. <https://moodle.isg.rnu.tn>
- JHA, R. K. (2023). From quantum computing to quantum-inspired computation for neuromorphic advancement—a survey. *Authorea Preprints*.
- KARYPIS, G., & KUMAR, V. (1998). A fast and high quality multilevel scheme for partitioning irregular graphs. *Proceedings of the International Conference on Parallel Processing*, 113-122. <https://doi.org/10.1109/IPPS.1998.669755>
- KATOCH, S., CHAUHAN, S. S., & KUMAR, V. (2021). A review on genetic algorithm : past, present, and future. *Multimedia tools and applications*, 80, 8091-8126.
- KERNIGHAN, B. W., & LIN, S. (1970). An efficient heuristic procedure for partitioning graphs. *Bell System Technical Journal*, 49(2), 291-307.
- KOŁODZIEJ, M. (2021). Qubit Toaster : A Fast Quantum Circuit Simulator in C++.
- KRYLOV, L., & LUKAC, M. (2018). Quantum Encoded Quantum Evolutionary Algorithm for the Design of Quantum Circuits. *Proceedings of the 2018 International Conference on Computing and Network Communications (CoCoNet)*, 90-96. <https://doi.org/10.1109/CoCoNet.2018.8510727>
- KUMAR, S., & PATHAK, S. (2019). A comprehensive review on swarm intelligence & particle swarm optimization. *Indian Journal of Scientific Research*, 18(2), 288-294.
- LEE, S., LEE, S.-J., KIM, T., LEE, J.-S., BIAMONTE, J., & PERKOWSKI, M. (2006). The Cost of Quantum Gate Primitives. *Journal of Multiple-Valued Logic & Soft Computing*, 12.
- LUKAC, M., & PERKOWSKI, M. (2002). Evolving quantum circuits using genetic algorithm. *Proceedings 2002 NASA/DoD Conference on Evolvable Hardware*, 177-185.
- MIRANDA, F. T., BALBI, P. P., & COSTA, P. C. (2023). Synthesis of Quantum Circuits with an Island Genetic Algorithm. *arXiv preprint arXiv :2106.03115*. <https://arxiv.org/abs/2106.03115>
- MITCHELL, M. (1998). *An Introduction to Genetic Algorithms*. MIT press.
- MOORE, G. E. (1975). Progress in Digital Integrated Electronics. *Proceedings of the IEEE International Electron Devices Meeting (IEDM)*, 11-13.
- NAKAMURA, Y., PASHKIN, Y. A., & TSAI, J. S. (1999). Coherent control of macroscopic quantum states in a single-Cooper-pair box. *Nature*, 398, 786-788. <https://doi.org/10.1038/19718>

- NATIONAL ACADEMIES OF SCIENCES, ENGINEERING, AND MEDICINE. (2019). *Quantum Computing : Progress and Prospects*. National Academies Press. <https://doi.org/10.17226/25196>
- NIELSEN, M. A. (2003). Implementation of a Quantum Controlled-NOT Gate with Trapped Ions. *Nature*, 422, 412-415.
- NIELSEN, M. A., & CHUANG, I. L. (2000). *Quantum Computation and Quantum Information*. Cambridge University Press.
- PEDNAULT, E., GUNNELS, J., NANNICINI, G., et al. (2017). Breaking the 49-qubit barrier in the simulation of quantum circuits. *arXiv preprint*. <https://arxiv.org/abs/1710.05867>
- PERUZZO, A., MCCLEAN, J., SHADBOLT, P., YUNG, M.-H., ZHOU, X.-Q., LOVE, P. J., ASPURU-GUZZI, A., & O'BRIEN, J. L. (2014). A variational eigenvalue solver on a photonic quantum processor. *Nature Communications*, 5, 4213. <https://doi.org/10.1038/ncomms5213>
- POTOČEK, V., REYNOLDS, A. P., FEDRIZZI, A., & CORNE, D. W. (2018). Multi-objective evolutionary algorithms for quantum circuit discovery. *arXiv preprint arXiv :1812.04458*. <https://arxiv.org/abs/1812.04458>
- PRESKILL, J. (1998). Reliable quantum computers. *Proceedings of the Royal Society of London. Series A : Mathematical, Physical and Engineering Sciences*, 454(1969), 385-410.
- RAHMAN, M. S. I., ZHANG, S., HAQUE, M. R., & SHATABDA, S. (2025). Quantum Circuit Synthesis Using Fuzzy-Logic-Assisted Genetic Algorithms. *Algorithms*, 18(4), 178. <https://doi.org/10.3390/a18040178>
- RAZALI, N. M., GERAGHTY, J., et al. (2011). Genetic algorithm performance with different selection strategies in solving TSP. *Proceedings of the world congress on engineering*, 2(1), 1-6.
- RESEARCH, I. (2016). *IBM Quantum Experience : premier processeur supraconducteur 5 qubits*. <https://research.ibm.com/ibm-q>
- RESEARCH, I. (2021). *IBM Eagle : processeur supraconducteur 127 qubits*. <https://www.ibm.com/quantum/blog/127-qubit-quantum-processor-eagle>
- RICHARD & FEYNMAN. (1982). Simulating physics with computers. *International Journal of Theoretical Physics*, 21(6-7), 467-488. <https://doi.org/10.1007/BF02650179>
- RINDELL, T., YENILEN, B., HALONEN, N., PÖNNI, A., TITTONEN, I., & RAASAKKA, M. (2023). Exploring the optimality of approximate state preparation quantum circuits with a genetic algorithm. *arXiv preprint*. <https://arxiv.org/abs/2210.06411>
- SCHRÖDINGER, E. (1935). Die gegenwärtige Situation in der Quantenmechanik [Traduction anglaise : *The present situation in quantum mechanics*]. *Naturwissenschaften*, 23, 807-812, 823-828, 844-849.
- SHAMI, T. M., EL-SALEH, A. A., ALSWAITTI, M., AL-TASHI, Q., SUMMAKIEH, M. A., & MIRJALILI, S. (2022). Particle Swarm Optimization : A Comprehensive Survey. *IEEE Access*, 10, 10031-10062. <https://doi.org/10.1109/ACCESS.2022.3142859>
- SHENDE, V. V., BULLOCK, S. S., & MARKOV, I. L. (2006). Synthesis of Quantum Logic Circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 25(6), 1000-1010. <https://doi.org/10.1109/TCAD.2005.855930>
- SHOR, P. W. (1994). Algorithms for quantum computation : discrete logarithms and factoring. *Proceedings 35th Annual Symposium on Foundations of Computer Science*, 124-134.

- SHOR, P. W. (1995). Scheme for reducing decoherence in quantum computer memory. *Physical Review A*, 52(4), R2493-R2496.
- SMITH, R. S., CURTIS, M. J., & ZENG, W. J. (2016). A Practical Quantum Instruction Set Architecture. *Proceedings of the 2016 ACM Conference on Programming Language Design and Implementation (PLDI)*.
- STEANE, A. M. (1996). Error Correcting Codes in Quantum Theory. *Physical Review Letters*, 77, 793-797.
- STEIGER, D. S., HÄNER, T., & TROYER, M. (2018). ProjectQ : An Open Source Software Framework for Quantum Computing. *Quantum*, 2, 49. <https://doi.org/10.22331/q-2018-01-31-49>
- SÜNKEL, L., MARTYNIUK, D., MATTERN, D., JUNG, J., & PASCHKE, A. (2023). Ga4qco : genetic algorithm for quantum circuit optimization. *arXiv preprint arXiv :2302.01303*.
- SYSTEMS, D.-W. (2019). *Quantum Annealing for Combinatorial Optimization*. <https://www.dwavesys.com>
- SYSWERDA, G. (1989). Uniform crossover in genetic algorithms. *Proceedings of the 3rd International Conference on Genetic Algorithms*, 2-9.
- TALBI, E.-G. (2009). *Metaheuristics : from design to implementation*. John Wiley & Sons.
- TANDEITNIK, D., & GUERREIRO, T. (2022). Evolving Quantum Circuits. *arXiv preprint arXiv :2210.05058*. <https://arxiv.org/abs/2210.05058>
- TEAM, Q. D. (2023). Qiskit : An Open-source Framework for Quantum Computing.
- TERHAL, B. M. (2015). Quantum error correction for quantum memories. *Reviews of Modern Physics*, 87, 307-346.
- THEIS, T. N., & WONG, H.-S. P. (2017). The end of Moore's Law : A new beginning for information technology. *Computing in Science & Engineering*, 19(2), 41-50. <https://doi.org/10.1109/MCSE.2017.29>
- UMEANO, C. (2021). *Optimisation of Quantum Circuits Using Evolutionary Algorithms* [MSc Dissertation]. Imperial College London. <https://www.imperial.ac.uk/media/imperial-college/research-centres-and-groups/theoretical-physics/msc/dissertations/2021/Chukwudubem-Umeano-Dissertation.pdf>
- VERT, T. (2021). Introduction au calcul quantique. <https://thomasvert.fr/quantum>
- WU, X.-C., DAVIS, M. G., CHONG, F. T., & IANCU, C. (2020). QGo : Scalable quantum circuit optimization using automated synthesis. *arXiv preprint arXiv :2012.09835*.
- YABUKI, T., & IBA, H. (2000). Genetic algorithms for quantum circuit design-evolving a simpler teleportation circuit. *Late Breaking Papers at the 2000 Genetic and Evolutionary Computation Conference*, 421-425.
- YEPEZ, J. (2013). Lecture notes : Qubit representations and rotations [Department of Physics and Astronomy, University of Hawai'i at Manoa, Lecture 1, January 9, 2013].
- YOUNG, T. (1804). The Bakerian Lecture : Experiments and Calculations relative to Physical Optics. *Philosophical Transactions of the Royal Society of London*, 94, 1-16.
- ZHONG, J., HU, X., ZHANG, J., & GU, M. (2005). Comparison of performance between different selection strategies on simple genetic algorithms. *International conference on computational intelligence for modelling, control and automation and international conference on intelligent agents, web technologies and internet commerce (CIMCA-IAWTIC'06)*, 2, 1115-1121.

Annexes

Annexe A

Notions mathématiques

A.1 Vecteur Ket

Selon la notation de Dirac, la représentation d'un état quantique ψ est donnée par le vecteur ket $|\psi\rangle$.

Exemple : Soit un qubit avec une amplitude de probabilité complexe :

$$|\psi\rangle = \frac{1}{2}|0\rangle + i\frac{1}{2}|1\rangle$$

alors sous forme de vecteur colonne :

$$|\psi\rangle = \begin{bmatrix} \frac{1}{2} \\ \frac{i}{2} \end{bmatrix} \quad (\text{A.1})$$

A.2 Vecteur Bra

La transposée conjuguée complexe d'un vecteur Ket est appelée vecteur Bra, noté $\langle\psi|$.

Exemple : Le Bra correspondant à l'équation (A.1) est :

$$\langle\psi| = \begin{bmatrix} \frac{1}{2} & -\frac{i}{2} \end{bmatrix} \quad (\text{A.2})$$

De manière générale, pour un nombre complexe $(a + ib)$, son conjugué est $(a - ib)$. Si les amplitudes d'un ket sont réelles, le bra correspondant est simplement sa transposée.

A.3 Produit tensoriel

Le produit tensoriel agit de la manière suivante :

$$(A \otimes B) (|v\rangle \otimes |w\rangle) = (A|v\rangle) \otimes (B|w\rangle) \quad (\text{A.3})$$

Soient deux vecteurs $|v\rangle \in \mathbb{R}^m$ et $|w\rangle \in \mathbb{R}^n$, leur produit tensoriel est :

$$|v\rangle \otimes |w\rangle = \begin{bmatrix} v_1|w\rangle \\ v_2|w\rangle \\ \vdots \\ v_m|w\rangle \end{bmatrix} \quad (\text{A.4})$$

Propriétés utiles :

— La matrice conjuguée d'une matrice $A \otimes B$ à coefficients complexes est donnée par :

$$(A \otimes B)^* = A^* \otimes B^*.$$

— La transposée de la matrice $A \otimes B$ est donnée par :

$$(A \otimes B)^T = A^T \otimes B^T.$$

— La transposée de la matrice conjuguée d'une matrice complexe $A \otimes B$ représente la matrice adjointe, donnée par :

$$(A \otimes B)^\dagger = A^\dagger \otimes B^\dagger.$$

De même, pour deux matrices $A \in \mathbb{R}^{m \times n}$ et $B \in \mathbb{R}^{p \times q}$:

$$A \otimes B = \begin{bmatrix} a_{11}B & \cdots & a_{1n}B \\ \vdots & \ddots & \vdots \\ a_{m1}B & \cdots & a_{mn}B \end{bmatrix} \quad (\text{A.5})$$

Exemple (Pauli X et Z) :

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

Alors :

$$X \otimes Z = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \\ 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \end{bmatrix}$$

A.4 Opérateur hermitien

Un opérateur A est dit **hermitien** (ou **auto-adjoint**) s'il vérifie $A = A^\dagger$, où $A^\dagger$ désigne le conjugué transposé (adjoint) de A .

Propriétés :

- Les valeurs propres d'un opérateur hermitien sont **réelles**.
- Les vecteurs propres associés à des valeurs propres **distinctes** sont **orthogonaux** entre eux (en cas de dégénérescence, on peut choisir une base orthonormée de l'espace propre).

Exemple : la matrice suivante est hermitienne car $A^\dagger = A$:

$$A = \begin{bmatrix} 1 & 3 - 4i \\ 3 + 4i & 2 \end{bmatrix}, \quad A^\dagger = \begin{bmatrix} 1 & 3 - 4i \\ 3 + 4i & 2 \end{bmatrix} = A.$$

A.5 Hamiltonien

En mécanique quantique, à chaque grandeur mesurable est associé un opérateur ; l'opérateur associé à l'**énergie** totale est le **Hamiltonien** $\hat{H}$. Pour une particule de masse m en une dimension soumise à un potentiel $V(x)$:

$$\hat{H} = \frac{\hat{p}^2}{2m} + V(\hat{x}), \quad \hat{p} = -i\hbar \frac{d}{dx}.$$

Ainsi,

$$\hat{H} = -\frac{\hbar^2}{2m} \frac{d^2}{dx^2} + V(x). \quad (\text{A.6})$$

Où : $-\frac{\hbar^2}{2m} \frac{d^2}{dx^2}$ représente l'opérateur associé à l'**énergie cinétique** de la particule ,et $V(x)$ représente l'**énergie potentielle** due au champ ou au potentiel appliqué.

A.6 Matrice unitaire et adjointe

Une matrice U est dite **unitaire** si :

$$U^\dagger U = U U^\dagger = I \quad (\text{A.7})$$

où $U^\dagger$ désigne le **conjugué transposé** (ou adjoint) de U .

Exemple : La matrice de Pauli X est :

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

Son conjugué transposé est identique :

$$X^\dagger = X^T = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

On vérifie :

$$X^\dagger X = X X^\dagger = I$$

Ainsi, X est bien une matrice unitaire.

Exemple 2 : Soit un circuit agissant sur deux qubits q_0 (haut) et q_1 (bas) : on applique d'abord X sur q_0 , puis un CNOT avec q_0 comme contrôle et q_1 comme cible, et enfin une rotation $R_z(\theta)$ sur q_1 .

L'unitaire global du circuit est alors :

$$U = (I \otimes R_z(\theta)) \text{CNOT}_{0 \rightarrow 1} (X \otimes I).$$

Matrices élémentaires.

$$X \otimes I = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}, \quad \text{CNOT}_{0 \rightarrow 1} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix},$$

et

$$R_z(\theta) = \begin{bmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{bmatrix}, \quad I \otimes R_z(\theta) = \begin{bmatrix} e^{-i\theta/2} & 0 & 0 & 0 \\ 0 & e^{i\theta/2} & 0 & 0 \\ 0 & 0 & e^{-i\theta/2} & 0 \\ 0 & 0 & 0 & e^{i\theta/2} \end{bmatrix}.$$

Matrice unitaire du circuit. En multipliant :

$$U = (I \otimes R_z(\theta)) \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}.$$

Soit :

$$U = \begin{bmatrix} 0 & 0 & e^{-i\theta/2} & 0 \\ 0 & 0 & 0 & e^{i\theta/2} \\ 0 & e^{-i\theta/2} & 0 & 0 \\ e^{i\theta/2} & 0 & 0 & 0 \end{bmatrix}.$$

Conjugué transposé.

$$U^\dagger = \begin{bmatrix} 0 & 0 & 0 & e^{-i\theta/2} \\ 0 & 0 & e^{i\theta/2} & 0 \\ e^{i\theta/2} & 0 & 0 & 0 \\ 0 & e^{-i\theta/2} & 0 & 0 \end{bmatrix}.$$

Vérification de l'unitarité. On vérifie bien que :

$$U^\dagger U = U U^\dagger = I.$$

Évolution d'état (exemple). À partir de $|00\rangle$:

$$|00\rangle \xrightarrow{X \otimes I} |10\rangle \xrightarrow{\text{CNOT}_{0 \rightarrow 1}} |11\rangle \xrightarrow{I \otimes R_z(\theta)} e^{i\theta/2} |11\rangle.$$

Annexe B

Notions de graphe

Dans cette partie, nous détaillons les fondements mathématiques utilisés pour le partitionnement du graphe d'interaction associé à un circuit quantique.

B.1 Notations de base

Un graphe non orienté et pondéré est défini par $G = (V, E)$ où V est l'ensemble des sommets et E l'ensemble des arêtes. Chaque arête (i, j) est associée à un poids w_{ij} représentant l'intensité de la relation entre i et j .

On note la matrice d'adjacence $A = (A_{ij})$ avec :

$$A_{ij} = \begin{cases} w_{ij} & \text{si une arête relie } i \text{ et } j, \\ 0 & \text{sinon.} \end{cases} \quad (\text{B.1})$$

Quelques notations classiques :

- $k_i = \sum_j A_{ij}$: degré pondéré du sommet i .
- $m = \frac{1}{2} \sum_{i,j} A_{ij}$: poids total des arêtes.
- $G[Q]$: sous-graphe induit par un ensemble $Q \subseteq V$.

B.2 Le partitionnement

Le partitionnement d'un graphet d'un graphe consiste à découper V en sous-ensembles de sommets, de manière à minimiser le nombre (ou le poids) des arêtes coupées entre les sous-ensembles, tout en maximisant la densité interne de connexions. Cette opération est utilisée dans plusieurs contextes tels que l'optimisation, l'analyse de réseaux et la modélisation de systèmes complexes, car elle permet de mettre en évidence la structure modulaire du graphe.

Dans ce qui suit, nous présentons quelques techniques classiques de partitionnement de graphes, chacune reposant sur des principes différents mais visant le même objectif : obtenir une division équilibrée tout en minimisant les arêtes coupées.

B.2.1 Détection de communautés par Louvain

L'algorithme de Louvain (BLONDEL et al., 2008) repose sur la maximisation de la **modularité**, une mesure de la qualité d'une partition en communautés. Pour un graphe $G = (V, E)$ avec matrice d'adjacence A , la modularité est définie par :

$$\mathcal{Q} = \frac{1}{2m} \sum_{i,j} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \delta(c_i, c_j) \quad (\text{B.2})$$

où c_i désigne la communauté du sommet i , k_i le degré pondéré de i , et m le poids total des arêtes.

L'algorithme progresse de manière itérative en alternant deux phases. Dans un premier temps, chaque sommet est déplacé dans la communauté de son voisin qui procure le plus grand gain de modularité. Le gain correspondant au déplacement d'un sommet i vers une communauté C s'écrit :

$$\Delta Q_{i \rightarrow C} = \frac{1}{2m} \left(2 \sum_{j \in C} A_{ij} - \frac{2 k_i K_C}{2m} \right) \quad (\text{B.3})$$

où $K_C = \sum_{u \in C} k_u$ est le degré total des sommets de C . Lorsque plus aucun déplacement n'améliore la modularité, on passe à la seconde phase : les communautés obtenues sont agrégées en super-sommets, formant un graphe réduit sur lequel on réapplique le même principe.

Le processus se répète jusqu'à ce qu'aucune amélioration supplémentaire ne soit observée. En pratique, l'algorithme est très efficace et son coût de calcul croît presque linéairement avec le nombre d'arêtes, ce qui le rend adapté aux graphes de grande taille et souvent clairsemés. L'algorithme correspondant est présenté ci-dessous.

Algorithm 16: LOUVAINPARTITION($G = (V, E)$)

Input: Un graphe non orienté et pondéré $G = (V, E)$ avec matrice d'adjacence A .

Output: Une partition $\mathcal{C} = \{C_1, \dots, C_k\}$ des sommets.

Initialisation : chaque sommet $i \in V$ est sa propre communauté $c_i \leftarrow \{i\}$.

repeat

 // Phase 1 : optimisation locale

foreach sommets $i \in V$ **do**

foreach communauté C' voisine de i **do**

 Calculer le gain de modularité

$$\Delta Q_{i \rightarrow C'} = \left[\sum_{j \in C'} A_{ij} - \frac{k_i K_{C'}}{2m} \right] - \left[\sum_{j \in c_i} A_{ij} - \frac{k_i K_{c_i}}{2m} \right]$$

if $\Delta Q_{i \rightarrow C'} > 0$ **then**

 déplacer i dans C'

end

end

end

 // Phase 2 : agrégation des communautés

 Construire un graphe réduit G' où chaque communauté devient un super-sommet.

until aucune amélioration de la modularité

return $\mathcal{C}$

B.2.2 Bisection équilibrée par METIS

L'algorithme METIS (KARYPIS & KUMAR, 1998) cherche à diviser un ensemble de sommets Q en deux sous-ensembles (Q_1, Q_2) de manière à **minimiser la coupe** entre

eux, définie par :

$$\text{Cut}(Q_1, Q_2) = \sum_{\substack{i \in Q_1 \\ j \in Q_2}} w_{ij}. \quad (\text{B.4})$$

Cette partition ne doit pas seulement réduire le nombre d'arêtes traversant la coupe, mais également respecter certaines contraintes. D'une part, les deux parties doivent être équilibrées, c'est-à-dire que la différence de tailles reste bornée par un facteur proportionnel à $|Q|$:

$$||Q_1| - |Q_2|| \leq \epsilon |Q|.$$

D'autre part, chaque sous-ensemble est limité en taille, avec $|Q_k| \leq k_{\max}$ pour tout k .

La méthode s'appuie sur un *schéma multilevel* particulièrement efficace. Dans un premier temps, le graphe est progressivement réduit par une phase de *coarsening*, qui condense les sommets tout en préservant sa structure globale. Le graphe ainsi simplifié est ensuite partitionné par une bisection, avant de subir une phase d'*uncoarsening* où la partition est projetée sur le graphe original et raffinée localement pour améliorer la qualité de la coupe.

Bien que l'on puisse interpréter la bisection équilibrée à travers une relaxation spectrale, en particulier via la seconde valeur propre du Laplacien, METIS repose avant tout sur un heuristique multilevel robuste, ce qui en fait un outil rapide et efficace pour traiter de grands graphes. L'algorithme correspondant est présenté ci-dessous.

Algorithm 17: APPLYMETIS($G[Q]$)

Input: Sous-graphe $G[Q]$ avec poids w_{ij} .

Output: Deux sous-ensembles (Q_1, Q_2) formant une bisection équilibrée.

// Phase de *coarsening*

Construire une suite de graphes réduits $G_0, G_1, \dots, G_k$ en contractant
itérativement des paires de sommets fortement connectés.

// Partition sur le graphe réduit

Sur le graphe le plus petit G_k , trouver une partition (Q_1, Q_2) minimisant

$$\text{Cut}(Q_1, Q_2) = \sum_{\substack{i \in Q_1 \\ j \in Q_2}} w_{ij}.$$

// Phase d'*uncoarsening* et raffinement

Projeter la partition obtenue sur chaque niveau $G_{k-1}, G_{k-2}, \dots, G_0$, en raffinant
localement pour améliorer la qualité de la coupe tout en respectant l'équilibre :

$$||Q_1| - |Q_2|| \leq \epsilon |Q|.$$

return (Q_1, Q_2)

B.2.3 Raffinement Kernighan–Lin (KL)

L'algorithme de Kernighan–Lin (KERNIGHAN & LIN, 1970) part d'une bisection initiale (Q_1, Q_2) et cherche à améliorer la qualité de la partition par une série d'échanges successifs entre sommets. L'idée est de sélectionner des paires (u, v) , avec $u \in Q_1$ et $v \in Q_2$, dont l'échange permet de réduire la coupe.

Pour guider ce processus, chaque sommet est associé à une fonction de gain. On définit pour $u \in Q_1$:

$$D(u) = \sum_{x \in Q_2} w_{ux} - \sum_{x \in Q_1} w_{ux},$$

et pour $v \in Q_2$:

$$D(v) = \sum_{x \in Q_1} w_{vx} - \sum_{x \in Q_2} w_{vx}.$$

Le gain associé à l'échange de (u, v) s'écrit alors :

$$\text{gain}(u, v) = D(u) + D(v) - 2w_{uv}. \quad (\text{B.5})$$

À chaque itération, l'algorithme sélectionne la meilleure paire disponible et construit une séquence d'échanges en accumulant les gains. Une fois cette séquence formée, on applique le préfixe qui maximise l'amélioration globale de la coupe, garantissant ainsi que chaque raffinement reste bénéfique.

Le processus s'arrête dès qu'aucun échange positif ne peut être trouvé, que l'amélioration cumulée devient nulle, ou encore lorsqu'un nombre maximal d'itérations est atteint. Grâce à cette stratégie itérative, Kernighan–Lin permet d'affiner efficacement une bisection donnée, en réduisant progressivement les arêtes traversant la coupe.

L'algorithme correspondant est présenté ci-dessous.

Algorithm 18: KERNIGHANLINREFINEMENT($G[Q], Q_1, Q_2$)

Input: Sous-graphe $G[Q]$ et partition initiale (Q_1, Q_2) .

Output: Partition affinée (Q_1, Q_2) .

repeat

foreach $u \in Q_1$ **do**

$D(u) \leftarrow \sum_{x \in Q_2} w_{ux} - \sum_{x \in Q_1} w_{ux}$

end

foreach $v \in Q_2$ **do**

$D(v) \leftarrow \sum_{x \in Q_1} w_{vx} - \sum_{x \in Q_2} w_{vx}$

end

 Choisir (u, v) maximisant

$$\text{gain}(u, v) = D(u) + D(v) - 2w_{uv}$$

if $\text{gain}(u, v) > 0$ **then**

 échanger u et v

end

else

break

end

until aucun échange bénéfique

return (Q_1, Q_2)
